DM545/DM871
Linear and Integer Programming

Slides: Full Deck

Marco Chiarandini

Department of Mathematics and Computer Science

7. november 2026

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Who are we?

47 in total registered in ItsLearning

DM545 (5 ECTS)
33 enrolled

- Math-economics
  (Bachelor, 5th semester)
  Section M1
  6 enrolled
- Others?
  Section H1
  27 enrolled

DM871 (5 ECTS)
13 enrolled

- Section H1, 13 enrolled
  - Computer Science
    (Master)
  - Mathematics
    (Bachelor/Master)
  - Applied Mathematics
    (Bachelor/Master)
  - Data Science (Master)
  - Others?

Aims of the course

Learn about **mathematical optimization**:

- linear programming (continuous/numerical linear optimization)
- integer linear programming (discrete/combinatorial linear optimization)

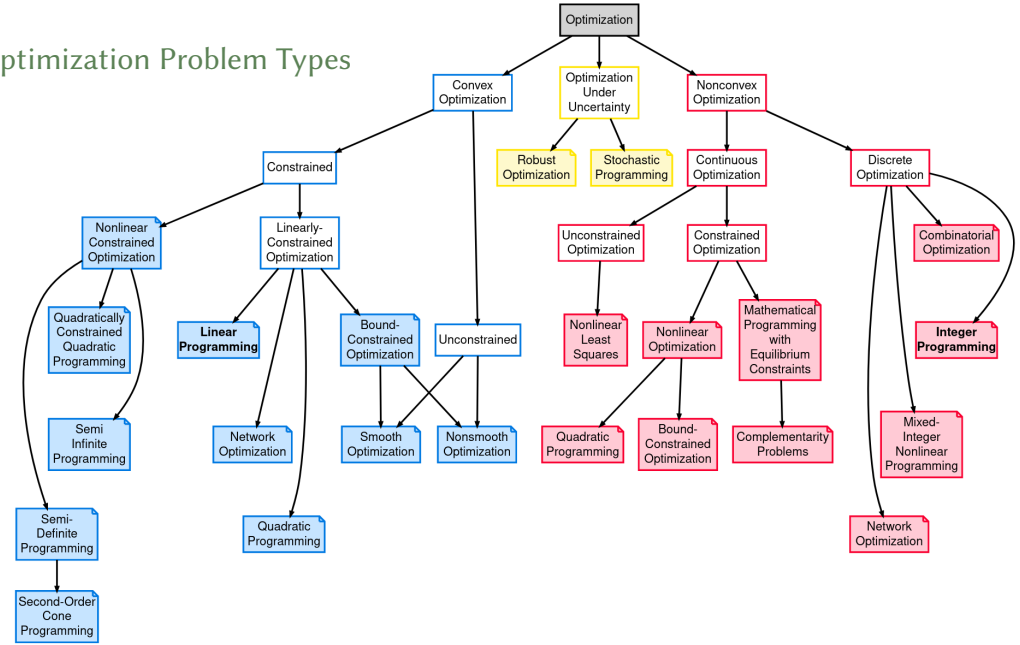
Hence, an alternative course title could be: **Linear Optimization**

Optimization is an important tool in **decision making** and in analyzing physical systems.

In mathematical terms, an **optimization problem** is the problem of finding the **best solution** from the set of all **feasible solutions**.

The first step in the optimization process is constructing an appropriate **mathematical model**

Optimization Problem Types



Contents of the Course (aka Syllabus)

Linear Programming

1 Introduction - Linear Programming, Notation & Modeling
2 Linear Programming, Simplex Method
3 Exception Handling
4 Duality Theory
5 Sensitivity
6 Revised Simplex Method

Integer Linear Programming

7 Modeling Examples, Good Formulations, Relaxations
8 Well Solved Problems
9 Cutting Planes & Branch and Bound
10 Network Optimization Models (Max Flow, Min cost flow, Matching)
11 More on Modeling

Practical Information

Teacher: Marco Chiarandini (imada.sdu.dk/u/marco/)
Instructor: (Marco Chiarandini ?)

Schedule, alternative views:

- mitsdu.sdu.dk, SDU Mobile
- Official course description (læserplanen)
- ItsLearning

Schedule (15 weeks, up to week 51):

- Introductory classes: 28 hours (14 classes), scheduled 32
- Training classes: 14 hours (7 classes), scheduled 28

Communication Means

- Announcements in ItsLearning
- Ask peers and teacher in class
- ⇔ External Web Page
  (link `https://dm871.github.io`)
- Write to Marco (`marco@imada.sdu.dk`) for confidential and individual issues
- Feedback from classes
- Midway evaluation

⇝ **It is good to ask questions!!**

⇝ **Let me know if you think we should do things differently!**

Sources — Reading Material

Main references:

[LN] Lecture Notes (continously updated)



[F] M. Fischetti, **Introduction to Mathematical Optimization**, Independently published, 2019

Linear Programming:



[VA] R. Vanderbei. Linear Programming: Foundations and Extensions. Springer US, 2008

Integer Programming:



[Wo] L.A. Wolsey. Integer programming. John Wiley & Sons, New York, USA, 1998

Others

[HL] Frederick S Hillier and Gerald J Lieberman, Introduction to Operations Research, 9th edition, 2010

Course Material

External Web Page is the main reference for list of contents

It contains:

- List of units with topics and references
- Slides
- Exercises and some solutions
- Links
- Tutorials for programming tasks

Assessment

- One **3 h Written Exam**
  January 11, 2027
  all helping tools allowed, but not GenAI
  - style: short open answers about calculations and modeling
  - (language: Danish and/or English)

- (only DM871) Home implementation assignment in the second half of the course

Graded with internal censorship
you will get the solutions right after but you will not get feedback. The exam is not the place for formative feedback.

Training Sessions

- Do the plus exercises in advance
- Prepare the starred exercises in advance to get out the most from the session
- Participate actively in class
- Try the others, unsolved in class, after the session
- Best if carried out in small groups
- (Starred) Exercises are examples of exam questions

Exercise classes are about material presented the same week.

Prerequisites: Linear Algebra

Linear Algebra:
manipulation of matrices and vectors with some theoretical background

Linear Algebra

- Matrices and vectors - Matrix algebra
- Dot (scalar, Euclidean inner) product
- Geometric insights
- Systems of Linear Equations - Row echelon form, Gaussian elimination
- Matrix inversion and determinants
- Rank and linear dependency

Unit 0: sheet0.pdf

Prerequisites: Coding

- gives you the ability to create new and useful artifacts
- allows you to have more control of your world as more and more of it becomes digital
- is just fun.

It can also help you to **understand math**.

- listening to lectures
- watching an instructor work through a derivation
- working through numerical examples by hand
- learn **by doing, interacting with Python**.
- Python 3.14+, numpy
- ipython, jupyter, jupyterLab (= interactive python) or **Visual Code** or Spyder3.
- Commercial software: Gurobi, Cplex, Xpress $\approx$ 100 000 Dkk
  Open source: Highs, SCIP
- Python APIs: gurobipy, Pyomo, Python-MIP, PySCIPOpt (Python interfaces to SCIP Optimization Suite)

Unit 0: `sheet0_programming.pdf`

A Note on Generative AI

- Read SDU rules on the topic https://mitsdu.dk/en/aiatsdu. They apply here.
  In short: give credit when using
- Positive aspects:
  better explanations, examples, immediate feedback
- Negative aspects:
  economical costs, impact on environment, sovereignty risk, the social aspect, personal skills
- First learn, then use, and always check
  Comapnies recommend to learn to use but want experts who can verify the outcomes

Use it responsibly, to enhance your learning!
Be aware that the exam form requires you to have achieved the learning goals not GenAI.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Sets

- A **set** is a collection of objects. eg.: $A = \{1, 2, 3\}$
- $A = \{n \mid n \text{ is a whole number and } 1 \leq n \leq 3\}$
  ('|' reads 'such that')
- $B = \{x \mid x \text{ is a student of this course}\}$
- $x \in A$
  x belongs to A
- set of no members: **empty set**, denoted $\emptyset$
- if a set S is a **(proper) subset** of a set T, we write ($S \subset T$) $T \subseteq S$
  $\{1, 2, 5\} \subset \{1, 2, 4, 5, 6, 30\}$
- for two sets A and B, the **union** $A \cup B$ is $\{x \mid x \in A \text{ or } x \in B\}$
- for two sets A and B, the **intersection** $A \cap B$ is $\{x \mid x \in A \text{ and } x \in B\}$
  $A = \{1, 2, 3, 5\}$ and $B = \{2, 4, 5, 7\}$, then $A \cap B = \{2, 5\}$

Numbers

- set of real numbers: $\mathbb{R}$
- set of natural numbers: $\mathbb{N} = \{1, 2, 3, 4, \ldots\}$ (positive integers); $\mathbb{N}_0$ to include zero
- set of all integers: $\mathbb{Z} = \{\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots\}$; $\mathbb{Z}_0^+$ only positives and zero
- set of rational numbers: $\mathbb{Q} = \{p/q \mid p, q \in \mathbb{Z}, q \neq 0\}$
- set of complex numbers: $\mathbb{C}$
- absolute value (non-negative):

$$|a| = \begin{cases} a & \text{if } a \geq 0 \\ -a & \text{if } a \leq 0 \end{cases}$$

- the set $\mathbb{R}^2$ is the set of ordered pairs (x, y) of real numbers
  (eg, coordinates of a point wrt a pair of axes, the **Cartesian plane**)

Matrices and Vectors

- A **matrix** is a rectangular array of numbers or symbols. It can be written as

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

- An $n \times 1$ matrix is a **column vector**, or simply a vector:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

- the set $\mathbb{R}^n$ is the set of vectors $[x_1, x_2, \ldots, x_n]^T$ of real numbers (eg, coordinates of a point wrt an n-dimensional space, the **Euclidean Space**)

Graphs

- An **undirected graph G=(V,E)** is a structure made of a pair of sets: V the set of **vertices** and $E \subseteq \{e = \{u, v\} \mid u, v \in V\}$ the set of **edges** (undirected links)
- A **directed graph (digraph)** $D = (V, A)$ is a pair of sets: V the set of **vertices** and $A \subseteq \{uv = (u, v) \mid u, v \in V\}$ the set of **arcs** (directed links)
- Graph drawing (2D embeddings)
- Labelled graphs and weighted graphs
- Walks, Paths, Cycles, Connectedness
- Problems on graphs: eg, determining intrinsic properties such as shortest st-path
- Matrix representations: **adjacency matrix**, **incidence matrix**

- adjacency set (neighborhood set) for $v \in V$: $N(v) = \{u \mid u \in V, uv \in E\}$; vertex degree (number of incident edges), $\delta(v)$; outdegree (number of outgoing edges), $\delta^+(v)$; indegree (number of incoming edges), $\delta^-(v)$;

- subgraph and **induced subgraph** $G' = G[V] \subseteq G$

- cut: set of edges that separates a connected graphs into two connected components $G[X]$, $G[Y]$: $C = \{xy \mid x \in X, y \in Y\}$

- bipartite graphs $G = (U, V, E)$, trees (= connected acycle graphs)

- multi-graphs, hypergraphs, mixed graphs

- Graphs are the basic subject studied by graph theory. The word "graph" was first used in this sense by J.J. Sylvester in 1878 due to a direct relation between mathematics and chemical structure.

Basic Algebra

Elementary Algebra: the study of symbols and the rules for manipulating symbols. It differs from **arithmetic** in the use of abstractions, such as using letters to stand for numbers that are either unknown or allowed to take on many values

- collecting up terms: eg. $2a + 3b - a + 5b = a + 8b$
- multiplication of variables: eg:

$$a(-b) - 3ab + (-2a)(-4b) = -ab - 3ab + 8ab = 4ab$$

- expansion of bracketed terms: eg:

$$\begin{aligned} -(a - 2b) &= -a + 2b, \\ (2x - 3y)(x + 4y) &= 2x^2 - 3xy + 8xy - 12y^2 \\ &= 2x^2 + 5xy - 12y^2 \end{aligned}$$

- $a^r a^s = a^{r+s}$, $(a^r)^s = a^{rs}$, $a^{-n} = 1/a^n$,
  $a^{m/n} = (a^{1/n})^m$, $a^{1/n} = x \iff x^n = a$, $a^x = n \iff x = \log_a n$

Variables

- In Mathematics and Statistics, a **variable** is an alphabetic character representing a **value**, which is unknown. They are used in **symbolic** calculations.
  Commonly given one-character names.

- in contrast, a **parameter** or **constant** or **given** is a known real number

- in contrast, in **Computer Science**, a **variable** is a storage location paired with an associated identifier, which contains a value that may be known or unknown.
  Commonly given long, explanatory names.

Functions

- a **function** f on a set $\mathcal{X}$ into a set $\mathcal{Y}$ is a rule that assigns a **unique** element $f(x)$ in $\mathcal{Y}$ to each element x in $\mathcal{X}$.

$$y = f(x)$$

y dependent variable — x independent variable

- scalar (element of a field, eg, $\mathbb{R}$, vector of a single component) and vectors

- a **linear function** is a polynomial of degree one or less.
  If only one variable:

  $$f(x) := ax + b$$

  where a and b are constants, often real numbers.
  The graph is a nonvertical line (a is the slope and b the intercept).
  For any finite number of variables: $f(x_1, \ldots, x_k) = b + a_1 x_1 + \ldots + a_k x_k$ and the graph is a hyperplane of dimension k. (actually these are affine functions: linear + translation)

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

What is Operations Research?

Operations Research (aka, Management Science, Analytics):
is the discipline that uses a **scientific approach to decision making**.

It seeks to determine how best to design and operate a system,
usually under conditions requiring the allocation of scarce resources,
by means of **mathematics** and **computer science**.

Quantitative methods for planning and analysis.

It encompasses a wide range of problem-solving techniques and methods applied in the pursuit of improved decision-making and efficiency:

- simulation,
- **mathematical optimization**,
- queueing theory and other stochastic-process models,
- Markov decision processes
- econometric methods,
- data envelopment analysis,
- machine learning,
- expert systems

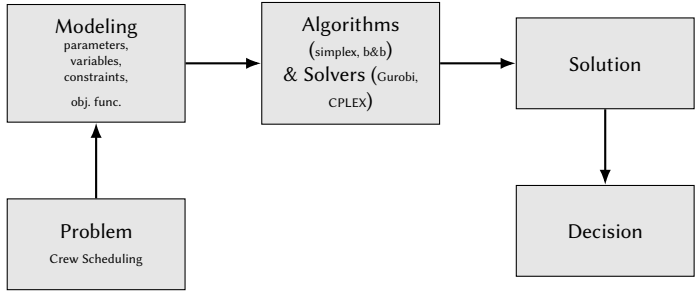
Some Examples ...

- Production Planning and Inventory Control
- Budget Investment
- Blending and Refining
- Manpower Planning
  - Crew Rostering (airline crew, rail crew, nurses)
- Packing Problems
  - Knapsack Problem
- Cutting Problems
  - Cutting Stock Problem
- Routing
  - Vehicle Routing Problem (trucks, planes, trains ...)
- Locational Decisions
  - Facility Location
- Scheduling/Timetabling
  - Examination timetabling/ train timetabling
- .... + many more

Common Characteristics

- Planning decisions must be made
- The problems relate to quantitative issues
  - Cheapest
  - Shortest route
  - Fewest number of people
- Not all plans are feasible - there are constraining rules
  - Limited amount of available resources
- It can be extremely difficult to figure out what to do

OR - The Process?



1. Observe the System
2. Formulate the Problem
3. Formulate Mathematical Model
4. Verify Model
5. Select Alternative
6. Show Results to Company
7. Implementation

Central Idea

Build a mathematical model describing exactly what one wants, and what the “rules of the game” are. However, **what is a mathematical model and how?**

Mathematical Modeling

- Find out exactly what the decision maker needs to know:
  - which investment?
  - which product mix?
  - which job j should a person i do?
- Define **Parameters**, that is the given, known elements of the problem.
- Define **Decision Variables** of suitable type (continuous, integer, binary) corresponding to the needs
- Formulate **Objective Function** computing the benefit/cost
- Formulate mathematical **Constraints** indicating the interplay between the different variables.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Resource Allocation

In manufacturing industry, **factory planning**: find the best product mix.

Example

A factory makes two products **standard** and **deluxe**. Eg, yogurt, sleeping beds, etc.

A unit of standard gives a profit of 6(k) Dkk.
A unit of deluxe gives a profit of 8(k) Dkk.

The warming|grinding and cooling|polishing times in terms of hours per week for a **unit** of each type of product are given below:

	Standard	Deluxe
(Machine 1) Warming|Grinding	5	10
(Machine 2) Cooling|Polishing	4	4

Warming|Grinding capacity: 60 hours per week
Cooling|Polishing capacity: 40 hours per week

Q: How much of each product, **standard** and **deluxe**, should we produce to maximize the profit?

Mathematical Model

Decision Variables

$x_1 \geq 0$ units of product standard
$x_2 \geq 0$ units of product deluxe

Object Function

$\max\ 6x_1 + 8x_2$ maximize profit

Constraints

$5x_1 + 10x_2 \leq 60$ machine 1 capacity
$4x_1 + 4x_2 \leq 40$ machine 2 capacity

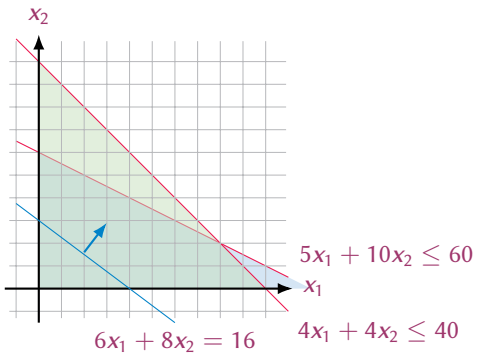
Mathematical Model

Machines/Materials A and B
Products 1 and 2

$$
\begin{array}{rrcrcl}
\max & 6x_1 & + & 8x_2 & & \\
 & 5x_1 & + & 10x_2 & \leq & 60 \\
 & 4x_1 & + & 4x_2 & \leq & 40 \\
 & & & x_1 & \geq & 0 \\
 & & & x_2 & \geq & 0
\end{array}
$$

a_{ij}	1	2	b_i
A	5	10	60
B	4	4	40
c_j	6	8	

Graphical Representation:



Resource Allocation - General Model

Managing a production facility

$j = 1, 2, \ldots, n$	products
$i = 1, 2, \ldots, m$	materials
b_i	units of raw material at disposal
a_{ij}	units of raw material i to produce one unit of product j
σ_j	market price of unit of jth product
ρ_i	prevailing market value for material i
$c_j = \sigma_j - \sum_{i=1}^{m} \rho_i a_{ij}$	profit per unit of product j
x_j	amount of product j to produce

$$
\begin{array}{rrcrcrcrcl}
\max & c_1 x_1 & + & c_2 x_2 & + & c_3 x_3 & + & \ldots & + & c_n x_n & = & z \\
\text{subject to} & a_{11} x_1 & + & a_{12} x_2 & + & a_{13} x_3 & + & \ldots & + & a_{1n} x_n & \leq & b_1 \\
 & a_{21} x_1 & + & a_{22} x_2 & + & a_{23} x_3 & + & \ldots & + & a_{2n} x_n & \leq & b_2 \\
 & \ldots & & & & & & & & & & \\
 & a_{m1} x_1 & + & a_{m2} x_2 & + & a_{m3} x_3 & + & \ldots & + & a_{mn} x_n & \leq & b_m \\
 & & & & & & & & & x_1, x_2, \ldots, x_n & \geq & 0
\end{array}
$$

Notation

$$
\begin{array}{rrcrcrcrcrcl}
\max & c_1x_1 & + & c_2x_2 & + & c_3x_3 & + & \dots & + & c_nx_n & = & z \\
\text{s.t.} & a_{11}x_1 & + & a_{12}x_2 & + & a_{13}x_3 & + & \dots & + & a_{1n}x_n & \leq & b_1 \\
& a_{21}x_1 & + & a_{22}x_2 & + & a_{23}x_3 & + & \dots & + & a_{2n}x_n & \leq & b_2 \\
& \dots & & & & & & & & & & \\
& a_{m1}x_1 & + & a_{m2}x_2 & + & a_{m3}x_3 & + & \dots & + & a_{mn}x_n & \leq & b_m \\
& & & & & & & & & x_1, x_2, \dots, x_n & \geq & 0
\end{array}
$$

$$
\begin{aligned}
\max \quad & \sum_{j=1}^{n} c_j x_j \\
& \sum_{j=1}^{n} a_{ij} x_j \leq b_i, \quad i = 1, \dots, m \\
& x_j \geq 0, \quad j = 1, \dots, n
\end{aligned}
$$

In Matrix Form

$$
\begin{array}{rrcrcrcrcl}
\max & c_1x_1 & + & c_2x_2 & + & c_3x_3 & + \ \ldots \ + & c_nx_n & = & z \\
\text{s.t.} & a_{11}x_1 & + & a_{12}x_2 & + & a_{13}x_3 & + \ \ldots \ + & a_{1n}x_n & \leq & b_1 \\
& a_{21}x_1 & + & a_{22}x_2 & + & a_{23}x_3 & + \ \ldots \ + & a_{2n}x_n & \leq & b_2 \\
& \ldots & & & & & & & & \\
& a_{m1}x_1 & + & a_{m2}x_2 & + & a_{m3}x_3 & + \ \ldots \ + & a_{mn}x_n & \leq & b_m \\
& & & & & & & x_1, x_2, \ldots, x_n & \geq & 0
\end{array}
$$

$$
\mathbf{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}, \quad A = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ a_{21} & a_{22} & \ldots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \ldots & a_{mn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}
$$

$$
\begin{array}{rrcl}
\max & z & = & \mathbf{c}^T\mathbf{x} \\
& A\mathbf{x} & \leq & \mathbf{b} \\
& \mathbf{x} & \geq & 0
\end{array}
$$

Our Numerical Example

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} c_j x_j \\ & \sum_{j=1}^{n} a_{ij} x_j \leq b_i, \quad i = 1, \ldots, m \\ & x_j \geq 0, \quad j = 1, \ldots, n \end{aligned}$$

$$\begin{aligned} \max \quad & 6x_1 + 8x_2 \\ & 5x_1 + 10x_2 \leq 60 \\ & 4x_1 + 4x_2 \leq 40 \\ & x_1, x_2 \geq 0 \end{aligned}$$

$$\begin{aligned} \max \quad & \mathbf{c}^T \mathbf{x} \\ & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq 0 \end{aligned}$$

$$\mathbf{x} \in \mathbb{R}^n, \mathbf{c} \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$$

$$\max \quad \begin{bmatrix} 6 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 10 \\ 4 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 60 \\ 40 \end{bmatrix}$$

$$x_1, x_2 \geq 0$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Duality

Resource Valuation problem: Determine the value of the raw materials in hand such that: (i) it would be convenient selling and (ii) an outside company would be willing to buy them.

z_i	value of a unit of raw material i
$\sum_{i=1}^{m} b_i z_i$	total expenses for buying (or opportunity cost, cost of having instead of selling)
ρ_i	prevailing unit market value of material i
σ_j	prevailing unit product price

Goal: for the outside company to minimize the total expenses;
(for the owing company the minimum amount of opportunity cost to accept for selling)

$$\min \sum_{i=1}^{m} b_i z_i \tag{1}$$

$$\sum_{i=1}^{m} z_i a_{ij} \geq \sigma_j, \quad j = 1 \ldots n \tag{2}$$

$$z_i \geq \rho_i, \quad i = 1 \ldots m \tag{3}$$

(2) otherwise not selling and (3) otherwise selling to someone else

Let

$$y_i = z_i - \rho_i$$

markup that the company would make by selling the raw material instead of producing.

$$\min \sum_{i=1}^{m} y_i b_i + \cancel{\sum_{i} \rho_i b_i}$$

$$\sum_{i=1}^{m} y_i a_{ij} \geq c_j, \quad j = 1 \ldots n$$

$$y_i \geq 0, \quad i = 1 \ldots m$$

Dual Problem

$$\max \sum_{j=1}^{n} c_j x_j$$

$$\sum_{j=1}^{n} a_{ij} x_j \leq b_i, \quad i = 1, \ldots, m$$

$$x_j \geq 0, \quad j = 1, \ldots, n$$

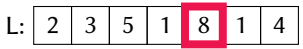
Primal Problem

Notions of Computer Science

Algorithm: a finite, well-defined sequence of operations to perform a calculation

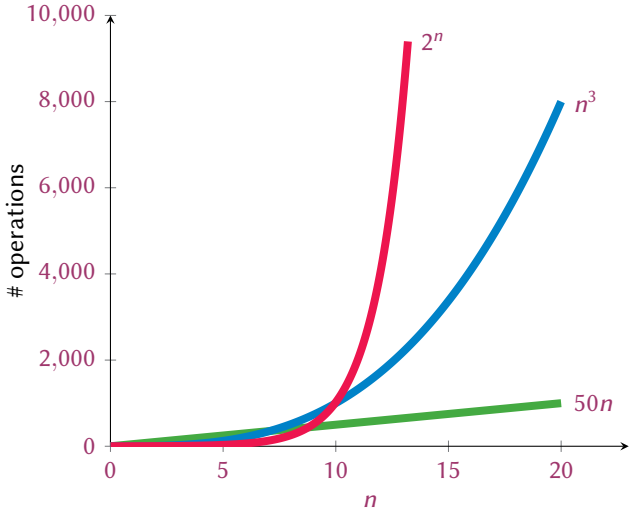
Algorithm: LargestNumber

Input: A non-empty list of numbers L
Output: The largest number in the list L
largest ← L[0]
foreach each item in the list L **do**
    if the item > largest **then**
        largest ← the item
return largest



Running time: proportional to number of operations, eg $O(n)$

Growth Functions



NP-hard problems: bad if we have to solve them, good for cryptology

Outline

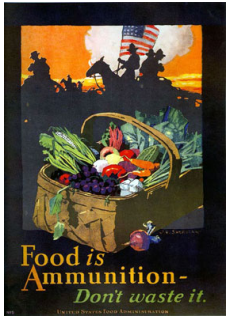
1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

The Diet Problem (Blending Problems)

- Select a set of foods that will satisfy a set of daily nutritional requirements at minimum cost.
- Motivated in the 1930s and 1940s by US army.
- Formulated as a **linear programming problem** by George Stigler
- (programming intended as planning not computer code)



min cost/weight
subject to nutrition requirements:
eat enough but not too much of Vitamin A
eat enough but not too much of Sodium
eat enough but not too much of Calories
...

The Diet Problem

Suppose there are:

- 3 foods available: corn, milk, and bread, and
- there are restrictions on the number of calories (between 2000 and 2250) and the amount of Vitamin A (between 5,000 and 50,000)

Food	Cost per serving	Vitamin A	Calories
Corn	$0.18	107	72
2% Milk	$0.23	500	121
Wheat Bread	$0.05	0	65

The Mathematical Model

Parameters (given data)

F := set of foods

N := set of nutrients

a_{ij} := amount of nutrient i in food j, $\forall i \in N, \forall j \in F$

c_j := cost per serving of food $j, \forall j \in F$

$F_{min,j}$:= minimum number of required servings of food $j, \forall j \in F$

$F_{max,j}$:= maximum allowable number of servings of food $j, \forall j \in F$

$N_{min,i}$:= minimum required level of nutrient $i, \forall i \in N$

$N_{max,i}$:= maximum allowable level of nutrient $i, \forall i \in N$

Decision Variables

x_j := number of servings of food j to purchase/consume, $\forall j \in F$

The Mathematical Model

Objective Function: Minimize the total cost of the food

$$\text{Minimize} \sum_{j \in F} c_j x_j$$

Constraint Set 1: For each nutrient $i \in N$, at least meet the minimum required level

$$\sum_{j \in F} a_{ij} x_j \geq N_{min,i}, \qquad \forall i \in N$$

Constraint Set 2: For each nutrient $i \in N$, do not exceed the maximum allowable level.

$$\sum_{j \in F} a_{ij} x_j \leq N_{max,i}, \qquad \forall i \in N$$

Constraint Set 3: For each food $j \in F$, select at least the minimum required number of servings

$$x_j \geq F_{min,j}, \qquad \forall j \in F$$

Constraint Set 4: For each food $j \in F$, do not exceed the maximum allowable number of servings.

$$x_j \leq F_{max,j}, \qquad \forall j \in F$$

The Mathematical Model

system of equalities and inequalities

$$\min \quad \sum_{j \in F} c_j x_j$$

$$\sum_{j \in F} a_{ij} x_j \geq N_{min,i}, \qquad \forall i \in N$$

$$\sum_{j \in F} a_{ij} x_j \leq N_{max,i}, \qquad \forall i \in N$$

$$x_j \geq F_{min,j}, \qquad \forall j \in F$$

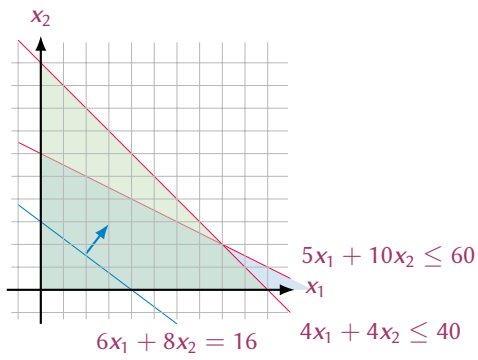
$$x_j \leq F_{max,j}, \qquad \forall j \in F$$

Mathematical Model

Machines/Materials A and B
Products 1 and 2

$$
\begin{array}{rrcrcl}
\max & 6x_1 & + & 8x_2 & & \\
 & 5x_1 & + & 10x_2 & \leq & 60 \\
 & 4x_1 & + & 4x_2 & \leq & 40 \\
 & & & x_1 & \geq & 0 \\
 & & & x_2 & \geq & 0
\end{array}
$$

Graphical Representation:



In Matrix Form

$$
\begin{array}{rrcrcrcrcl}
\max & c_1x_1 & + & c_2x_2 & + & c_3x_3 & + \ \ldots \ + & c_nx_n & = & z \\
\text{s.t.} & a_{11}x_1 & + & a_{12}x_2 & + & a_{13}x_3 & + \ \ldots \ + & a_{1n}x_n & \leq & b_1 \\
& a_{21}x_1 & + & a_{22}x_2 & + & a_{23}x_3 & + \ \ldots \ + & a_{2n}x_n & \leq & b_2 \\
& \ldots & & & & & & & & \\
& a_{m1}x_1 & + & a_{m2}x_2 & + & a_{m3}x_3 & + \ \ldots \ + & a_{mn}x_n & \leq & b_m \\
& & & & & & & x_1, x_2, \ldots, x_n & \geq & 0
\end{array}
$$

$$
\mathbf{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}, \quad A = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ a_{21} & a_{22} & \ldots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \ldots & a_{mn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}
$$

$$
\begin{array}{rrcl}
\max & z & = & \mathbf{c}^T\mathbf{x} \\
& A\mathbf{x} & \leq & \mathbf{b} \\
& \mathbf{x} & \geq & 0
\end{array}
$$

Linear Programming

Abstract mathematical model:
Parameters, Decision Variables, Objective, Constraints
(+ Domains & Quantifiers)

The Syntax of a Linear Programming Problem

objective func.	$\max / \min \quad \mathbf{c}^T\mathbf{x}$		$\mathbf{c} \in \mathbb{R}^n$
constraints	$\text{s.t.} \quad A\mathbf{x}$	$\leqslant \geqslant$ $\mathbf{b}$	$A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$
	$\mathbf{x}$	$\geq$ $\mathbf{0}$	$\mathbf{x} \in \mathbb{R}^n, \mathbf{0} \in \mathbb{R}^n$

Essential features: continuity, linearity (proportionality and additivity), certainty of parameters

- Any vector $\mathbf{x} \in \mathbb{R}^n$ satisfying all constraints is a **feasible solution**. $\mathbf{x} \in F$, F feasibility region.
- Each $\mathbf{x}^* \in \mathbb{R}^n$ that gives the best possible value for $\mathbf{c}^T\mathbf{x}$ among all feasible $\mathbf{x}$ is an **optimal solution** or **optimum**: x^* is optimum iff $\nexists\, \mathbf{x} \in F \mid \mathbf{c}^T\mathbf{x} < \mathbf{c}^T\mathbf{x}^*$
- The value $\mathbf{c}^T\mathbf{x}^*$ is the **optimum value**

Diet Problem — History

- The linear programming model consisted of 9 equations in 77 variables
- In 1944, Stigler guessed a near-optimal solution using a heuristic method
- In 1947, the National Bureau of Standards used the newly developed simplex method to solve Stigler's model.
  It took 9 clerks using hand-operated desk calculators 120 man days to solve for the optimal solution
- The original instance:
  https://developers.google.cn/optimization/lp/stigler_diet

AMPL Model

```
# diet.mod
set NUTR;
set FOOD;

param cost {FOOD} > 0;
param f_min {FOOD} >= 0;
param f_max { j in FOOD} >= f_min[j];
param n_min { NUTR } >= 0;
param n_max {i in NUTR } >= n_min[i];
param amt {NUTR,FOOD} >= 0;

var Buy { j in FOOD} >= f_min[j], <= f_max[j]

minimize total_cost: sum { j in FOOD } cost [j] * Buy[j];
subject to diet { i in NUTR }:
       n_min[i] <= sum {j in FOOD} amt[i,j] * Buy[j] <= n_max[i];
```

AMPL Model

```
# diet.dat
data;

set NUTR := A B1 B2 C ;
set FOOD := BEEF CHK FISH HAM MCH MTL SPG TUR;

param:   cost  f_min  f_max :=
  BEEF   3.19    0      100
  CHK    2.59    0      100
  FISH   2.29    0      100
  HAM    2.89    0      100
  MCH    1.89    0      100
  MTL    1.99    0      100
  SPG    1.99    0      100
  TUR    2.49    0      100 ;

param:   n_min  n_max :=
    A      700   10000
    C      700   10000
    B1     700   10000
    B2     700   10000 ;
# %
```

```
param amt (tr):
          A    C    B1   B2 :=
   BEEF   60   20   10   15
   CHK     8    0   20   20
   FISH    8   10   15   10
   HAM    40   40   35   10
   MCH    15   35   15   15
   MTL    70   30   15   15
   SPG    25   50   25   15
   TUR    60   20   15   10 ;
```

Python Script

Model

```
# Model diet.py
m = Model("diet")

# Create decision variables for the foods to buy
buy = {}
for f in foods:
    buy[f] = m.addVar(obj=cost[f], name=f)

# Nutrition constraints
for c in categories:
    m.addConstr(
      quicksum(nutritionValues[f,c] * buy[f] for f in foods) <= maxNutrition[c], name=c+'max')
    m.addConstr(
      quicksum(nutritionValues[f,c] * buy[f] for f in foods) >= minNutrition[c], name=c+'min')

# Solve
m.optimize()
```

Python Script

Data

```
import gurobipy as gp

categories, minNutrition, maxNutrition = gp.
    ↪multidict({
  'calories': [1800, 2200],
  'protein':  [91, gp.GRB.INFINITY],
  'fat':      [0, 65],
  'sodium':   [0, 1779] })

foods, cost = gp.multidict({
  'hamburger': 2.49,
  'chicken':   2.89,
  'hot dog':   1.50,
  'fries':     1.89,
  'macaroni':  2.09,
  'pizza':     1.99,
  'salad':     2.49,
  'milk':      0.89,
  'ice cream': 1.59 })
```

```
# Nutrition values for the foods
nutritionValues = {
  ('hamburger', 'calories'): 410,
  ('hamburger', 'protein'):  24,
  ('hamburger', 'fat'):      26,
  ('hamburger', 'sodium'):   730,
  ('chicken',   'calories'): 420,
  ('chicken',   'protein'):  32,
  ('chicken',   'fat'):      10,
  ('chicken',   'sodium'):   1190,
  ('hot dog',   'calories'): 560,
  ('hot dog',   'protein'):  20,
  ('hot dog',   'fat'):      32,
  ('hot dog',   'sodium'):   1800,
  ('fries',     'calories'): 380,
  ('fries',     'protein'):  4,
  ('fries',     'fat'):      19,
  ('fries',     'sodium'):   270,
  ('macaroni',  'calories'): 320,
```

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

History of Linear Programming (LP)

System of linear equations

$\rightsquigarrow$ It is impossible to find out who knew what, when, first.
Just two "references":

- Egyptians and Babylonians considered about 2000 B.C. the solution of special linear equations. But, of course, they described examples and did not describe the methods in "today's style".

- What we call "**Gaussian elimination**" today has been explicitly described in Chinese "Nine Books of Arithmetics" which is a compendium written in the period 2000 B.C. to A.D. 9, but the methods were probably known long before that.

- Gauss, by the way, never described "Gaussian elimination". He just used it and stated that the linear equations he used can be solved "per eliminationem vulgarem"

History of Linear Programming (LP)

George B. Dantzig, (2002) Linear Programming. Operations Research 50(1):42-47.
https://doi.org/10.1287/opre.50.1.42.17798

- Origins of LP date back to Newton, Leibnitz, Lagrange, etc.
- In 1827, Fourier described a variable elimination method for **systems of linear inequalities**, today often called Fourier-Motzkin elimination (Motzkin, 1937). It can be turned into an LP solver but inefficient.
- In 1932, Leontief (1905-1999) Input-Output model to represent interdependencies between branches of a national economy (1976 Nobel prize)
- In 1939, Kantorovich (1912-1986): Foundations of linear programming (Nobel prize in economics with Koopmans on LP, 1975) on Optimal use of scarce resources: foundation and economic interpretation of LP
- The math subfield of **Linear Programming** was created by George Dantzig, John von Neumann (Princeton), and Leonid Kantorovich in the 1940s.

History of LP (cntd)

- In 1947, Dantzig (1914-2005) invented the (primal) simplex algorithm working for the US Air Force at the Pentagon. (program=plan)
- In 1954, Lemke: dual simplex algorithm,
- In 1954, Dantzig and Orchard Hays: revised simplex algorithm
- In 1970, Victor Klee and George Minty created an example that showed that the classical simplex algorithm has exponential worst-case behavior.
- In 1979, L. Khachain found a new efficient algorithm for linear programming. It was terribly slow. (Ellipsoid method)
- In 1984, Karmarkar discovered yet another new efficient algorithm for linear programming. It proved to be a strong competitor for the simplex method. (Interior point method)

History of Optimization

- In 1951, Nonlinear Programming began with the Karush-Kuhn-Tucker Conditions
- In 1952, Commercial Applications and Software began
- In 1950s, Network Flow Theory began with the work of Ford and Fulkerson.
- In 1955, Stochastic Programming began
- In 1958, Integer Programming began by R.E. Gomory.
- In 1962, Complementary Pivot Theory

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Fourier Motzkin elimination method

Has $A\mathbf{x} \leq \mathbf{b}$ a solution? (Assumption: $A \in \mathbb{Q}^{m \times n}, \mathbf{b} \in \mathbb{Q}^n$)
Idea:

1. transform the system into another by eliminating some variables such that the two systems have the same solutions over the remaining variables.
2. reduce to a system of constant inequalities that can be easily decided

Let x_r be the variable to eliminate
Let $M = \{1 \dots m\}$ indices of the constraints
For a variable j let's partition the rows of the matrix in

$$\begin{aligned} N &= \{i \in M \mid a_{ij} < 0\} \\ Z &= \{i \in M \mid a_{ij} = 0\} \\ P &= \{i \in M \mid a_{ij} > 0\} \end{aligned}$$

$$\begin{cases} x_r \geq b'_{ir} - \sum_{k=1}^{r-1} a'_{ik} x_k, & a_{ir} < 0 \\ x_r \leq b'_{ir} - \sum_{k=1}^{r-1} a'_{ik} x_k, & a_{ir} > 0 \\ \text{all other constraints} & i \in Z \end{cases} \qquad \begin{cases} x_r \geq A_i(x_1, \ldots, x_{r-1}), & i \in N \\ x_r \leq B_i(x_1, \ldots, x_{r-1}), & i \in P \\ \text{all other constraints} & i \in Z \end{cases}$$

Hence the original system is equivalent to

$$\begin{cases} \max\{A_i(x_1, \ldots, x_{r-1}), i \in N\} \leq x_r \leq \min\{B_i(x_1, \ldots, x_{r-1}), i \in P\} \\ \text{all other constraints} \quad i \in Z \end{cases}$$

which is equivalent to

$$\begin{cases} A_i(x_1, \ldots, x_{r-1}) \leq B_j(x_1, \ldots, x_{r-1}) & i \in N, j \in P \\ \text{all other constraints} & i \in Z \end{cases}$$

we eliminated x_r but:

$$\begin{cases} |N| \cdot |P| \text{ inequalities} \\ |Z| \text{ inequalities} \end{cases}$$

after d iterations if $|P| = |N| = m/2$ exponential growth: $(1/4^d)(m/2)^{2^d}$

Example

$$\begin{array}{rcrcl} -7x_1 & + & 6x_2 & \leq & 25 \\ x_1 & - & 5x_2 & \leq & 1 \\ x_1 & & & \leq & 7 \\ -x_1 & + & 2x_2 & \leq & 12 \\ -x_1 & - & 3x_2 & \leq & 1 \\ 2x_1 & - & x_2 & \leq & 10 \end{array}$$

x_2 variable to eliminate
$N = \{2, 5, 6\}, Z = \{3\}, P = \{1, 4\}$
$|Z \cup (N \times P)| = 7$ constraints

By adding one variable and one inequality, Fourier-Motzkin elimination can be turned into an LP solver.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Definitions

- $[a, b] = \{x \in \mathbb{R} \mid a \leq x \leq b\}$ closed interval
  $(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$ open interval
- column vectors and matrices
  scalar product: $\mathbf{y}^T\mathbf{x} = \sum_{i=1}^{n} y_i x_i$
- $A\mathbf{x}$ column vector combination of the columns of A;
  $\mathbf{u}^T A$ row vector combination of the rows of A
- **linear combination**

$$\begin{matrix} \mathbf{v}_1, \mathbf{v}_2 \ldots, \mathbf{v}_k \in \mathbb{R}^n \\ \boldsymbol{\lambda} = [\lambda_1, \ldots, \lambda_k]^T \in \mathbb{R}^k \end{matrix} \qquad \mathbf{x} = \lambda_1\mathbf{v}_1 + \cdots + \lambda_k\mathbf{v}_k = \sum_{i=1}^{k} \lambda_i \mathbf{v}_i$$

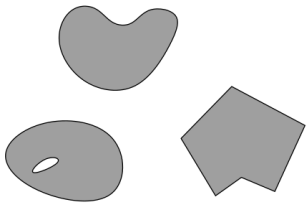
moreover:

$\boldsymbol{\lambda} \geq \mathbf{0}$		**conic combination**
	$\boldsymbol{\lambda}^T\mathbf{1} = 1$	**affine combination**
$\boldsymbol{\lambda} \geq \mathbf{0}$	and $\boldsymbol{\lambda}^T\mathbf{1} = 1$	**convex combination**

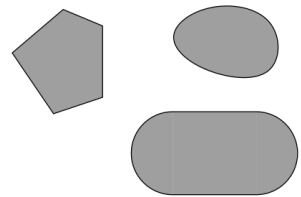
$$\left(\sum_{i=1}^{k} \lambda_i = 1\right)$$

Definitions

- set S is **linear (affine) independent** if no element of it can be expressed as linear combination of the others
  Eg: $S \subseteq \mathbb{R}^n \implies$ max n lin. indep. (max $n+1$ aff. indep.)
- **convex set**: if $\mathbf{x}, \mathbf{y} \in S$ and $0 \leq \lambda \leq 1$ then $\lambda\mathbf{x} + (1-\lambda)\mathbf{y} \in S$



nonconvex



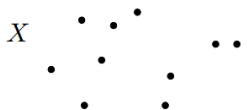
convex

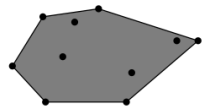
- $f : X \to \mathbb{R}$ is a **convex function** if its epigraph $\{(x, y) \in \mathbb{R}^2 : y \geq f(x)\}$ is a convex set or if $\forall x, y \in X, \lambda \in [0, 1]$ it holds that $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$

Definitions

- For a set of points $S \subseteq \mathbb{R}^n$

  $\mathrm{lin}(S)$ linear hull (span)
  $\mathrm{cone}(S)$ conic hull
  $\mathrm{aff}(S)$ affine hull
  $\mathrm{conv}(S)$ convex hull





the convex hull of X

$$\mathbf{conv}(X) = \left\{ \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2 + \ldots + \lambda_n \mathbf{x}_n \;\middle|\; \mathbf{x}_i \in X,\ \lambda_1, \ldots, \lambda_n \geq 0 \text{ and } \textstyle\sum_i \lambda_i = 1 \right\}$$

Definitions

- **rank** of a matrix for columns (= for rows)
  if (m, n)-matrix has rank $= \min\{m, n\}$ then the matrix is full rank
  if (n, n)-matrix is full rank then it is regular and admits an inverse

- $G \subseteq \mathbb{R}^n$ is an **hyperplane** if $\exists\, \mathbf{a} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ and $\alpha \in \mathbb{R}$:

$$G = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a}^T\mathbf{x} = \alpha\}$$

- $H \subseteq \mathbb{R}^n$ is an **halfspace** if $\exists\, \mathbf{a} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ and $\alpha \in \mathbb{R}$:

$$H = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a}^T\mathbf{x} \leq \alpha\}$$

($\mathbf{a}^T\mathbf{x} = \alpha$ is a supporting hyperplane of H)

Definitions

- a set $P \subset \mathbb{R}^n$ is a **polyhedron** if $\exists\, m \in \mathbb{Z}^+, A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$:

  $$P = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} \leq \mathbf{b}\} = \bigcap_{i=1}^{m} \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a}_{i,\cdot}^T \mathbf{x} \leq b_i\}$$

  i.e., a polyhedron $P \neq \mathbb{R}^n$ is determined by finitely many halfspaces

- a polyhedron P is a **polytope** if it is bounded: $\exists\, B \in \mathbb{R}, B > 0$:

  $$P \subseteq \{\mathbf{x} \in \mathbb{R}^n \mid \|\mathbf{x}\| \leq B\}$$

  ($\|\mathbf{x}\| = \sqrt{\sum_{i=1}^{n} x_i^2}$ is the Euclidean norm of the vector $\mathbf{x} \in \mathbb{R}$)

- a point $\mathbf{x}$ of a polyhedron P is said to be an **extreme point** or a **vertex** of P if it cannot be expressed as a strict convex combination of other two points of the polyhedron, i.e., if there exist no $\mathbf{y}, \mathbf{z} \in P$, $\mathbf{y} \neq \mathbf{z}$ and $\lambda \in (0, 1)$ such that $\mathbf{x} = \lambda\mathbf{y} + (1 - \lambda)\mathbf{z}$

- every point of a **polytope** can be obtained as the convex combination of its vertices. (Minkowski-Weyl Theorem)

Definitions

- If A and $\mathbf{b}$ are made of rational numbers, $P = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} \leq \mathbf{b}\}$ is a **rational polyhedron**
- General optimization problem: $\max\{\varphi(\mathbf{x}) \mid \mathbf{x} \in F\}$, F is feasible region for $\mathbf{x}$
- Note: if F is open, eg, $x < 5$ then: $\sup\{x \mid x < 5\}$
  supremum: least element of $\mathbb{R}$ greater or equal than any element in F
- $\arg\min\{f(i) \mid i \in I\}$ argument $i^* \in I$ such that $f(i^*) = \min\{f(i) \mid i \in I\}$

Definitions

- The inequality denoted by $(\mathbf{a}, \alpha)$ is called a **valid inequality for P** if $\mathbf{ax} \leq \alpha, \forall \mathbf{x} \in P$.
  Note that $(\mathbf{a}, \alpha)$ is a valid inequality if and only if P lies in the half-space $\{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{ax} \leq \alpha\}$.

- A **face** of P is $F = \{\mathbf{x} \in P \mid \mathbf{ax} = \alpha\}$ where $(\mathbf{a}, \alpha)$ is a valid inequality for P. Hence, it is the intersection of P with the hyperplane of a valid inequality. It is said to be **proper** if $F \neq \emptyset$ and $F \neq P$.

- If $F \neq \emptyset$ we say that the corresponding hyperplane **supports** P.
  If $\mathbf{c}$ is a non zero vector for which $\delta = \max\{\mathbf{c}^T\mathbf{x} \mid \mathbf{x} \in P\}$ is finite,
  then the set $\{\mathbf{x} \mid \mathbf{c}^T\mathbf{x} = \delta\}$ is called **supporting hyperplane**.

- A point $\mathbf{x}$ for which $\{\mathbf{x}\}$ is a face is called a **vertex** of P and also a **basic solution** of $A\mathbf{x} \leq \mathbf{b}$
  (0 dim face)

- A **facet** is a maximal face distinct from P
  $\mathbf{cx} \leq d$ is facet defining if $\mathbf{cx} = d$ is a supporting hyperplane of P of $n - 1$ dim

Linear Programming Problem

Input: a matrix $A \in \mathbb{R}^{m \times n}$ and column vectors $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$

Task:

1. decide that $\{\mathbf{x} \in \mathbb{R}^n; A\mathbf{x} \leq \mathbf{b}\}$ is empty (prob. infeasible), or
2. find a column vector $\mathbf{x} \in \mathbb{R}^n$ such that $A\mathbf{x} \leq \mathbf{b}$ and $\mathbf{c}^T\mathbf{x}$ is max, or
3. decide that for all $\alpha \in \mathbb{R}$ there is an $\mathbf{x} \in \mathbb{R}^n$ with $A\mathbf{x} \leq \mathbf{b}$ and $\mathbf{c}^T\mathbf{x} > \alpha$ (prob. unbounded)

1. $F = \emptyset$
2. $F \neq \emptyset$ and $\exists$ solution
   1. one solution
   2. infinite solutions
3. $F \neq \emptyset$ and $\nexists$ solution

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Fundamental Theorem of LP

Theorem (Fundamental Theorem of Linear Programming)

Given:

$$\min\{\mathbf{c}^T\mathbf{x} \mid \mathbf{x} \in P\} \quad \textit{where } P = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} \leq \mathbf{b}\}$$

If P is a bounded polyhedron and not empty and $\mathbf{x}^*$ is an optimal solution to the problem, then:

- $\mathbf{x}^*$ *is an extreme point (vertex) of P, or*
- $\mathbf{x}^*$ *lies on a face $F \subset P$ of optimal solutions*



Proof idea:

- assume $\mathbf{x}^*$ not on the boundary of P then $\exists$ a ball around it still in P. Show that a point in the ball has better cost
- if $\mathbf{x}^*$ is not unique then any convex combination of other optimal points are also optimal.

Implications:

- the optimal solution is at the intersection of supporting hyperplanes.
- hence finitely many possibilities
- solution method: write all inequalities as equalities and solve all $\binom{m}{n}$ systems of linear equalities (n # variables, m # equality constraints)
- for each point we then need to check if feasible and if best in cost.
- each system is solved by Gaussian elimination
- Stirling approximation:

$$\binom{2m}{m} \approx \frac{4^m}{\sqrt{\pi m}} \text{ as } m \to \infty$$

Simplex Method

1. find a solution that is at the intersection of some n hyperplanes
2. try systematically to produce the other points by exchanging one hyperplane with another
3. check optimality, proof provided by duality theory

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Gaussian Elimination

1. Forward elimination
   reduces the system to row echelon form by elementary row operations
   - multiply a row by a non-zero constant
   - interchange two rows
   - add a multiple of one row to another

   (or LU decomposition)

2. Back substitution (or reduced row echelon form - RREF)

Example

$$
\begin{aligned}
2x + y - z &= 8 && (R1)\\
-3x - y + 2z &= -11 && (R2)\\
-2x + y + 2z &= -3 && (R3)
\end{aligned}
$$

R1	2	1	-1	8
R2	-3	-1	2	-11
R3	-2	1	2	-3

$$
\begin{aligned}
2x + y - z &= 8 && (R1)\\
+ \tfrac{1}{2}y + \tfrac{1}{2}z &= 1 && (R2)\\
+ 2y + 1z &= 5 && (R3)
\end{aligned}
$$

R1'=1/2 R1	1	1/2	-1/2	4
R2'=R2+3/2xR1	0	1/2	1/2	1
R3'=R3+R1	0	2	1	5

$$
\begin{aligned}
2x + y - z &= 8 && (R1)\\
+ \tfrac{1}{2}y + \tfrac{1}{2}z &= 1 && (R2)\\
- z &= 1 && (R3)
\end{aligned}
$$

R1'=R1	1	1/2	-1/2	4
R2'=2 R2	0	1	1	2
R3'=R3-4xR2	0	0	-1	1

$$
\begin{aligned}
1x + 1/2y &= 7/2 && (R1)\\
1y &= 3 && (R2)\\
z &= -1 && (R3)
\end{aligned}
$$

R1'=R1-1/2xR3	1	1/2	0	7/2
R2'=R2+R3	0	1	0	3
R3'=-R3	0	0	1	-1

$$
\begin{aligned}
x &= 2 && (R1)\\
y &= 3 && (R2)\\
z &= -1 && (R3)
\end{aligned}
$$

R1'=R1-1/2xR2	1	0	0	2 => x=2
R2'=R2	0	1	0	3 => y=3
R3'=R3	0	0	1	-1 => z=-1

LU Factorization

$$\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}$$

$$A\mathbf{x} = \mathbf{b}$$

$$\mathbf{x} = A^{-1}\mathbf{b}$$

$$\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

$$A = PLU$$

$$\mathbf{x} = A^{-1}\mathbf{b} = U^{-1}L^{-1}P^{T}\mathbf{b}$$

$$\mathbf{z}_1 = P^{T}\mathbf{b}, \quad \mathbf{z}_2 = L^{-1}\mathbf{z}_1, \quad \mathbf{x} = U^{-1}\mathbf{z}_2$$

In Python

```
In [1]: import scipy as sc

In [2]: A = sc.array([[2,1,-1],[-3,-1,2],[-2,1,2]])

In [3]: from scipy import linalg as sl

In [4]: P,L,U = sl.lu(A)

In [5]: print(P,L,U)
        [[0. 0. 1.]
         [1. 0. 0.]
         [0. 1. 0.]]
        [[ 1.          0.          0.        ]
         [ 0.66666667  1.          0.        ]
         [-0.66666667  0.2         1.        ]]
        [[-3.         -1.          2.        ]
         [ 0.          1.66666667  0.66666667]
         [ 0.          0.          0.2       ]]
```

Polynomial time $O(n^2 m)$ but needs to guarantee that all the numbers during the run can be represented by polynomially bounded bits

Summary

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

A Numerical Example

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} c_j x_j \\ & \sum_{j=1}^{n} a_{ij} x_j \leq b_i, \quad i = 1, \ldots, m \\ & x_j \geq 0, \quad j = 1, \ldots, n \end{aligned}$$

$$\begin{aligned} \max \quad & 6x_1 + 8x_2 \\ & 5x_1 + 10x_2 \leq 60 \\ & 4x_1 + 4x_2 \leq 40 \\ & x_1, x_2 \geq 0 \end{aligned}$$

$$\begin{aligned} \max \quad & \mathbf{c}^T \mathbf{x} \\ & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

$$\mathbf{x} \in \mathbb{R}^n, \mathbf{c} \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$$

$$\max \quad \begin{bmatrix} 6 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 10 \\ 4 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 60 \\ 40 \end{bmatrix}$$

$$x_1, x_2 \geq 0$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Standard Form

Every LP problem can be converted in the **standard form**:

$$\begin{aligned} \max \quad & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \in \mathbb{R}^n \end{aligned}$$

$\mathbf{c} \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$

- if equations, then put two constraints, $\mathbf{a}x \leq b$ and $\mathbf{a}x \geq b$
- if $\mathbf{a}x \geq b$ then $-\mathbf{a}x \leq -b$
- if $\min \mathbf{c}^T\mathbf{x}$ then $-\max(-\mathbf{c}^T\mathbf{x})$

and then be put in **equational standard form**:

$$\begin{aligned} \max \quad & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

$\mathbf{x} \in \mathbb{R}^n, \mathbf{c} \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$

1. "$=$" constraints
2. $\mathbf{x} \geq 0$ nonnegativity constraints
3. $(\mathbf{b} \geq 0)$
4. max

Transformation to Std Form

Every LP problem can be transformed in eq. std. form

1. introduce slack variables (or surplus)

$$\begin{aligned} 5x_1 &+ 10x_2 &+ x_3 &= 60 \\ 4x_1 &+ 4x_2 &+ x_4 &= 40 \end{aligned}$$

2. if $x_1 \gtreqless 0$ then $\begin{aligned} x_1 &= x_1' - x_1'' \\ x_1' &\geq 0 \\ x_1'' &\geq 0 \end{aligned}$

3. $(b \geq 0)$

4. $\min c^T x \equiv -\max(-c^T x)$

LP in $m \times n$ converted into LP with at most $(m + 2n)$ variables and m equations (n # original variables, m # constraints)

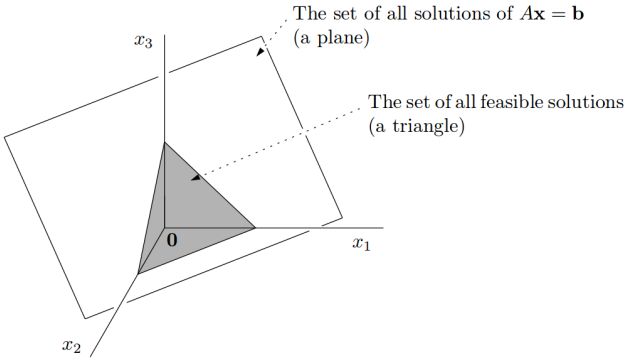
Geometry of LP in Eq. Std. Form

$$\max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$$

From linear algebra:

- the set of solutions of $A\mathbf{x} = \mathbf{b}$ is an affine space (hyperplane not passing through the origin).
- $\mathbf{x} \geq \mathbf{0}$ nonegative orthant (octant in $\mathbb{R}^3$)

In $\mathbb{R}^3$:



- $A\mathbf{x} = \mathbf{b}$ is a system of equations that we can solve by Gaussian elimination
- Elementary row operations of $\left[A \mid \mathbf{b}\right]$ do not affect set of feasible solutions
  - multiplying all entries in some row of $\left[A \mid \mathbf{b}\right]$ by a nonzero real number λ
  - replacing the ith row of $\left[A \mid b\right]$ by the sum of the ith row and jth row for some $i \neq j$
- Let n' be the number of vars in eq. std. form.

  $$\text{we assume } n' \geq m \text{ and } \operatorname{rank}(\left[A \mid \mathbf{b}\right]) = \operatorname{rank}(A) = m$$

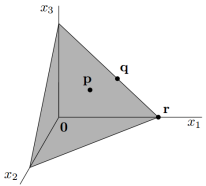
  ie, rows of A are linearly independent
  otherwise, remove linear dependent rows
  If $A \in \mathbb{R}^{m \times n'}$ has rank m, then $A\mathbf{x} = \mathbf{b}$ is an $(n' - m)$-dimensional affine plane.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Basic Feasible Solutions

Basic feasible solutions are the vertices of the feasible region:



More formally:
Let $B = \{1 \dots m\}$, $N = \{m+1 \dots n+m = n'\}$ be subsets partitioning the columns of A: A_B be made of columns of A indexed by B:

Definition

$\mathbf{x} \in \mathbb{R}^n$ is a **basic feasible solution** of the linear program $\max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ for an index set B if:

- $x_j = 0 \; \forall j \notin B$
- the square matrix A_B is nonsingular, ie, all columns indexed by B are lin. indep.
- $\mathbf{x}_B = A_B^{-1}\mathbf{b}$ is nonnegative, ie, $\mathbf{x}_B \geq 0$ (feasibility)

We call x_j for $j \in B$ **basic variables** and remaining variables **nonbasic variables**.

Theorem

A **basic feasible solution** *is uniquely determined by the set* B.

Proof:

$$
\begin{aligned}
A\mathbf{x} &= A_B\mathbf{x}_B + A_N\mathbf{x}_N = b \\
\mathbf{x}_B + A_B^{-1}A_N\mathbf{x}_N &= A_B^{-1}b \\
\mathbf{x}_B &= A_B^{-1}b \qquad\qquad A_B \text{ is nonsingular hence one solution}
\end{aligned}
$$

Note: we call B a **(feasible) basis**

Definition

A **basic feasible solution** of a linear program with n variables is a feasible solution for which some n linearly independent constraints hold with equality.

Extreme points and basic feasible solutions are geometric and algebraic manifestations of the same concept:

Theorem

Let P be a (convex) polyhedron made of all feasible solutions of an LP in std. form. For a point $v \in P$ the following are equivalent:

(i) *v is an extreme point (vertex) of P*

(ii) *v is a basic feasible solution of LP*

Proof: see text book [MG] sec. 4.4.

Theorem

Let $LP = \max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ be feasible and bounded,
then there is an optimal solution that is a basic feasible solution.

Proof. consequence of previous theorem and fundamental theorem of linear programming

A similar theorem is valid for arbitrary linear programs (not in eq. form)

However, an optimal solution does not need to be basic:

$\max\ x_1 + x_2$ subject to $x_1 + x_2 \leq 1$

(bounded case with infinite solutions)

- Idea for solution method:
- examine all basic solutions.
- There are finitely many: $\binom{m+n}{m}$.
- However, if $n = m$ then $\binom{2m}{m} \approx 4^m$.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Simplex Method

$$\max \quad z = \begin{bmatrix} 6 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 10 & 1 & 0 \\ 4 & 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 60 \\ 40 \end{bmatrix}$$

$$x_1, x_2, x_3, x_4 \geq 0$$

Canonical eq. std. form: one decision variable is isolated in each constraint with coefficient 1 and does not appear in the other constraints nor in the obj. func. and b terms are positive

It gives immediately a basic feasible solution:

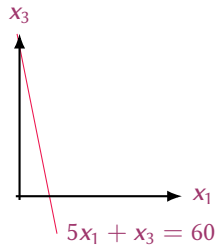
$$x_1 = 0,\ x_2 = 0,\ x_3 = 60,\ x_4 = 40$$

Is it optimal? Look at signs in $z \rightsquigarrow$ if positive then an increase would improve.

Let's try to increase a promising variable, ie, x_1, one with positive coefficient in z

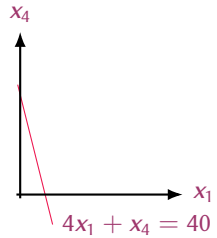
$$5x_1 + x_3 = 60$$
$$x_1 = \frac{60}{5} - \frac{x_3}{5}$$
$$x_3 = 60 - 5x_1 \geq 0$$

If $x_1 > 12$ then $x_3 < 0$



$$4x_1 + x_4 = 40$$
$$x_1 = \frac{40}{4} - \frac{x_4}{4}$$
$$x_4 = 40 - 4x_1 \geq 0$$

If $x_1 > 10$ then $x_4 < 0$



we can take the minimum of the two $\rightsquigarrow$ x_1 increased to 10
x_1 **enters** the basis and x_4 **leaves** it.

Dictionary Form

Hence, we increase x_1 to 10 and consequently set $x_4 = 0$
That is, x_1 **enters** the basis and x_4 **leaves** it.

$$
\begin{aligned}
\max\ & 6x_1 + 8x_2 \\
& 5x_1 + 10x_2 + x_3 = 60 \\
& 4x_1 + 4x_2 + x_4 = 40 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

$$
\begin{aligned}
x_3 &= 60 - 5x_1 - 10x_2 \\
x_4 &= 40 - 4x_1 - 4x_2 \\
\hline
z &= + 6x_1 + 8x_2
\end{aligned}
$$

$$
\begin{aligned}
x_3 &= 10 - 5x_2 + 5/4x_4 \\
x_1 &= 10 - x_2 - 1/4x_4 \\
\hline
z &= 60 + 2x_2 - 6/4x_4
\end{aligned}
$$

Simplex Tableau

First simplex tableau:

	x_1	x_2	x_3	x_4	$-z$	b
x_3	5	10	1	0	0	60
x_4	4	4	0	1	0	40
	6	8	0	0	1	0

we want to reach this new tableau

	x_1	x_2	x_3	x_4	$-z$	b
x_3	0	?	1	?	0	?
x_1	1	?	0	?	0	?
	0	?	0	?	1	?

Pivot operation:

1. Choose pivot:

   column: one s with positive coefficient in obj. func.

   row: ratio between coefficient b and pivot column: choose the one with smallest ratio:

$$\theta = \min_i \left\{ \frac{b_i}{a_{is}} : a_{is} > 0 \right\}, \qquad \theta \text{ increase value of entering var.}$$

2. elementary row operations to update the tableau

- x_4 leaves the basis, x_1 enters the basis
  - Divide pivot row by pivot
  - Send to zero the coefficient in the pivot column of the first row
  - Send to zero the coefficient of the pivot column in the third (cost) row

	x1	x2	x3	x4	-z	b
I'=I-5II'	0	5	1	-5/4	0	10
II'=II/4	1	1	0	1/4	0	10
III'=III-6II'	0	2	0	-6/4	1	-60

From the last row we read: $2x_2 - 3/2x_4 - z = -60$, that is: $z = 60 + 2x_2 - 3/2x_4$.
Since x_2 and x_4 are nonbasic we have $z = 60$ and $x_1 = 10, x_2 = 0, x_3 = 10, x_4 = 0$.

- Done? No! Let x_2 enter the basis

	x1	x2	x3	x4	-z	b
I'=I/5	0	1	1/5	-1/4	0	2
II'=II-I'	1	0	-1/5	1/2	0	8
III'=III-2I'	0	0	-2/5	-1	1	-64

Definition (Reduced costs)

We call **reduced costs** the coefficients in the objective function of the nonbasic variables, $\bar{c}_N$

Proposition (Optimality Condition)

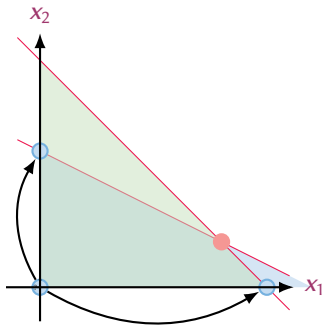
The basic feasible solution is **optimal** *when the* **reduced costs** *in the corresponding simplex tableau are nonpositive, ie, such that:*

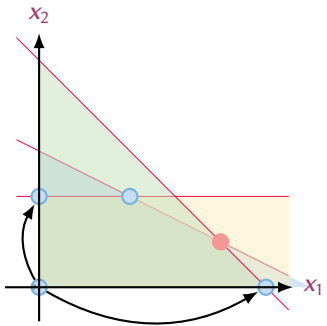
$$\bar{c}_N \leq 0$$

Proof: Let z_0 be the obj value when $\bar{\mathbf{c}}_N \leq 0$.
For any other feasible solution $\tilde{\mathbf{x}} = [\tilde{\mathbf{x}}_B, \tilde{\mathbf{x}}_N]$, $\tilde{\mathbf{x}}_N \geq 0$ reachable from the tableau we have:

$$z = \mathbf{c}^T\tilde{\mathbf{x}} \quad \text{and from the last line of the tableau} \quad \mathbf{c}^T\tilde{\mathbf{x}} = z = z_0 + \bar{\mathbf{c}}_B^T\tilde{\mathbf{x}}_B + \bar{\mathbf{c}}_N^T\tilde{\mathbf{x}}_N \leq z_0$$

Graphical Representation





Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Tableaux and Dictionaries

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} c_j x_j \\ & \sum_{j=1}^{n} a_{ij} x_j \le b_i, \quad i = 1, \ldots, m \\ & x_j \ge 0, \quad j = 1, \ldots, n \end{aligned}$$

$$x_{n+i} = b_i - \sum_{j=1}^{n} a_{ij} x_j, \quad i = 1, \ldots, m$$

$$z = \sum_{j=1}^{n} c_j x_j$$

Tableau

$$\left[\begin{array}{c|c|c|c} I & \bar{A}_N & 0 & \bar{b} \\ \hline 0 & \bar{c}_N & 1 & -\bar{d} \end{array}\right]$$

Dictionary

$$x_r = \bar{b}_r - \sum_{s \notin B} \bar{a}_{rs} x_s, \quad r \in B$$

$$z = \bar{d} + \sum_{s \notin B} \bar{c}_s x_s$$

pivot operations in dictionary form:
choose col s with r.c. > 0
choose row with $\min\{-\bar{b}_i/\bar{a}_{is} \mid a_{is} < 0, i = 1, \ldots, m\}$
update: express entering variable and substitute in other rows

Example

$$
\begin{aligned}
\max\ & 6x_1 + 8x_2 \\
& 5x_1 + 10x_2 \leq 60 \\
& 4x_1 + 4x_2 \leq 40 \\
& x_1, x_2 \geq 0
\end{aligned}
$$

	x_1	x_2	x_3	x_4	$-z$	b
x_3	5	10	1	0	0	60
x_4	4	4	0	1	0	40
	6	8	0	0	1	0

$$
\begin{aligned}
x_3 &= 60 - 5x_1 - 10x_2 \\
x_4 &= 40 - 4x_1 - 4x_2 \\
\hline
z &= + 6x_1 + 8x_2
\end{aligned}
$$

After 2 iterations:

	x_1	x_2	x_3	x_4	$-z$	b
x_2	0	1	$1/5$	$-1/4$	0	2
x_1	1	0	$-1/5$	$1/2$	0	8
	0	0	$-2/5$	-1	1	-64

$$
\begin{aligned}
x_2 &= 2 - 1/5x_3 + 1/4x_4 \\
x_1 &= 8 + 1/5x_3 - 1/2x_4 \\
\hline
z &= 64 - 2/5x_3 - 1x_4
\end{aligned}
$$

Summary

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Simplex Algorithm so far

while there exists a nonbasic variable with **positive** reduced cost **do**

1. Choose pivot:

   column: one s with positive coefficient in obj. func. later)
   ($\equiv$ choosing the entering variable)

   row: one with the smallest ratio between coefficient b and coefficient in the pivot column (aka **minimum ratio test**):

$$\theta = \min_i \left\{ \frac{b_i}{a_{is}} : a_{is} > 0 \right\}, \qquad \theta \text{ increase value of entering var.}$$

   ($\equiv$ choosing the leaving variable)

2. Elementary row operations to update the tableau

Simplex: Exception Handling, Overview

Solution of an LP problem:

a. $F \neq \emptyset$ and $\nexists$ solution

b. $F \neq \emptyset$ and $\exists$ solution

   i) **one solution**

   ii) infinite solutions

c. $F = \emptyset$

Handling exceptions in the Simplex Method

1. Unboundedness
2. More than one solution
3. Degeneracies
   - benign
   - cycling
4. Infeasible starting
   Phase I + Phase II

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Unboundedness

$$\begin{array}{rrcl} \max & 2x_1 & + & x_2 \\ & & & x_2 \leq 5 \\ & -x_1 & + & x_2 \leq 1 \\ & & & x_1, x_2 \geq 0 \end{array}$$

- Initial tableau

	x1	x2	x3	x4	-z	b
x3	0	1	1	0	0	5
x4	-1	1	0	1	0	1
	2	1	0	0	1	0

- x_2 entering, x_4 leaving

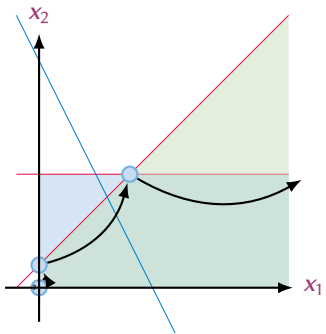
	x1	x2	x3	x4	-z	b
II'=II-I'	1	0	1	-1	0	4
I'=I	-1	1	0	1	0	1
III'=III-I'	3	0	0	-1	1	-1

$-x_1 + x_2 + x_4 = 1$, x_1 can increase without restriction, $\theta = \min\{\frac{b_i}{a_{is}} : a_{is} > 0, i = 1 \ldots, n\}$

- x_1 entering, x_3 leaving

	x1	x2	x3	x4	-z	b
I'=I	1	0	1	-1	0	4
II'=II+I'	0	1	1	0	0	5
III'=III-3I'	0	0	-3	2	1	-13

  x_4 was already in basis but for both I and II ($x_2 + x_3 + 0x_4 = 5$), x_4 can increase arbitrarily



∞ solutions

$$\begin{array}{lrcrcl} \max & x_1 & + & x_2 & & \\ & 5x_1 & + & 10x_2 & \leq & 60 \\ & 4x_1 & + & 4x_2 & \leq & 40 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

- Initial tableau

	x1	x2	x3	x4	-z	b
x3	5	10	1	0	0	60
x4	4	4	0	1	0	40
	1	1	0	0	1	0

- x_2 enters, x_3 leaves

	x1	x2	x3	x4	-z	b
I'=I/10	1/2	1	1/10	0	0	6
II'=II-4Ix4	2	0	-2/5	1	0	16
III'=III-I	1/2	0	-1/6	0	1	-6

- x_1 enters, x_4 leaves

	x1	x2	x3	x4	-z	b
I'=I-II'/2	0	1	1/5	-1/4	0	2
II'=II/2	1	0	-1/5	1/2	0	8
III'=III-II'/2	0	0	0	-1/4	1	-10

  $\mathbf{x} = (8, 2, 0, 0), z = 10$

  nonbasic variables typically have reduced costs $\neq 0$. Here x_3 has r.c. $= 0$. Let's make it enter the basis

- x_3 enters, x_2 leaves

	x1	x2	x3	x4	-z	b
I'=5I	0	5	1	-5/4	0	10
II'=II+I'/5	1	1	0	4	0	10
III'=III	0	0	0	-1/4	1	-10

  $\mathbf{x} = (10, 0, 10, 0), z = 10$

  There are 2 optimal solutions $\rightsquigarrow$ all their convex combinations are optimal solutions (from the proof of the fundamental theorem of LP) $\rightsquigarrow$

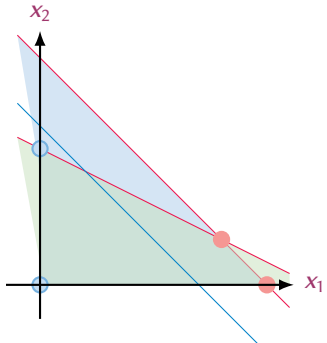
$$\mathbf{x} = \sum_{i=1}^{2} \alpha_i \mathbf{x}_i$$

$$\alpha_i \geq 0$$

$$\sum_{i=1}^{2} \alpha_i = 1$$

$$\mathbf{x}_1^T = [8, 2, 0, 0]$$
$$\mathbf{x}_2^T = [10, 0, 10, 0]$$
$$\alpha_1 = \alpha$$
$$\alpha_2 = 1 - \alpha$$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \alpha \begin{bmatrix} 8 \\ 2 \\ 0 \\ 0 \end{bmatrix} + (1 - \alpha) \begin{bmatrix} 10 \\ 0 \\ 10 \\ 0 \end{bmatrix}$$



$$x_1 = 8\alpha + 10(1 - \alpha)$$
$$x_2 = 2\alpha$$
$$x_3 = 10(1 - \alpha)$$
$$x_4 = 0$$

Degeneracy

$$\begin{array}{lrcl} \max & x_2 & & \\ & -x_1 + x_2 & \leq & 0 \\ & x_1 & \leq & 2 \\ & x_1, x_2 & \geq & 0 \end{array}$$

- Initial tableau

	x1	x2	x3	x4	-z	b
x3	-1	1	1	0	0	0
x4	1	0	0	1	0	2
	0	1	0	0	1	0

  $b_i = 0$ (one basic var. is zero) might lead to cycling

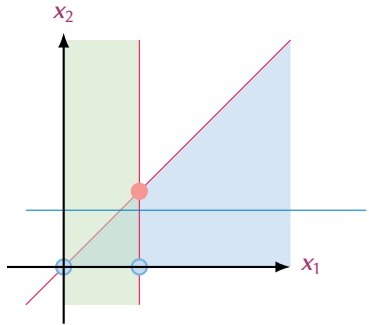
- degenerate pivot step: not improving, the entering variable stays at zero

	x1	x2	x3	x4	-z	b
	-1	1	1	0	0	0
	1	0	0	1	0	2
	1	0	-1	0	1	0

- now nondegenerate:

	x1	x2	x3	x4	-z	b
	0	1	0	1	0	2
	1	0	0	1	0	2
	0	0	-1	-1	1	-2

$x_1 = 2, x_2 = 2, z = 2$



$\geq n + 1$ constraints meet at a vertex

Def: An **improving variable** is one with positive reduced cost

Def: A **degenerate iteration** is one in which the objective function does not increase.

Def: The simplex method **cycles** if the same tableau appears in two iterations.

Degenerate conditions may appear often in practice but cycling is rare.
(see compendium for the smallest possible example)

Theorem

If the simplex fails to terminate, then it must cycle.

Proof:

- there is a finite number of basis and simplex chooses to always increase the objective value
- hence the only situation for not terminating is that a basis appears again and iterations in between are degenerate. Two tabelaux with the same basis are the same (related to uniqueness of basic solutions)

Pivot Rules

Some pivot rules can prevent the occurrence of cycling alltogether.

So far we chose an **arbitrary improving variable** to enter. Rules for breaking ties in selecting **entering** improving variables (more important than selecting leaving variables):

- **Largest Coefficient**: the improving var with largest coefficient in last row of the tableau. Original Dantzig's rule, can cycle
- **Largest increase**: absolute improvement: $\text{argmax}_j \{c_j \theta_j\}$ computationally more costly
- **Steepest edge**: the improving var that if entering in the basis moves the current basic feasible sol in a direction closest to the direction of the vector $\mathbf{c}$ (ie, maximizes the cosine of the angle between the two vectors):

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta \quad \Longrightarrow \quad \max_{\mathbf{x}_{\text{new}}} \frac{\mathbf{c}^T(\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{old}})}{\cancel{\|\mathbf{c}\|} \, \| \mathbf{x}_{\text{new}} - \mathbf{x}_{\text{old}} \|}$$

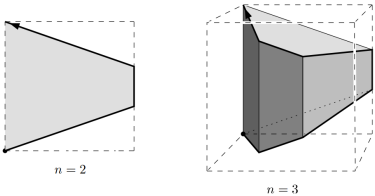
- **Bland's rule (smallest-subscript rule)** chooses the improving var with the lowest index and, if there are more than one leaving variable, the one with the lowest index.
  Prevents cycling but is slow (no smart choice for entering variable)

- **Random edge** rule selects var uniformly at random among the improving ones and the leaving ones among those selected by the ratio test

- Charnes'**perturbation method**: perturbs values of b_i terms to avoid $b_i = 0$, which must occur for cycling. To avoid cancellations: $0 < \epsilon_m \ll \epsilon_{m-1} \ll \cdots \ll \epsilon_1 \ll 1$
  It affects the choice of the leaving variable.
  It is equivalent to Dantzig-Orden-Wolfe's lexicographic formulation (imposes a strict ordering on the bases encountered by the simplex algorithm)
  It prevents cycling.

Efficiency of Simplex Method

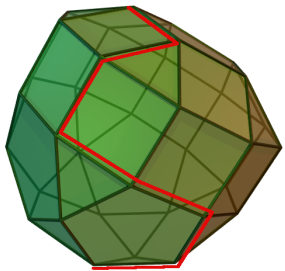
- Trying all points is $\approx 4^m$
- In practice between $2m$ and $3m$ iterations
- Klee and Minty 1978 constructed an example that requires $2^n - 1$ iterations in $\mathbb{R}^n$:



- random shuffle of indexes + lowest index for entering + lexicographic for leaving: expected iterations $< e^{C\sqrt{n \ln n}}$

Efficiency of Simplex Method

- unknown if there exists a pivot rule that leads to polynomial time.
- Clairvoyant's rule: shortest possible sequence of steps
  Hirsh conjecture $O(n-d)$ for an n-facet polytope in d-dimensional Euclidean space but best known $n^{1+\ln n}$



Smoothed Complexity

Smoothed complexity: slight random perturbations of worst-case inputs
D. Spielman and S. Teng (2001), *Smoothed analysis of algorithms: why the simplex algorithm usually takes polynomial time*

Let $\bar{A} \in \mathbb{R}^{m \times n}$ have rows of norm ≤ 1, $\bar{\mathbf{b}} \in [-1, 1]^m$, $\mathbf{c} \in \mathbb{R}^n$

Let $\hat{A}, \hat{\mathbf{b}}$ have i.i.d. $N(0, \sigma^2)$ entries

Theorem

$$\max_{\bar{A}, \bar{\mathbf{b}}, \mathbf{c}} \mathbb{E}_{\hat{A}, \hat{\mathbf{b}}} \begin{bmatrix} \textit{time to solve} \\ \textit{maximize } \mathbf{c}^T x \\ s.t. (\bar{A} + \hat{A})\mathbf{x} \leq \bar{\mathbf{b}} + \hat{\mathbf{b}} \end{bmatrix} \leq poly(m, n, \sigma^{-1}) = O(max\{n^5 \log^2 m, n^9 log^4 n, n^3 \sigma^{-4}\})$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Initial Infeasibility

$$\begin{aligned} \max\ & x_1 - x_2 \\ & x_1 + x_2 \leq 2 \\ & 2x_1 + 2x_2 \geq 5 \\ & x_1, x_2 \geq 0 \end{aligned}$$

$$\begin{aligned} \max\ & x_1 - x_2 \\ & x_1 + x_2 + x_3 = 2 \\ & -2x_1 - 2x_2 + x_4 = -5 \\ & x_1, x_2, x_3, x_4 \geq 0 \end{aligned}$$

- Initial tableau

	x1	x2	x3	x4	-z	b
x3	1	1	1	0	0	2
x4	-2	-2	0	1	0	-5
	1	-1	0	0	1	0

$\rightsquigarrow$ we do not have an initial basic feasible solution!!

In general finding any feasible solution is difficult as finding an optimal solution, otherwise we could do binary search

Auxiliary Problem (I Phase of Simplex)

We introduce auxiliary variables:

$$
\begin{aligned}
w^* = \max\ -x_5 \ \equiv\ \min x_5 & \\
x_1 + x_2 + x_3 &= 2 \\
2x_1 + 2x_2 - x_4 + x_5 &= 5 \\
x_1, x_2, x_3, x_4, x_5 &\geq 0
\end{aligned}
$$

if $w^* = 0$ then $x_5 = 0$ and the two problems are equivalent
if $w^* > 0$ then not possible to set x_5 to zero.

- Initial tableau

	x1	x2	x3	x4	x5	-z	-w	b
	1	1	1	0	0	0	0	2
	2	2	0	-1	1	0	0	5
z	1	-1	0	0	0	1	0	0
w	0	0	0	0	-1	0	1	0

  Keep z always in basis

- we reach a canonical form simply by letting x_5 enter the basis:

	x1	x2	x3	x4	x5	-z	-w	b
	1	1	1	0	0	0	0	2
	2	2	0	-1	1	0	0	5
z	1	-1	0	0	0	1	0	0
IV+II	2	2	0	-1	0	0	1	5

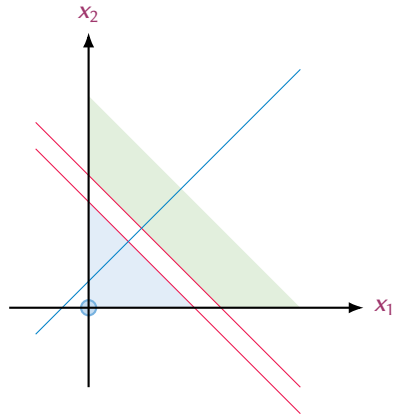
  now we have a basic feasible solution!

- x_1 enters, x_3 leaves

	x1	x2	x3	x4	x5	-z	-w	b
	1	1	1	0	0	0	0	2
II-2I'	0	0	-2	-1	1	0	0	1
III-I'	0	-2	-1	0	0	1	0	-2
IV-2I'	0	0	-2	-1	0	0	1	1

  $w^* = -1$ then no solution with $x_5 = 0$ exists then no feasible solution to initial problem

$$
\begin{array}{lrcrcl}
\max & x_1 & - & x_2 & & \\
 & x_1 & + & x_2 & \leq & 2 \\
 & 2x_1 & + & 2x_2 & \geq & 5 \\
 & & & x_1, x_2 & \geq & 0
\end{array}
$$



Initial Infeasibility - Another Example

$$\begin{array}{rrrrrl} \max & x_1 & - & x_2 & & \\ & x_1 & + & x_2 & \leq & 2 \\ & 2x_1 & + & 2x_2 & \geq & 2 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

$$\begin{array}{rrrrrrrrrl} \max & x_1 & - & x_2 & & & & & & \\ & x_1 & + & x_2 & + & x_3 & & & = & 2 \\ & 2x_1 & + & 2x_2 & & & - & x_4 & = & 2 \\ & & & & & x_1, x_2, x_3, x_4 & & & \geq & 0 \end{array}$$

Auxiliary problem (I phase):

$$\begin{array}{rrrrrrrrrl} w = \max & -x_5 & \equiv & \min x_5 & & & & & & \\ & x_1 & + & x_2 & + & x_3 & & & = & 2 \\ & 2x_1 & + & 2x_2 & & & - & x_4 + x_5 & = & 2 \\ & & & & & x_1, x_2, x_3, x_4, x_5 & & & \geq & 0 \end{array}$$

- Initial tableau

	x1	x2	x3	x4	x5	-z	-w	b
	1	1	1	0	0	0	0	2
	2	2	0	-1	1	0	0	2
z	1	-1	0	0	0	1	0	0
w	0	0	0	0	-1	0	1	0

$\rightsquigarrow$ we do not have an initial basic feasible solution.

- set in canonical form:

	x1	x2	x3	x4	x5	-z	-w	b
	1	1	1	0	0	0	0	2
	2	2	0	-1	1	0	0	2
z	1	-1	0	0	0	1	0	0
IV+II	2	2	0	-1	0	0	1	2

- x_1 enters, x_5 leaves

	x1	x2	x3	x4	x5	-z	-w	b
	0	0	1	1/2	-1/2	0	0	1
	1	1	0	-1/2	1/2	0	0	1
z	0	-2	0	1/2	-1/2	1	0	-1
w	0	0	0	0	-1	0	1	0

$w^* = 0$ hence $x_5 = 0$ we have a starting feasible solution for the initial problem.

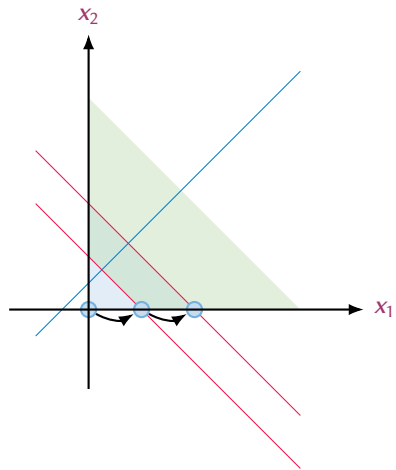
- (II phase) We keep only what we need:

	x1	x2	x3	x4	-z	b
	0	0	1	1/2	0	1
	1	1	0	-1/2	0	1
z	0	-2	0	1/2	1	-1

	x1	x2	x3	x4	-z	b
	0	0	2	1	0	2
	1	1	1	0	0	2
z	0	-2	-1	0	1	-2

  Optimal solution: $x_1 = 2, x_2 = 0, x_3 = 0, x_4 = 2$, $z = 2$.

$$
\begin{array}{lrcrcl}
\max & x_1 & - & x_2 & & \\
 & x_1 & + & x_2 & \leq & 2 \\
 & 2x_1 & + & 2x_2 & \geq & 2 \\
 & & & x_1, x_2 & \geq & 0
\end{array}
$$



In Dictionary Form

$$\begin{aligned} \max \quad & x_1 - x_2 \\ & x_1 + x_2 \leq 2 \\ & 2x_1 + 2x_2 \geq 5 \\ & x_1, x_2 \geq 0 \end{aligned}$$

$$\begin{aligned} x_3 &= 2 - x_1 - x_2 \\ x_4 &= -5 + 2x_1 + 2x_2 \\ \hline z &= x_1 + x_2 \end{aligned}$$

sol. infeasible

We introduce corrections of infeasibility

$$\begin{aligned} \max \quad -x_0 \equiv \min \quad & x_0 \\ & x_1 + x_2 \leq 2 \\ & 2x_1 + 2x_2 - x_0 \geq 5 \\ & x_1, x_2, x_0 \geq 0 \end{aligned}$$

$$\begin{aligned} x_3 &= 2 - x_1 - x_2 \\ x_4 &= -5 + 2x_1 + 2x_2 + x_0 \\ \hline z &= -x_0 \end{aligned}$$

It is still infeasible but it can be made feasible by letting x_0 enter the basis
which variable should leave?
the most infeasible: the var with the b term whose negative value has the largest magnitude

Simplex: Exception Handling, Summary

Solution of an LP problem:

a. $F \neq \emptyset$ and $\nexists$ solution

b. $F \neq \emptyset$ and $\exists$ solution

   i) **one solution**

   ii) infinite solutions

c. $F = \emptyset$

Handling exceptions in the Simplex Method

1. Unboundedness
2. More than one solution
3. Degeneracies
   - benign
   - cycling
4. Infeasible starting
   Phase I + Phase II

Summary

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Dual Problem

Dual variables $\mathbf{y}$ in one-to-one correspondence with the constraints:

Primal problem:

$$\begin{aligned} \max \quad z &= \mathbf{c}^T\mathbf{x} \\ A\mathbf{x} &\leq \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0} \end{aligned}$$

Dual Problem:

$$\begin{aligned} \min \quad w &= \mathbf{b}^T\mathbf{y} \\ A^T\mathbf{y} &\geq \mathbf{c} \\ \mathbf{y} &\geq \mathbf{0} \end{aligned}$$

Bounding approach

$$
\begin{array}{rrrrrrrrr}
z^* = & \max & 4x_1 & + & x_2 & + & 3x_3 & & \\
 & & x_1 & + & 4x_2 & & & \leq & 1 \\
 & & 3x_1 & + & x_2 & + & x_3 & \leq & 3 \\
 & & & & & & x_1, x_2, x_3 & \geq & 0
\end{array}
$$

a feasible solution is a **lower bound** but how good?
By tentatives:

$$(x_1, x_2, x_3) = (1, 0, 0) \rightsquigarrow z^* \geq 4$$
$$(x_1, x_2, x_3) = (0, 0, 3) \rightsquigarrow z^* \geq 9$$

What about **upper bounds**?

$$
\begin{array}{rcrcrcl}
 & & 2 \cdot (& x_1 + 4x_2 &) & \leq & 2 \cdot 1 \\
 & & +3 \cdot (& 3x_1 + x_2 + x_3 &) & \leq & 3 \cdot 3 \\
\hline
4x_1 + x_2 + 3x_3 & \leq & & 11x_1 + 11x_2 + 3x_3 & & \leq & 11
\end{array}
$$

$$\mathbf{c}^T\mathbf{x} \leq \mathbf{y}^T A\mathbf{x} \leq \mathbf{y}^T\mathbf{b}$$

Hence $z^* \leq 11$. Is this the best upper bound we can find?

multipliers $y_1, y_2 \geq 0$ that preserve sign of inequality

$$\begin{array}{rcl} y_1 \cdot (\ x_1 + 4x_2 \quad) & \leq & y_1(1) \\ y_2 \cdot (\ 3x_1 + x_2 + x_3) & \leq & y_2(3) \\ \hline (y_1 + 3y_2)x_1 + (4y_1 + y_2)x_2 + y_2x_3 & \leq & y_1 + 3y_2 \end{array}$$

Coefficients

$$\begin{array}{rcl} y_1 + 3y_2 & \geq & 4 \\ 4y_1 + y_2 & \geq & 1 \\ y_2 & \geq & 3 \end{array}$$

$z = 4x_1 + x_2 + 3x_3 \leq (y_1 + 3y_2)x_1 + (4y_1 + y_2)x_2 + y_2x_3 \leq y_1 + 3y_2$
then to attain the best upper bound:

$$\begin{array}{rrcl} \min & y_1 + 3y_2 & & \\ & y_1 + 3y_2 & \geq & 4 \\ & 4y_1 + y_2 & \geq & 1 \\ & y_2 & \geq & 3 \\ & y_1, y_2 & \geq & 0 \end{array}$$

Multipliers Approach

$$
\begin{array}{c|c:ccc:ccc:c:c}
\pi_1 & a_{11} & a_{12} & \dots & a_{1n} & a_{1,n+1} & a_{1,n+2} & \dots & a_{1,m+n} & 0 & b_1 \\
\vdots & \vdots & \ddots & & & & & & & & \\
\pi_m & a_{m1} & a_{m2} & \dots & a_{mn} & a_{m,n+1} & a_{m,n+2} & \dots & a_{m,m+n} & 0 & b_m \\
\hdashline
\pi_{m+1} & c_1 & c_2 & \dots & c_n & 0 & 0 & \dots & 0 & 1 & 0
\end{array}
$$

Working columnwise, since at optimum $\bar{c}_k \leq 0$ for all $k = 1, \dots, n+m$:

$$
\begin{cases}
\pi_1 a_{11} + \pi_2 a_{21} \dots + \pi_m a_{m1} + \pi_{m+1} c_1 \leq 0 \\
\vdots \quad \ddots \qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \vdots \\
\pi_1 a_{1n} + \pi_2 a_{2n} \dots + \pi_m a_{mn} + \pi_{m+1} c_n \leq 0 \\
\hdashline
\pi_1 a_{1,n+1}, \quad \pi_2 a_{2,n+1}, \quad \dots \quad \pi_m a_{m,n+1} \qquad\qquad \leq 0 \\
\vdots \qquad\qquad \vdots \qquad\quad \vdots \qquad\quad \vdots \qquad\qquad\quad \vdots \quad \vdots \\
\pi_1 a_{1,n+m}, \quad \pi_2 a_{2,n+m}, \quad \dots \quad \pi_m a_{m,n+m} \qquad\qquad \leq 0 \\
\hdashline
\pi_{m+1} = 1 \\
\hdashline
\pi_1 b_1 + \pi_2 b_2 \dots + \pi_m b_m \qquad\qquad (\leq 0)
\end{cases}
$$

(from the last row we have also $z = -\boldsymbol{\pi}\mathbf{b}$)

$$
\begin{array}{rccccccc}
-z = & \pi_1 b_1 & + & \pi_2 b_2 & \dots & + & \pi_m b_m & \\
 & \pi_1 a_{11} & + & \pi_2 a_{21} & \dots & + & \pi_m a_{m1} & \leq & -c_1 \\
 & \vdots & & \ddots & & & & & \vdots \\
 & \pi_1 a_{1n} & + & \pi_2 a_{2n} & \dots & + & \pi_m a_{mn} & \leq & -c_n \\
 & & & & & & \pi_1, \pi_2, \dots \pi_m & \leq & 0
\end{array}
$$

$\mathbf{y} = -\boldsymbol{\pi}$

$$
\begin{array}{rccccccc}
-z = & (-y_1 b_1) & + & (-y_2 b_2) & \dots & + & (-y_m b_m) & \\
 & (-y_1 a_{11}) & + & (-y_2 a_{21}) & \dots & + & (-y_m a_{m1}) & \leq & -c_1 \\
 & \vdots & & \ddots & & & & & \vdots \\
 & (-y_1 a_{1n}) & + & (-y_2 a_{2n}) & \dots & + & (-y_m a_{mn}) & \leq & -c_n \\
 & & & & & & -y_1, -y_2, \dots - y_m & \leq & 0
\end{array}
$$

as we will see $\mathbf{b}^T\mathbf{y} \geq \mathbf{c}^T\mathbf{x}$, hence it is more interesting to minimize. It yields:

$$
\begin{array}{ll}
\min & \mathbf{b}^T\mathbf{y} \\
 & A^T\mathbf{y} \geq \mathbf{c} \\
 & \mathbf{y} \geq \mathbf{0}
\end{array}
$$

Example

$$\begin{array}{rrrcl} \max & 6x_1 & + & 8x_2 & \\ & 5x_1 & + & 10x_2 & \leq 60 \\ & 4x_1 & + & 4x_2 & \leq 40 \\ & & & x_1, x_2 & \geq 0 \end{array}$$

$$\begin{cases} 5\pi_1 & + & 4\pi_2 & + & 6\pi_3 & \leq & 0 \\ 10\pi_1 & + & 4\pi_2 & + & 8\pi_3 & \leq & 0 \\ 1\pi_1 & + & 0\pi_2 & + & 0\pi_3 & \leq & 0 \\ 0\pi_1 & + & 1\pi_2 & + & 0\pi_3 & \leq & 0 \\ 0\pi_1 & + & 0\pi_2 & + & 1\pi_3 & = & 1 \\ 60\pi_1 & + & 40\pi_2 & & & & \end{cases}$$

$$\begin{array}{rcl} y_1 & = & -\pi_1 \geq 0 \\ y_2 & = & -\pi_2 \geq 0 \end{array}$$

...

Duality Recipe

	Primal linear program	Dual linear program
Variables	$x_1, x_2, \ldots, x_n$	$y_1, y_2, \ldots, y_m$
Matrix	A	A^T
Right-hand side	$\mathbf{b}$	$\mathbf{c}$
Objective function	$\max \mathbf{c}^T\mathbf{x}$	$\min \mathbf{b}^T\mathbf{y}$
Constraints	ith constraint has $\leq$ $\geq$ $=$	$y_i \geq 0$ $y_i \leq 0$ $y_i \in \mathbb{R}$
	$x_j \geq 0$ $x_j \leq 0$ $x_j \in \mathbb{R}$	jth constraint has $\geq$ $\leq$ $=$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Symmetry

The dual of the dual is the primal:

Primal Problem:

$$\begin{aligned} \max \quad z &= c^T x \\ Ax &\leq b \\ x &\geq 0 \end{aligned}$$

Dual Problem:

$$\begin{aligned} \min \quad w &= b^T y \\ A^T y &\geq c \\ y &\geq 0 \end{aligned}$$

Let's put the dual in the standard form

Dual Problem:

$$\begin{aligned} \min \quad b^T y &\equiv -\max -b^T y \\ -A^T y &\leq -c \\ y &\geq 0 \end{aligned}$$

Dual of Dual:

$$\begin{aligned} -\min \quad -c^T x & \\ -Ax &\geq -b \\ x &\geq 0 \end{aligned}$$

Weak Duality Theorem

As we saw the dual produces upper bounds. This is true in general:

Theorem (Weak Duality Theorem)

Given:

$$\begin{aligned} &(P)\ \max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\} \\ &(D)\ \min\{\mathbf{b}^T\mathbf{y} \mid A^T\mathbf{y} \geq \mathbf{c}, \mathbf{y} \geq \mathbf{0}\} \end{aligned}$$

for any feasible solution $\mathbf{x}$ *of (P) and any feasible solution* $\mathbf{y}$ *of (D):*

$$\mathbf{c}^T\mathbf{x} \leq \mathbf{b}^T\mathbf{y}$$

Proof:
From (D) $c_j \leq \sum_{i=1}^{m} y_i a_{ij}\ \forall j$ and from (P) $\sum_{j=1}^{n} a_{ij} x_j \leq b_i\ \forall i$
From (D) $y_i \geq 0$ and from (P) $x_j \geq 0$

$$\sum_{j=1}^{n} c_j x_j \leq \sum_{j=1}^{n} \left(\sum_{i=1}^{m} y_i a_{ij} \right) x_j = \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} x_j \right) y_i \leq \sum_{i=1}^{m} b_i y_i$$

Strong Duality Theorem

Due to Von Neumann and Dantzig 1947 and Gale, Kuhn and Tucker 1951.

Theorem (Strong Duality Theorem)

Given:

$$\begin{aligned} &(P)\ \max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\} \\ &(D)\ \min\{\mathbf{b}^T\mathbf{y} \mid A^T\mathbf{y} \geq \mathbf{c}, \mathbf{y} \geq \mathbf{0}\} \end{aligned}$$

exactly one of the following occurs:

1. *(P) and (D) are both infeasible*
2. *(P) is unbounded and (D) is infeasible*
3. *(P) is infeasible and (D) is unbounded*
4. *(P) has feasible solution, then let an optimal be:* $\mathbf{x}^* = [x_1^*, \ldots, x_n^*]$
   (D) has feasible solution, then let an optimal be: $\mathbf{y}^* = [y_1^*, \ldots, y_m^*]$*, then:*

$$\mathbf{c}^T\mathbf{x}^* = \mathbf{b}^T\mathbf{y}^*$$

Proof:

- all other combinations of 3 possibilities (Optimal, Infeasible, Unbounded) for (P) and 3 for (D) are ruled out by weak duality theorem.
- we use the simplex method. (Other proofs independent of the simplex method exist, eg, Farkas Lemma and convex polyhedral analysis)
- The last row of the final tableau will give us that for every $\mathbf{x}$:

$$z = z^* + \sum_{k=1}^{n+m} \bar{c}_k x_k = z^* + \sum_{j=1}^{n} \bar{c}_j x_j + \sum_{i=1}^{m} \bar{c}_{n+i} x_{n+i} \qquad (*)$$

$$= z^* + \bar{\mathbf{c}}_B \mathbf{x}_B + \bar{\mathbf{c}}_N \mathbf{x}_N$$

In addition, $z^* = \sum_{j=1}^{n} c_j x_j^*$ (c_j, original values) because optimal value

- We define $\boxed{y_i^* = -\bar{c}_{n+i},} \quad i = 1, 2, \ldots, m$
- We claim that $(y_1^*, y_2^*, \ldots, y_m^*)$ is a dual feasible solution satisfying $\mathbf{c}^T \mathbf{x}^* = \mathbf{b}^T \mathbf{y}^*$.

- Let's verify the claim:
  We substitute in (*): i) $z = \sum_{j=1}^{n} c_j x_j$; ii) $\bar{c}_{n+i} = -y_i^*$; and iii) $x_{n+i} = b_i - \sum_{j=1}^{n} a_{ij} x_j$ for $i = 1, 2, \ldots, m$ ($n + i$ are the slack variables)

$$\sum_{j=1}^{n} c_j x_j = z^* + \sum_{j=1}^{n} \bar{c}_j x_j - \sum_{i=1}^{m} y_i^* \left(b_i - \sum_{j=1}^{n} a_{ij} x_j \right)$$

$$= \left(z^* - \sum_{i=1}^{m} y_i^* b_i \right) + \sum_{j=1}^{n} \left(\bar{c}_j + \sum_{i=1}^{m} a_{ij} y_i^* \right) x_j$$

This must hold for every $(x_1, x_2, \ldots, x_n)$ hence:

$$z^* = \sum_{i=1}^{m} b_i y_i^* \qquad \Longrightarrow \; y^* \text{ satisfies } c^T x^* = b^T y^*$$

$$c_j = \bar{c}_j + \sum_{i=1}^{m} a_{ij} y_i^*, j = 1, 2, \ldots, n$$

Since $\overline{c}_k \leq 0$ for every $k = 1, 2, \ldots, n+m$:

$$\overline{c}_j \leq 0 \rightsquigarrow \quad c_j - \sum_{i=1}^{m} y_i^* a_{ij} \leq 0 \rightsquigarrow \quad \sum_{i=1}^{m} y_i^* a_{ij} \geq c_j \quad j = 1, 2, \ldots, n$$

$$\overline{c}_{n+i} \leq 0 \rightsquigarrow \quad y_i^* = -\overline{c}_{n+i} \geq 0, \quad i = 1, 2, \ldots, m$$

$\Longrightarrow$ y^* **is also dual feasible solution**

Complementary Slackness Theorem

Theorem (Complementary Slackness)

A feasible solution $\mathbf{x}^*$ *for (P)*
A feasible solution $\mathbf{y}^*$ *for (D)*
Necessary and sufficient conditions for optimality of both:

$$\left(c_j - \sum_{i=1}^{m} y_i^* a_{ij}\right) x_j^* = 0, \quad j = 1, \ldots, n$$

If $x_j^* \neq 0$ *then* $\sum y_i^* a_{ij} = c_j$ *(no surplus)*
If $\sum y_i^* a_{ij} > c_j$ *then* $x_j^* = 0$

Proof:

$$z^* = \mathbf{c}^T \mathbf{x}^* \leq \mathbf{y}^* A \mathbf{x}^* \leq \mathbf{b}^T \mathbf{y}^* = w^*$$

Hence from strong duality theorem:

$$\mathbf{c}\mathbf{x}^* - \mathbf{y}^* A \mathbf{x}^* = \mathbf{0}$$

In scalars

$$\sum_{j=1}^{n} \underbrace{\left(c_j - \sum_{i=1}^{m} y_i^* a_{ij}\right)}_{\leq 0} \underbrace{x_j^*}_{\geq 0} = 0$$

Hence each term must be $= 0$

Proof in scalar form:

$$c_j x_j^* \leq \left(\sum_{i=1}^{m} a_{ij} y_i^* \right) x_j^* \quad j = 1, 2, \ldots, n \quad \text{from feasibility in D}$$

$$\left(\sum_{j=1}^{n} a_{ij} x_j^* \right) y_i^* \leq b_i y_i^* \quad i = 1, 2, \ldots, m \quad \text{from feasibility in P}$$

Summing in j and in i:

$$\sum_{j=1}^{n} c_j x_j^* \leq \sum_{j=1}^{n} \left(\sum_{i=1}^{m} a_{ij} y_i^* \right) x_j^* = \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} x_j^* \right) y_i^* \leq \sum_{i=1}^{m} b_i y_i^*$$

For the strong duality theorem the left hand side is equal to the right hand side and hence all inequalities become equalities.

$$\sum_{j=1}^{n} \underbrace{\left(c_j - \sum_{i=1}^{m} y_i^* a_{ij} \right)}_{\leq 0} \underbrace{x_j^*}_{\geq 0} = 0$$

Economic Interpretation of Duality Theory

$$
\begin{array}{lrcrcrcl}
\max & 5x_0 & + & 6x_1 & + & 8x_2 & & \\
 & 6x_0 & + & 5x_1 & + & 10x_2 & \leq & 60 \\
 & 8x_0 & + & 4x_1 & + & 4x_2 & \leq & 40 \\
 & 4x_0 & + & 5x_1 & + & 6x_2 & \leq & 50 \\
 & & & & & x_0, x_1, x_2 & \geq & 0
\end{array}
$$

final tableau:

$x0$	$x1$	$x2$	$s1$	$s2$	$s3$	$-z$	b
	0	1		0			$5/2$
	1	0		0			7
	0	0		1			2
$-1/5$	0	0	$-1/5$	0	-1		-62

- What is the maximum number of products that can be produced? at most m
- What are the values of the variables, the reduced costs, the dual variables, and the shadow (or marginal) prices?
- What is the value of additional manpower capacity (1st constr.)? In +1 out +1/5
- If one slack variable > 0 then overcapacity: $s_2 = 2$ then the second constraint is not tight
- How much more expensive would a non-selected product need to be? look at reduced costs: $c_j + \pi \mathbf{a}_j > 0$

Economic Interpretation of Duality Theory

Game: Suppose two economic operators:

- P owns the factory and produces goods
- D is in the market buying and selling raw material and resources
- D asks P to close and sell him/her all resources
- P considers if the offer is convenient
- D wants to spend the least possible
- $\mathbf{y}$ are prices that D offers for the resources
- $\sum y_i b_i$ is the amount D has to pay to have all resources of P
- $\sum y_i a_{ij} \geq c_j$ the total value to make j is greater or eq. than the price per unit of product j
- P either sells all resources $\sum y_i a_{ij}$ or produces product j (c_j)
- without $\geq$ there would not be negotiation because P would be better off producing and selling

- at optimality the situation is indifferent (strong th.)
- resource 2 that was not totally utilized in the primal has been given value 0 in the dual. (complementary slackness th.) Plausible, since we do not use all the resource, likely to place not so much value on it.
- for product 0 $\sum y_i a_{ij} > c_j$ hence not profitable producing it. (complementary slackness th.)

Duality - Summary

- Derivation:
  - Economic Interpretation
  - Bounding Approach
  - Multiplier Approach
  - Recipe
  - Lagrangian Multipliers Approach (next time)
- Theory:
  - Symmetry
  - Weak Duality Theorem
  - Strong Duality Theorem
  - Complementary Slackness Theorem

Summary

- Derivation:
  1. economic interpretation
  2. bounding
  3. multipliers
  4. recipe
  5. Lagrangian
- Theory:
  - Symmetry
  - Weak duality theorem
  - Strong duality theorem
  - Complementary slackness theorem
- Economic interpretation
- Dual Simplex
- Sensitivity Analysis

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
   - Resource Allocation
   - Duality
4. Introduction
   - Diet Problem
5. Solving LP Problems
   - Fourier-Motzkin method
6. Mathematical Programming
   - Definitions
   - Fundamental Theorem of LP
   - Gaussian Elimination
7. Simplex Method
   - Standard Form
   - Basic Feasible Solutions
   - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Lagrangian Duality

Relaxation: if a problem is hard to solve then find an easier problem resembling the original one that provides information in terms of bounds. Then, search for the strongest bounds.

$$\begin{array}{rrrrrrrrrrr} \min & 13x_1 & + & 6x_2 & + & 4x_3 & + & 12x_4 & & \\ & 2x_1 & + & 3x_2 & + & 4x_3 & + & 5x_4 & = & 7 \\ & 3x_1 & + & & + & 2x_3 & + & 4x_4 & = & 2 \\ & & & & & & & x_1, x_2, x_3, x_4 & \geq & 0 \end{array}$$

We wish to reduce to a problem easier to solve, ie:

$$\begin{array}{rrrrrrrrrrr} \min & c_1x_1 & + & c_2x_2 & + & \ldots & + & c_nx_n & & \\ & & & & & & & x_1, x_2, \ldots, x_n & \geq & 0 \end{array}$$

solvable by inspection: if $c_j < 0$ then $x_j = +\infty$, if $c_j \geq 0$ then $x_j = 0$.

Measure of violation of the constraints:

$$\begin{array}{rrrrrrrrrrr} 7 & - & (2x_1 & + & 3x_2 & + & 4x_3 & + & 5x_4) \\ 2 & - & (3x_1 & + & & + & 2x_3 & + & 4x_4) \end{array}$$

We relax these measures in obj. function with **Lagrangian multipliers** y_1, y_2.
We obtain a family of problems:

$$PR(y_1, y_2) = \min_{x_1,x_2,x_3,x_4 \geq 0} \left\{ \begin{array}{rrrrrrrr} & 13x_1 & + & 6x_2 & + & 4x_3 & + & 12x_4 \\ +y_1(7- & 2x_1 & - & 3x_2 & - & 4x_3 & - & 5x_4) \\ +y_2(2- & 3x_1 & & & - & 2x_3 & - & 4x_4) \end{array} \right\}$$

1. for all $y_1, y_2 \in \mathbb{R}$: $\mathrm{opt}(PR(y_1, y_2)) \leq \mathrm{opt}(P)$
2. $\max_{y_1,y_2 \in \mathbb{R}} \{\mathrm{opt}(PR(y_1, y_2))\} \leq \mathrm{opt}(P)$

PR is easy to solve.
(It can be also seen as a proof of the weak duality theorem)

$$PR(y_1, y_2) = \min_{x_1, x_2, x_3, x_4 \geq 0} \left\{ \begin{array}{rrrrrr} & (13 & - & 2y_1 & - & 3y_2) & x_1 \\ + & (6 & - & 3y_1 & &) & x_2 \\ + & (4 & - & 4y_1 & - & 2y_2) & x_3 \\ + & (12 & - & 5y_1 & - & 4y_2) & x_4 \\ + & & & 7y_1 & + & 2y_2 & \end{array} \right\}$$

if coefficient of x is < 0 then bound is $-\infty$ then LB is useless

$$\begin{array}{rrrrrr} (13 & - & 2y_1 & - & 3y_2) & \geq 0 \\ (6 & - & 3y_1 & &) & \geq 0 \\ (4 & - & 4y_1 & - & 2y_2) & \geq 0 \\ (12 & - & 5y_1 & - & 4y_2) & \geq 0 \end{array}$$

If they all hold then we are left with $7y_1 + 2y_2$ because all go to 0.

$$\begin{array}{lrrrr} \max & 7y_1 & + & 2y_2 & \\ & 2y_1 & + & 3y_2 & \leq 13 \\ & 3y_1 & & & \leq 6 \\ & 4y_1 & + & 2y_2 & \leq 4 \\ & 5y_1 & + & 4y_2 & \leq 12 \end{array}$$

General Formulation

$$\begin{array}{lrcll} \min & z & = & \mathbf{c}^T\mathbf{x} & \mathbf{c} \in \mathbb{R}^n \\ & A\mathbf{x} & = & \mathbf{b} & A \in \mathbb{R}^{m\times n}, \mathbf{b} \in \mathbb{R}^m \\ & \mathbf{x} & \geq & \mathbf{0} & \mathbf{x} \in \mathbb{R}^n \end{array}$$

$$\max_{\mathbf{y}\in\mathbb{R}^m}\{\min_{\mathbf{x}\in\mathbb{R}^n_+}\{\mathbf{c}^T\mathbf{x} + \mathbf{y}^T(\mathbf{b} - A\mathbf{x})\}\}$$

$$\max_{\mathbf{y}\in\mathbb{R}^m}\{\min_{\mathbf{x}\in\mathbb{R}^n_+}\{(\mathbf{c}^T - \mathbf{y}^T A)\mathbf{x} + \mathbf{y}^T\mathbf{b}\}\}$$

$$\begin{array}{lrcl} \max & \mathbf{b}^T\mathbf{y} & & \\ & A^T\mathbf{y} & \leq & \mathbf{c} \\ & \mathbf{y} \in \mathbb{R}^m & & \end{array}$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Dual Simplex

- Dual simplex (Lemke, 1954): apply the simplex method to the dual problem and observe what happens in the primal tableau:

$$\begin{aligned}\max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\} &= \min\{\mathbf{b}^T\mathbf{y} \mid A^T\mathbf{y} \geq \mathbf{c}^T, \mathbf{y} \geq \mathbf{0}\} \\ &= -\max\{-\mathbf{b}^T\mathbf{y} \mid -A^T\mathbf{y} \leq -\mathbf{c}^T, \mathbf{y} \geq \mathbf{0}\}\end{aligned}$$

- We obtain a new algorithm for the primal problem: the **dual simplex**
  It corresponds to the primal simplex applied to the dual

- We obtain also another way to handle infeasibility

Primal Simplex on Dual Problem

Example: Primal feasible start

Primal:

$$\begin{aligned} \max \quad & 4x_1 + x_2 + 3x_3 & \\ & x_1 + 4x_2 & \leq 1 \\ & 3x_1 + x_2 + x_3 & \leq 3 \\ & x_1, x_2, x_3 & \geq 0 \end{aligned}$$

- Initial tableau

x1	x2	x3	w1	w2	-z	b
1	4	0	1	0	0	1
3	1	1	0	1	0	3
4	1	3	0	0	1	0

- primal feasible start

Dual:

$$\begin{aligned} \min \quad & y_1 + 3y_2 & \\ & y_1 + 3y_2 & \geq 4 \\ & 4y_1 + y_2 & \geq 1 \\ & y_2 & \geq 3 \\ & y_1, y_2 & \geq 0 \end{aligned}$$

- Initial tableau ($\min by \equiv -\max -by$)

y1	y2	z1	z2	z3	-p	b
-1	-3	1	0	0	0	-4
-4	-1	0	0	1	0	-1
0	-1	0	0	1	0	-3
-1	-3	0	0	0	1	0

- dual infeasible start

Primal Simplex on Dual Problem

Example: Primal infeasible start

Primal:

$$\begin{aligned} \max \quad & -x_1 - x_2 \\ & -2x_1 - x_2 \leq 4 \\ & -2x_1 + 4x_2 \leq -8 \\ & -x_1 + 3x_2 \leq -7 \\ & x_1, x_2 \geq 0 \end{aligned}$$

- Initial tableau

	x1	x2	w1	w2	w3	-z	b
	-2	-1	1	0	0	0	4
	-2	4	0	1	0	0	-8
	-1	3	0	0	1	0	-7
	-1	-1	0	0	0	1	0

infeasible start

- x_1 enters, w_2 leaves

Dual:

$$\begin{aligned} \min \quad & 4y_1 - 8y_2 - 7y_3 \\ & -2y_1 - 2y_2 - y_3 \geq -1 \\ & -y_1 + 4y_2 + 3y_3 \geq -1 \\ & y_1, y_2, y_3 \geq 0 \end{aligned}$$

- Initial tableau ($\min by \equiv -\max -by$)

	y1	y2	y3	z1	z2	-p	b
	2	2	1	1	0	0	1
	1	-4	-3	0	1	0	1
	-4	8	7	0	0	1	0

feasible start (thanks to $-x_1 - x_2$)

- y_2 enters, z_1 leaves

- x_1 enters, w_2 leaves

	x1	x2	w1	w2	w3	-z	b
	0	-5	1	-1	0	0	12
	1	-2	0	-0.5	0	0	4
	0	1	0	-0.5	1	0	-3
	0	-3	0	-0.5	0	1	4

- w_2 enters, w_3 leaves

	x1	x2	w1	w2	w3	-z	b
	0	-7	1	0	-2	0	18
	1	-3	0	0	-1	0	7
	0	-2	0	1	-2	0	6
	0	-4	0	0	-1	1	7

(note that we kept $c_j < 0$, ie, optimality)

- y_2 enters, z_1 leaves

	y1	y2	y3	z1	z2	-p	b
	1	1	0.5	0.5	0	0	0.5
	5	0	-1	2	1	0	3
	-4	0	3	-12	0	1	-4

- y_3 enters, y_2 leaves

	y1	y2	y3	z1	z2	-p	b
	2	2	1	1	0	0	1
	7	2	0	3	1	0	3
	-18	-6	0	-7	0	1	-7

Dual Simplex on Primal Problem

Primal simplex on primal problem:

1. pivot > 0
2. col c_j with wrong sign
3. row: $\min \left\{ \frac{b_i}{a_{ij}} : a_{ij} > 0, i = 1, .., m \right\}$

Dual simplex on primal problem:

1. pivot < 0
2. row $b_i < 0$
   (condition of feasibility)
3. col: $\min \left\{ \left| \frac{c_j}{a_{ij}} \right| : a_{ij} < 0, j = 1, 2, .., n + m \right\}$
   (least worsening solution)

- Primal works with feasible solutions towards optimality
- Dual works with optimal solutions towards feasibility

Dual Simplex

1. (primal) simplex on primal problem (the one studied so far)

2. Now: dual simplex on primal problem $\equiv$ primal simplex on dual problem
   (implemented as dual simplex, understood as primal simplex on dual problem)

Uses of the Dual Simplex:

- Infeasible start
  Dual based Phase I algorithm (**Dual-primal algorithm**)

- The dual simplex can work better than the primal in some cases.
  Eg. since running time in practice between $2m$ and $3m$, then if $m = 99$ and $n = 9$ then better the dual

Dual based Phase I

Example:

$$\begin{aligned} \text{maximize} \quad & z = x_1 - x_2 \\ \text{subject to} \quad & x_1 + x_2 \leq 2 \\ & 2x_1 + 2x_2 \geq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Summary

- Derivation:
  1. bounding
  2. multipliers
  3. recipe
  4. Lagrangian
- Theory:
  - Symmetry
  - Weak duality theorem
  - Strong duality theorem
  - Complementary slackness theorem
- Dual Simplex
- Sensitivity Analysis, Economic interpretation

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Sensitivity Analysis
aka Postoptimality Analysis

Instead of solving each modified problem from scratch, exploit results obtained from solving the original problem.

$$\max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}\} \qquad (*)$$

(I) changes to coefficients of objective function: $\max\{\tilde{\mathbf{c}}^T\mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}\}$ (primal)
$\mathbf{x}^*$ of (*) remains feasible hence we can restart the simplex from $\mathbf{x}^*$

(II) changes to RHS terms: $\max\{\mathbf{c}^T\mathbf{x} \mid A\mathbf{x} = \tilde{\mathbf{b}}, \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}\}$ (dual)
$\mathbf{x}^*$ optimal feasible solution of (*)
basic sol $\bar{\mathbf{x}}$ of (II): $\bar{\mathbf{x}}_N = \mathbf{x}^*_N$, $A_B\bar{\mathbf{x}}_B = \tilde{\mathbf{b}} - A_N\bar{\mathbf{x}}_N$
$\bar{\mathbf{x}}$ is dual feasible and we can start the dual simplex from there. If $\tilde{\mathbf{b}}$ differs from $\mathbf{b}$ only slightly it may be we are already optimal.

(III) introduce a new variable: (primal)

$$\max \quad \sum_{j=1}^{6} c_j x_j$$

$$\sum_{j=1}^{6} a_{ij} x_j = b_i, \ i = 1, \ldots, 3$$

$$l_j \le x_j \le u_j, \ j = 1, \ldots, 6$$

$[x_1^*, \ldots, x_6^*]$ feasible

$$\max \quad \sum_{j=1}^{7} c_j x_j$$

$$\sum_{j=1}^{7} a_{ij} x_j = b_i, \ i = 1, \ldots, 3$$

$$l_j \le x_j \le u_j, \ j = 1, \ldots, 7$$

$[x_1^*, \ldots, x_6^*, 0]$ feasible

(IV) introduce a new constraint: (dual)

$$\sum_{j=1}^{6} a_{4j} x_j = b_4$$

$$\sum_{j=1}^{6} a_{5j} x_j = b_5$$

$$l_j \le x_j \le u_j \qquad j = 7, 8$$

$[x_1^*, \ldots, x_6^*]$ optimal

$[x_1^*, \ldots, x_6^*, x_7^*, x_8^*]$dual feasible

$$x_7^* = b_4 - \sum_{j=1}^{6} a_{4j} x_j^*$$

$$x_8^* = b_5 - \sum_{j=1}^{6} a_{5j} x_j^*$$

Examples

(I) Variation of reduced costs:

$$\begin{array}{rrrcl} \max & (6+\rho)x_1 & + & 8x_2 & \\ & 5x_1 & + & 10x_2 & \leq 60 \\ & 4x_1 & + & 4x_2 & \leq 40 \\ & & & x_1, x_2 & \geq 0 \end{array}$$

The last tableau gives the possibility to estimate the effect of variations

	x_1	x_2	x_3	x_4	$-z$	b
x_3	5	10	1	0	0	60
x_4	4	4	0	1	0	40
	6	8	0	0	1	0

	x_1	x_2	x_3	x_4	$-z$	b
x_2	0	1	$1/5$	$-1/4$	0	2
x_1	1	0	$-1/5$	$1/2$	0	8
	0	0	$-2/5$	-1	1	-64

- For a variable in basis the perturbation goes unchanged in the red. costs. Eg:

$$\max (6+\delta)x_1 + 8x_2 \implies \bar{c}_1 = 1(6+\delta) - \frac{2}{5}\cdot 5 - 1\cdot 4 = \delta$$

  then need to bring in canonical form yielding changes in the reduced cost of other variables (might need to iterate the primal simplex) and in the obj value.

- For a variable not in basis, if it changes the sign of the reduced cost $\implies$ worth bringing in basis $\implies$ the δ term propagates to other columns

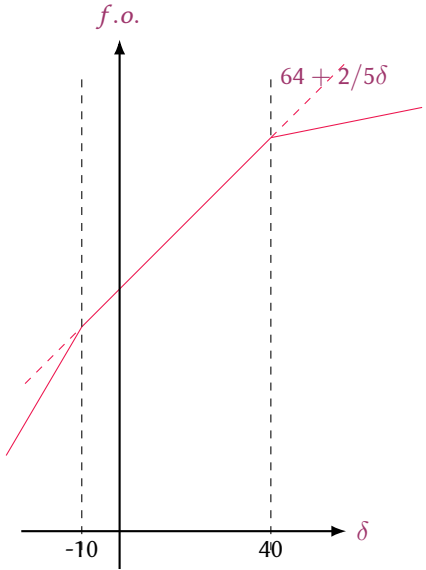
(II) Changes in RHS terms

	x_1	x_2	x_3	x_4	$-z$	b
x_3	5	10	1	0	0	$60+\delta$
x_4	4	4	0	1	0	$40+\epsilon$
	6	8	0	0	1	0

	x_1	x_2	x_3	x_4	$-z$	b
x_2	0	1	$1/5$	$-1/4$	0	$2+1/5\delta-1/4\epsilon$
x_1	1	0	$-1/5$	$1/2$	0	$8-1/5\delta+1/2\epsilon$
	0	0	$-2/5$	-1	1	$-64-2/5\delta-\epsilon$

(It would be more convenient to augment the second. But let's take $\epsilon = 0$.)
If $60+\delta \implies$ all RHS terms change and we must check feasibility
Which are the multipliers for the first row? $k_1 = \frac{1}{5}, k_2 = -\frac{1}{4}, k_3 = 0$
I: $1/5(60+\delta) - 1/4 \cdot 40 + 0 \cdot 0 = 12 + \delta/5 - 10 = 2 + \delta/5$
II: $-1/5(60+\delta) + 1/2 \cdot 40 + 0 \cdot 0 = -60/5 + 20 - \delta/5 = 8 - 1/5\delta$
Risk that RHS becomes negative
Eg: if $\delta = -10 \implies$ tableau stays optimal but not feasible $\implies$ apply dual simplex

Graphical Representation



(III) Add a variable

$$\begin{array}{rrcrcrcl} \max & 5x_0 & + & 6x_1 & + & 8x_2 & & \\ & 6x_0 & + & 5x_1 & + & 10x_2 & \leq & 60 \\ & 8x_0 & + & 4x_1 & + & 4x_2 & \leq & 40 \\ & & & & & x_0, x_1, x_2 & \geq & 0 \end{array}$$

Reduced cost of x_0? $c_j + \sum \pi_i a_{ij} = +1 \cdot 5 - \frac{2}{5} \cdot 6 + (-1)8 = -\frac{27}{5}$

To make worth entering in basis:

- increase its cost coefficient
- decrease the technological coefficient in constraint II: $5 - 2/5 \cdot 6 - a_{20} > 0$

(IV) Add a constraint

$$\begin{array}{rrrrrl} \max & 6x_1 & + & 8x_2 & & \\ & 5x_1 & + & 10x_2 & \leq & 60 \\ & 4x_1 & + & 4x_2 & \leq & 40 \\ & 5x_1 & + & 6x_2 & \leq & 50 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

Final tableau not in canonical form, need to iterate with dual simplex

	x_1	x_2	x_3	x_4	x_5	$-z$	b
x_2	0	1	$1/5$	$-1/4$		0	2
x_1	1	0	$-1/5$	$1/2$		0	8
	0	0	$-1/5$	-1	1	0	-2
	0	0	$-2/5$	-1	0	1	-64

(V) change in a technological coefficient:

	x_1	x_2	x_3	x_4	$-z$	b
x_3	5	$10+\delta$	1	0	0	60
x_4	4	4	0	1	0	40
	6	8	0	0	1	0

- first effect on its column
- then look at c
- finally look at b

	x_1	x_2	x_3	x_4	$-z$	b
x_2	0	$(10+\delta)1/5+4(-1/4)$	$1/5$	$-1/4$	0	2
x_1	1	$(10+\delta)(-1/5)+4(1/2)$	$-1/5$	$1/2$	0	8
	0	$-2/5\delta$	$-2/5$	-1	1	-64

Relevance of Sensistivity Analysis

- The dominant application of LP is mixed integer linear programming.
- In this context it is extremely important being able to begin with a model instantiated in one form followed by a sequence of problem modifications
  - row and column additions and deletions,
  - variable fixings

  interspersed with resolves

Summary

- Derivation:
  1. economic interpretation
  2. bounding
  3. multipliers
  4. recipe
  5. Lagrangian
- Theory:
  - Symmetry
  - Weak duality theorem
  - Strong duality theorem
  - Complementary slackness theorem
- Economic interpretation
- Dual Simplex
- Sensitivity Analysis

Midway Evaluation

https://pollev-embeds.com/surveys/Y9ZAhgIUlEpmi2E2ux7qu/respond



Motivation

Complexity of single pivot operation in standard simplex:

- entering variable $O(n)$
- leaving variable $O(m)$
- updating the tableau $O(mn)$

Problems with this:

- Time: we are doing operations that are not actually needed
  Space: we need to store the whole tableau: $O(mn)$ floating point numbers
- Most problems have sparse matrices (many zeros)
  sparse matrices are typically handled efficiently
  the standard simplex has the 'Fill in' effect: sparse matrices are lost
- accumulation of Floating Point Errors over the iterations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Revised Simplex Method

Several ways to improve wrt pitfalls in the previous slide, requires matrix description of the simplex.

$$\begin{aligned} \max \quad & \sum_{j=1}^{n} c_j x_j \\ & \sum_{j=1}^{n} a_{ij} x_j \leq b_i \quad i = 1..m \\ & x_j \geq 0 \quad j = 1..n \end{aligned}$$

$$\begin{aligned} \max \quad & \mathbf{c}^T \mathbf{x} \\ & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

$A \in \mathbb{R}^{m \times (n+m)}$

$\mathbf{c} \in \mathbb{R}^{(n+m)}, \mathbf{b} \in \mathbb{R}^m, \mathbf{x} \in \mathbb{R}^{n+m}$

$$\max\{\mathbf{c}^T \mathbf{x} \mid A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$$

At each iteration the simplex moves from a basic feasible solution to another.

For each basic feasible solution:

- $N = \{1 \ldots n\}$
- $B = \{n+1 \ldots n+m\}$ basis
- $A_N = [\mathbf{a}_1 \ldots \mathbf{a}_n]$
- $A_B = [\mathbf{a}_{n+1} \ldots \mathbf{a}_{n+m}]$ basis matrix
- $\mathbf{x}_N = 0$
- $\mathbf{x}_B \geq 0$

$$\left[\begin{array}{c|c|c|c} A_N & A_B & \mathbf{0} & \mathbf{b} \\ \hline \mathbf{c}_N^T & \mathbf{c}_B^T & 1 & 0 \end{array}\right]$$

$$A\mathbf{x} = A_N\mathbf{x}_N + A_B\mathbf{x}_B = \mathbf{b}$$
$$A_B\mathbf{x}_B = \mathbf{b} - A_N\mathbf{x}_N$$

Basic feasible solution $\iff$ A_B is non-singular

$$\mathbf{x}_B = A_B^{-1}\mathbf{b} - A_B^{-1}A_N\mathbf{x}_N$$

for the objective function:

$$z = \mathbf{c}^T\mathbf{x} = \mathbf{c}_B^T\mathbf{x}_B + \mathbf{c}_N^T\mathbf{x}_N$$

Substituting for $\mathbf{x}_B$ from above:

$$\begin{aligned} z &= \mathbf{c}_B^T(A_B^{-1}\mathbf{b} - A_B^{-1}A_N\mathbf{x}_N) + \mathbf{c}_N^T\mathbf{x}_N = \\ &= \mathbf{c}_B^TA_B^{-1}\mathbf{b} + (\mathbf{c}_N^T - \mathbf{c}_B^TA_B^{-1}A_N)\mathbf{x}_N \end{aligned}$$

Collecting together:

$$\begin{aligned} \mathbf{x}_B &= A_B^{-1}\mathbf{b} - A_B^{-1}A_N\mathbf{x}_N \\ z &= \mathbf{c}_B^TA_B^{-1}\mathbf{b} + (\mathbf{c}_N^T - \mathbf{c}_B^T\underbrace{A_B^{-1}A_N}_{\bar{A}})\mathbf{x}_N \end{aligned}$$

In tableau form, for a basic feasible solution corresponding to B we have:

$$\left[\begin{array}{c|c|c|c} A_B^{-1}A_N & I & \mathbf{0} & A_B^{-1}\mathbf{b} \\ \hline \mathbf{c}_N^T - \mathbf{c}_B^T A_B^{-1} A_N & 0 & 1 & -\mathbf{c}_B^T A_B^{-1}\mathbf{b} \end{array}\right]$$

We do not need to compute all elements of $\bar{A}$

Example

$$\begin{array}{rrcrcl} \max & x_1 & + & x_2 & & \\ & -x_1 & + & x_2 & \leq & 1 \\ & x_1 & & & \leq & 3 \\ & & & x_2 & \leq & 2 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

$$\begin{array}{rrcrcrcrcl} \max & x_1 & + & x_2 & & & & & & \\ & -x_1 & + & x_2 & + & x_3 & & & = & 1 \\ & x_1 & & & & & + & x_4 & = & 3 \\ & & & x_2 & & & & + \ x_5 & = & 2 \\ & & & & & x_1, x_2, x_3, x_4, x_5 & & & \geq & 0 \end{array}$$

Initial tableau

$x1$	$x2$	$x3$	$x4$	$x5$	$-z$	b
-1	1	1	0	0	0	1
1	0	0	1	0	0	3
0	1	0	0	1	0	2
1	1	0	0	0	1	0

After two iterations

$x1$	$x2$	$x3$	$x4$	$x5$	$-z$	b
1	0	-1	0	1	0	1
0	1	0	0	1	0	2
0	0	1	1	-1	0	2
0	0	1	0	-2	1	3

Basic variables x_1, x_2, x_4. Non basic: x_3, x_5. From the **initial tableau**:

$$A_B = \begin{bmatrix} -1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad A_N = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \quad x_B = \begin{bmatrix} x_1 \\ x_2 \\ x_4 \end{bmatrix} \quad x_N = \begin{bmatrix} x_3 \\ x_5 \end{bmatrix}$$

$$c_B^T = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \quad c_N^T = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

- **Entering variable**:
  in standard we look at tableau, in revised we need to compute: $\mathbf{c}_N^T - \mathbf{c}_B^T A_B^{-1} A_N$
  1. find $\mathbf{y}^T = \mathbf{c}_B^T A_B^{-1}$ (by solving $\mathbf{y}^T A_B = \mathbf{c}_B^T$, the latter can be done more efficiently)
  2. calculate $\mathbf{c}_N^T - \mathbf{y}^T A_N$

Step 1:

$$\begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} \begin{bmatrix} -1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \qquad \mathbf{y}^T A_B = \mathbf{c}_B^T$$

$$\begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 2 \end{bmatrix} \qquad \mathbf{c}_B^T A_B^{-1} = \mathbf{y}^T$$

Step 2:

$$\begin{bmatrix} 0 & 0 \end{bmatrix} - \begin{bmatrix} -1 & 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & -2 \end{bmatrix} \qquad \mathbf{c}_N^T - \mathbf{y}^T A_N$$

(Note that they can be computed individually: $c_j - \mathbf{y}^T \mathbf{a}_j > 0$)
Let's take the first we encounter x_3

- **Leaving variable**
  we increase variable by largest feasible amount θ

$$\begin{aligned} &\text{R1: } x_1 - x_3 + x_5 = 1 \\ &\text{R2: } x_2 + x_5 = 2 \\ &\text{R3: } -x_3 + x_4 - x_5 = 2 \end{aligned} \qquad \begin{aligned} x_1 &= 1 + x_3 \geq 0 \\ x_2 &= 2 \geq 0 \\ x_4 &= 2 - x_3 \geq 0 \end{aligned}$$

$\mathbf{x}_B = \mathbf{x}_B^* - A_B^{-1} A_N \mathbf{x}_N$
$\mathbf{x}_B = \mathbf{x}_B^* - \mathbf{d}\theta$

$\mathbf{d}$ is the column of $A_B^{-1} A_N$ that corresponds to the entering variable, ie, $\mathbf{d} = A_B^{-1}\mathbf{a}$ where $\mathbf{a}$ is the entering column

3. Find θ such that $\mathbf{x}_B$ stays positive:
Find $\mathbf{d} = A_B^{-1}\mathbf{a}$ (by solving $A_B\mathbf{d} = \mathbf{a}$)

Step 3:

$$\begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \implies \mathbf{d} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \implies \mathbf{x}_B = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} - \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \theta \geq 0$$

$2 - \theta \geq 0 \implies \theta \leq 2 \rightsquigarrow x_4$ leaves

- So far we have done computations, but now we save the pivoting update. The update of A_B is done by replacing the leaving column by the entering column

$$x_B^* = \begin{bmatrix} x_1 - d_1\theta \\ x_2 - d_2\theta \\ \theta \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix} \qquad A_B = \begin{bmatrix} -1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

- Many implementations depending on how $\mathbf{y}^T A_B = \mathbf{c}_B^T$ and $A_B\mathbf{d} = \mathbf{a}$ are solved. They are in fact solved from scratch.

- many operations saved especially if many variables!

- special ways to call the matrix A from memory

- better control over numerical issues since A_B^{-1} can be recomputed.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Solving the two Systems of Equations

$A_B\mathbf{x} = \mathbf{b}$ solved without computing A_B^{-1}
(costly and likely to introduce numerical inaccuracy)

Recall how the inverse is computed:

For a 2×2 matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

the matrix inverse is

$$A^{-1} = \frac{1}{|A|} \begin{bmatrix} d & -c \\ -b & a \end{bmatrix}^T = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

For a 3×3 matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

the matrix inverse is

$$A^{-1} = \frac{1}{|\mathbf{A}|} \begin{bmatrix} + \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} & - \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} & + \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} \\ - \begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} & + \begin{vmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{vmatrix} & - \begin{vmatrix} a_{11} & a_{12} \\ a_{31} & a_{32} \end{vmatrix} \\ + \begin{vmatrix} a_{12} & a_{13} \\ a_{22} & a_{23} \end{vmatrix} & - \begin{vmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{vmatrix} & + \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} \end{bmatrix}^T$$

Eta Factorization of the Basis

At the kth iteration of the simplex, let $B_k := A_{B_k}$,
B_k be the matrix with col $\mathbf{p}$ differing from B_{k-1}
Column $\mathbf{p}$ is the column $\mathbf{a}$ appearing in $B_{k-1}\mathbf{d} = \mathbf{a}$ solved in the third step of the revised
Hence:

$$B_k = B_{k-1}E_k$$

E_k is the **eta matrix** differing from identity matrix in only one column, which is set equal to $\mathbf{d}$

$$\begin{bmatrix} -1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} -1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & & -1 \\ & 1 & 0 \\ & & 1 \end{bmatrix}$$

No matter how we solve $\mathbf{y}^T B_{k-1} = \mathbf{c}_B^T$ and $B_{k-1}\mathbf{d} = \mathbf{a}$, their update always relays on $B_k = B_{k-1}E_k$ with E_k available. Plus when initial basis by slack variable $B_0 = I$ and $B_1 = E_1, B_2 = E_1E_2\cdots$:

$$B_k = E_1E_2\ldots E_k \qquad \text{eta factorization}$$

$$((((\mathbf{y}^T E_1)E_2)E_3)\cdots)E_k = \mathbf{c}_B^T, \qquad \mathbf{u}^T E_4 = \mathbf{c}_B^T,\ \mathbf{v}^T E_3 = \mathbf{u}^T,\ \mathbf{w}^T E_2 = \mathbf{v}^T,\ \mathbf{y}^T E_1 = \mathbf{w}^T$$

$$(E_1(E_2\cdots E_k\mathbf{d})) = \mathbf{a}, \qquad E_1\mathbf{u} = \mathbf{a},\ E_2\mathbf{v} = \mathbf{u},\ E_3\mathbf{w} = \mathbf{v},\ E_4\mathbf{d} = \mathbf{w}$$

Exercise

Solve the systems $\mathbf{y}^T E_1 E_2 E_3 E_4 = [1\ 2\ 3]$ and $E_1 E_2 E_3 E_4 \mathbf{d} = [1\ 2\ 3]^T$ with

$$E_1 = \begin{bmatrix} 1 & 3 & 0 \\ 0 & 0.5 & 0 \\ 0 & 4 & 1 \end{bmatrix} \qquad E_2 = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \qquad E_3 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \qquad E_4 = \begin{bmatrix} -0.5 & 0 & 0 \\ 3 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

We use backward transformation and solve the sequence of linear systems:

$$\mathbf{u}^T E_4 = [1\ 2\ 3], \quad \mathbf{v}^T E_3 = \mathbf{u}^T, \quad \mathbf{w}^T E_2 = \mathbf{v}^T, \quad \mathbf{y}^T E_1 = \mathbf{w}^T$$

$$\mathbf{u}^T \begin{bmatrix} -0.5 & 0 & 0 \\ 3 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} = [1, 2, 3]$$

Since the eta matrices have always one 1 in two columns then the solution can be read up easily. From the third column we find $u_3 = 3$. From the second column, we find $u_2 = 2$. Substituting in the first column, we find $-0.5u_1 + 3 \cdot 2 + 1 \cdot 3 = 1$, which yields $u_1 = 16$. The next syestem is:

$$\mathbf{v}^T \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} = [16, 2, 3]$$

From the first column we get $v_1 = 16$, from the second column $v_2 = 2$ from the last column $v_1 + 3v_2 + v_3 = 3$, which yields $v_3 = -19$. The next:

$$\mathbf{w}^T \begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ 4 & 0 & 1 \end{bmatrix} = [16, 2, -19]$$

...

- Solving $\mathbf{y}^T B_k = \mathbf{c}_B^T$ also called backward transformation (BTRAN)
- Solving $B_k \mathbf{d} = \mathbf{a}$ also called forward transformation (FTRAN)

- E_i matrices can be stored by only storing the column and the position
- If sparse columns then can be stored in compact mode, ie only nonzero values and their indices

LU factorization

Worth to consider also the case of $B_0 \neq I$:

$$B_k = B_0 E_1 E_2 \dots E_k \qquad \text{eta factorization}$$

$$((((\mathbf{y}^T B_0)E_1)E_2)\cdots)E_k = \mathbf{c}_B^T$$
$$(B_0(E_1 \cdots E_k \mathbf{d})) = \mathbf{a}$$

We need an LU factorization of B_0

LU Factorization

To solve the system $A\mathbf{x} = \mathbf{b}$ by Gaussian Elimination we put the A matrix in row echelon form by means of elementary row operations. Each row operation corresponds to multiply left and right side by a lower triangular matrix L and a permuation matrix P. Hence, the method:

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ L_1P_1A\mathbf{x} &= L_1P_1\mathbf{b} \\ L_2P_2L_1P_1A\mathbf{x} &= L_2P_2L_1P_1\mathbf{b} \\ &\vdots \\ L_mP_m\ldots L_2P_2L_1P_1A\mathbf{x} &= L_mP_m\ldots L_2P_2L_1P_1\mathbf{b} \end{aligned}$$

thus

$$U = L_mP_m\ldots L_2P_2L_1P_1A \qquad \text{triangular factorization of } A$$

where U is an upper triangular matrix whose entries in the diagonal are ones. (if A is nonsingular such triangularization is unique)

(see numerical example in [Va] sc 8.1)

We can compute the triangular factorization of B_0 before the initial iterations of the simplex:

$$L_m P_m \dots L_2 P_2 L_1 P_1 B_0 = U$$

We can then rewrite U as

$$U = U_m U_{m-1} \dots, U_1$$

Hence, for $B_k = B_0 E_1 E_2 \dots E_k$:

$$L_m P_m \dots L_2 P_2 L_1 P_1 B_k = U_m U_{m-1} \dots U_1 E_1 E_2 \cdots E_k$$

Then $\mathbf{y}^T B_k = \mathbf{c}_B^T$ can be solved by first solving:

$$((((\mathbf{y}^T U_m) U_{m-1}) \cdots) E_k = \mathbf{c}_B^T$$

and then replacing

$$\mathbf{y}^T \text{ by } ((\mathbf{y}^T L_m P_m) \cdots) L_1 P_1$$

$$B_k = \underbrace{(L_m P_m \cdots L_1 P_1)}_{L}{}^{-1} \underbrace{U_m \cdots E_k}_{U}$$

$$\mathbf{y} L^{-1} U = \mathbf{c}$$

$$\mathbf{w} U = \mathbf{c}$$

$$\mathbf{w} = \mathbf{y} L^{-1} \implies \mathbf{y} = L\mathbf{w}$$

LU Factorization

LU decomposition, the matrix form of Gaussian elimination.

$$\mathbf{A} = \mathbf{LU}$$
$$\mathbf{PA} = \mathbf{LU}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

If the diagonal of U is 1 and A is nonsingular, then there exists only one LU factorization

- it can be rewritten as $A_B = LU$, L lower triangular matrix, U upper triangular matrix, ie, LU-factorization

- $LU\mathbf{x} = \mathbf{b}$: setting $\mathbf{y} = U\mathbf{x}$ then
  1. $L\mathbf{y} = \mathbf{b}$ can be solved easily by forward substitution
  2. $U\mathbf{x} = \mathbf{y}$ can be solved easily by backward substitution

More on LP

- Tableau method is unstable: computational errors may accumulate. Revised method has a natural control mechanism: we can recompute A_B^{-1} at any time
- Commercial and freeware solvers differ from the way the systems $\mathbf{y}^T A_B = \mathbf{c}_B^T$ and $A_B \mathbf{d} = \mathbf{a}$ are resolved

Efficient Implementations

- Dual simplex with steepest descent (largest increase)
- Linear Algebra:
  - Dynamic LU-factorization using Markowitz threshold pivoting (Suhl and Suhl, 1990)
  - sparse linear systems: Typically these systems take as input a vector with a very small number of nonzero entries and output a vector with only a few additional nonzeros.
- Presolve, ie problem reductions: removal of redundant constraints, fixed variables, and other extraneous model elements.
- dealing with degeneracy, stalling (long sequences of degenerate pivots), and cycling:
  - bound-shifting (Paula Harris, 1974)
  - Hybrid Pricing (variable selection): start with partial pricing, then switch to devex (approximate steepest-edge, Harris, 1974)
- A model that might have taken a year to solve 10 years ago can now solve in less than 30 seconds (Bixby, 2002).

Interior Point Algorithms

- Ellipsoid method: cannot compete in practice but polynomial time (Khachyian, 1979)
- **Interior point algorithm(s)** (Karmarkar, 1984) competitive with simplex and polynomial in some versions
  - iterate through points interior to the feasibility region
    1. Start at an interior point of the feasible region
    2. Move in a direction that improves the objective function value at the fastest possible rate
    3. Transform the feasible region to place the current point at the center of it
  - because of patents reasons, also known as **barrier algorithm**
  - one single iteration is computationally more intensive than the simplex
  - particularly competitive in presence of many constraints (eg, for $m = 10,000$ may need less than 100 iterations)
  - bad for post-optimality analysis $\rightsquigarrow$ crossover algorithm to convert a sol of barrier method into a basic feasible solutions for the simplex
  - usually fastest for large, difficult models but numerically sensitive.

Methods Used in Practice

A method is a general template that lacks details to be fully defined. An algorithm is a well defined procedure that terminates.

- Simplex method
- Interior-Point methods

- Network Simplex
- Sifting methods
- Lagrangian relaxation
- Decomposition methods:
  - Dantzig Wolfe decomposition
  - Benders decomposition

- First-order methods: Primal-Dual Hybrid Gradient Method (matrix factorization free, good for GPUs)

Computational Bottlenecks

- Bottleneck: Matrix factorization to solve linear equations:
  - simplex: LU-factorization
  - interior point method: Cholesky factorization
- Constraint matrix is extremely sparse in practice while the factorization is much denser. Hence, more memory than just storing the LP instance itself. This may lead to out-of-memory errors when solving large instances.
  Factorization may still take a long time for large instances.
- Highly challenging to take advantage of modern computing architectures, such as GPUs or distributed systems, to solve the linear equations in these two methods.
- For simplex, multiple cores are generally employed only in the “pricing” step that selects variables entering or leaving the basis, but the speedups are generally minimal after two or three cores
- For interior point methods, multiple threads can be used for factorization but speedups are generally negligible after at most six shared-memory cores

How Large Problems Can We Solve?

Very large model

	Rows	Columns	Nonzeros
Original size	5034171	7365337	25596099
After presolve	1296075	2910559	10339042

Solution times were as follows:

Very large model—solution times

	Algorithm		
Version	Barrier	Dual	Primal
CPLEX 5.0	8642.6	350000.0	71039.7
CPLEX 7.1	5642.6	6413.1	1880.0

Source: Bixby, 2002

Midway Evaluation – Action Plan

https://PollEv.com/surveys/Y9ZAhgIUlEpmi2E2ux7qu/respond

Lectures:

- classify reading material
- slides, noted
- code after the lectures is in the solutions of the exercise sheets

Exercise classes:

- exercise classes change: mix of self work + worked out tasks by TA
- exam sets at the end of the course
- best time for solutions to appear?

General:

- on the workload, removed a week
- material earlier available, published all remaining classes

On the web page?

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Discrete Optimization

- Often we need to deal with integral inseparable quantities
- Sometimes rounding can go
- Other times rounding not feasible: eg, presence of a bus on a line is 0.3...

Integer Linear Programming

Linear Objective
Linear Constraints
but! integer variables

$$\begin{aligned} \max\ & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

Linear Programming (LP)

$$\begin{aligned} \max\ & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{x}\ \text{integer} \end{aligned}$$

Integer Linear Programming (ILP)

$$\begin{aligned} \max\ & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \in \{0,1\}^n \end{aligned}$$

Binary Integer Program (BIP)
0/1 Integer Programming

$$\begin{aligned} \max\ & \mathbf{c}^T\mathbf{x} + \mathbf{h}^T\mathbf{y} \\ & A\mathbf{x} + G\mathbf{y} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{y} \geq \mathbf{0} \\ & \mathbf{y}\ \text{integer} \end{aligned}$$

Mixed Integer Linear Programming (MILP)

$$\begin{aligned} \max\ & f(\mathbf{x}) \\ & g(\mathbf{x}) \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

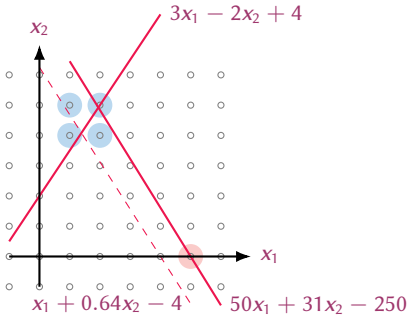
Non-linear Programming (NLP)

Recall:

- $\mathbb{Z}$ set of integers
- $\mathbb{Z}^+$ set of positive integer
- $\mathbb{Z}_0^+$ set of nonnegative integers ($\{0\} \cup \mathbb{Z}^+$)
- $\mathbb{N}_0$ set of natural numbers, ie, nonnegative integers $\{0, 1, 2, 3, 4, \ldots\}$

Rounding

$$\begin{aligned} \max\ & 100x_1 + 64x_2 \\ & 50x_1 + 31x_2 \leq 250 \\ & 3x_1 - 2x_2 \geq -4 \\ & x_1, x_2 \in \mathbb{Z}_0^+ \end{aligned}$$



Note: rounding does not help in the example above!

LP optimum $(376/193, 950/193)$
IP optimum $(5, 0)$

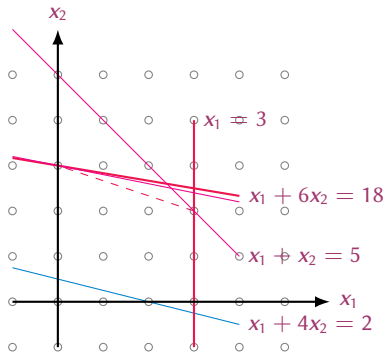
$\rightsquigarrow$ feasible region convex but not continuous: Now the optimum can be on the border (vertices) but also **internal**.

Possible way: solve the **relaxed** problem.

- If solution is **integer**, done.
- If solution is **rational** (never irrational) try rounding to the nearest integers (but may exit feasibility region)
  if in $\mathbb{R}^2$ then 2^2 possible roundings (up or down)
  if in $\mathbb{R}^n$ then 2^n possible roundings (up or down)

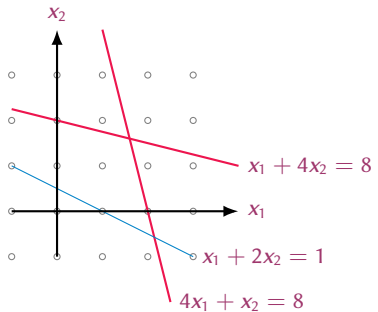
Cutting Planes

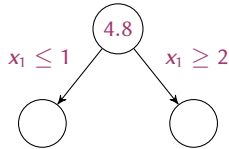
$$
\begin{array}{rrcl}
\max & x_1 + 4x_2 & & \\
& x_1 + 6x_2 & \leq & 18 \\
& x_1 & \leq & 3 \\
& x_1, x_2 & \geq & 0 \\
& x_1, x_2 \text{ integers} & &
\end{array}
$$

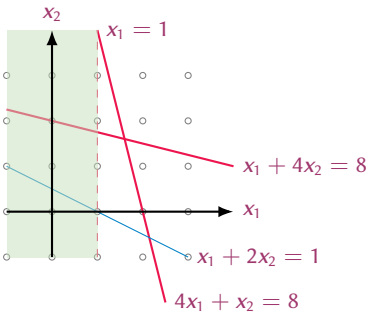


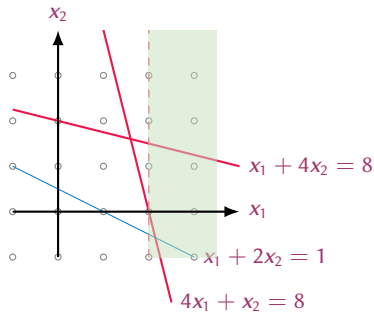
Branch and Bound

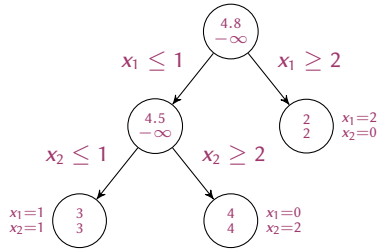
$$
\begin{aligned}
\max \quad & x_1 + 2x_2 \\
& x_1 + 4x_2 \leq 8 \\
& 4x_1 + x_2 \leq 8 \\
& x_1, x_2 \geq 0, \text{integer}
\end{aligned}
$$

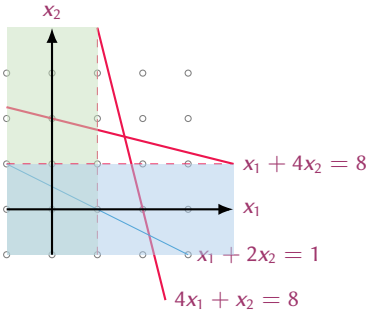


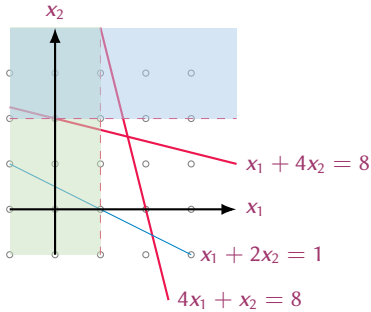

4.8
$x_1 \leq 1$
$x_1 \geq 2$


x_2
$x_1 = 1$
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$


x_2
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$


4.8
−∞
$x_1 \leq 1$
$x_1 \geq 2$
4.5
−∞
2
2
x_1=2
x_2=0
$x_2 \leq 1$
$x_2 \geq 2$
x_1=1
x_2=1
3
3
4
4
x_1=0
x_2=2


x_2
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$


x_2
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Mathematical Programming: Modeling

- Find out exactly what the decision maker needs to know:
  - which investment?
  - which product mix?
  - which job j should a person i do?
- Define **Decision Variables** of suitable type (continuous, integer valued, binary) corresponding to the needs and **Known Parameters** corresponding to given data.
- Formulate **Objective Function** computing the benefit/cost
- Formulate mathematical **Constraints** indicating the interplay between the different variables.

How to “build” a constraint

- Formulate relationship between the variables in plain words
- Then formulate your sentences using logical connectives **and, or, not, implies**
- Finally convert the logical statement to a mathematical constraint.

Example

- “The power plant must not work in both of two neighbouring time periods”
- on/off is modelled using **binary** integer variables
- $x_i = 1$ or $x_i = 0$
- $\neg(x_i = 1 \wedge x_{i+1} = 1) \equiv \neg(x_i = 1) \vee \neg(x_{i+1} = 1)$
- $(1 - x_i) + (1 - x_{i+1}) \geq 1 \implies x_i + x_{i+1} \leq 1$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

The Assignment Problem

Problem

Common application: **assignees** are being assigned to perform **tasks**.

Suppose we have n persons and n jobs
Each person has a certain proficiency at each job.

Formulate a mathematical model that can be used to find an assignment that maximizes the total proficiency.

The Assignment Problem

Model

Decision Variables:

$$x_{ij} = \begin{cases} 1 \text{ if person } i \text{ is assigned job } j \\ 0 \text{ otherwise,} \end{cases} \quad \text{for } i, j = 1, 2, \ldots, n$$

Objective Function:

$$\max \quad \sum_{i=1}^{n} \sum_{j=1}^{n} \rho_{ij} x_{ij}$$

where ρ_{ij} is person i's proficiency at job j

The Assignment Problem

Model

Constraints:

Each person is assigned one job:

$$\sum_{j=1}^{n} x_{ij} = 1 \text{ for all } i$$

e.g. for person 1 we get $x_{11} + x_{12} + x_{13} + \cdots + x_{1n} = 1$

Each job is assigned to one person:

$$\sum_{i=1}^{n} x_{ij} = 1 \text{ for all } j$$

e.g. for job 1 we get $x_{11} + x_{21} + x_{31} + \cdots + x_{n1} = 1$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

The Knapsack Problem

Problem ..

Input: Given a set of n items, each with a value v_i and weight w_i $(i = 1, \ldots, n)$

Task: determine the number of each item to include in a collection so that the total weight is less than a given limit, W, and the total value is as large as possible.

The "knapsack" name derives from the problem faced by someone who is constrained by a fixed-size knapsack and must fill it with the most useful items.

Assuming we can take at least one of any item and that $\sum_i w_i > W$, formulate a mathematical model to determine which items give the largest value.

Without loss of generality we can consider the 0/1 knapsack problem. Why?

Model used, eg, in capital budgeting, project selection, etc.

The 0/1 Knapsack Problem

Decision Variables:

$$x_i = \begin{cases} 1 \text{ if item } i \text{ is taken} \\ 0 \text{ otherwise,} \end{cases} \quad \text{for } i = 1, 2 \ldots, n$$

Objective Function:

$$\max \quad \sum_{i=1}^{n} v_i x_i$$

Constraints:
Knapsack capacity restriction:

$$\sum_{i=1}^{n} w_i x_i \leq W$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Set Covering

Problem

Given: a set of regions, a set of possible construction locations for emergency centers, regions that can be served in less than 8 minutes, cost of installing an emergency center in each location.

Task: decide where to install a set of emergency centers such that the total cost is minimized and all regions are safely served

As Combinatorial Optimization Problem (COP): $M = \{1, \ldots, m\}$ regions, $N = \{1, \ldots, n\}$ centers, $S_j \subseteq M$ regions serviced by $j \in N$ in 8 min. Example:
regions: $M = \{1, \ldots, 5\}$
centers: $N = \{1, \ldots, 6\}$
cost of centers: $c_j = 1 \quad \forall j = 1, \ldots, 6$
coverages: $S_1 = (1, 2), S_2 = (1, 3, 5), S_3 = (2, 4, 5), S_4 = (3), S_5 = (1), S_6 = (4, 5)$

$$\min_{T \subseteq N} \left\{ \sum_{j \in T} c_j \;\middle|\; \bigcup_{j \in T} S_j = M \right\}$$

Example

- regions: $M = \{1, \ldots, 5\}$
  centers: $N = \{1, \ldots, 6\}$
  cost of centers: $c_j = 1 \quad \forall j = 1, \ldots, 6$
  coverages: $S_1 = (1, 2), S_2 = (1, 3, 5), S_3 = (2, 4, 5), S_4 = (3), S_5 = (1), S_6 = (4, 5)$

-

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 \\ S_1 & S_2 & S_3 & S_4 & S_5 & S_6 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

As a BIP:

Variables:
$\mathbf{x} \in \mathbb{B}^n$, $x_j = 1$ if center j is selected, 0 otherwise

Objective:

$$\min \sum_{j=1}^{n} c_j x_j$$

Constraints:

- incidence matrix: $a_{ij} = \begin{cases} 1 \\ 0 \end{cases}$
- $\sum_{j=1}^{n} a_{ij} x_j \geq 1$ for all $i = 1..m$

Set covering
cover each of M at least once

1. min, $\geq$
2. all RHS terms are 1
3. all matrix elements are 1

$$\begin{aligned} \min \quad & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} \geq \mathbf{1} \\ & \mathbf{x} \in \mathbb{B}^n \end{aligned}$$

Set packing
cover as many of M without overlap

1. max, $\leq$
2. all RHS terms are 1
3. all matrix elements are 1

$$\begin{aligned} \max \quad & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} \leq \mathbf{1} \\ & \mathbf{x} \in \mathbb{B}^n \end{aligned}$$

Set partitioning
cover exactly once each element of M

1. max or min, $=$
2. all RHS terms are 1
3. all matrix elements are 1

$$\begin{aligned} \max \quad & \mathbf{c}^T\mathbf{x} \\ & A\mathbf{x} = \mathbf{1} \\ & \mathbf{x} \in \mathbb{B}^n \end{aligned}$$

Generalization: $RHS \geq 1$
Application examples:

- Aircrew scheduling: M: legs to cover, N: rosters
- Vehicle routing: M: customers, N: routes

A good written example of how to present a model: Notation + Formulation + Explanation

2.1. Notation

Let N be the set of operational flight legs and K the set of aircraft types. Denote by n^k the number of available aircraft of type $k \in K$. Define Ω^k, indexed by p, as the set of feasible schedules for aircraft of type $k \in K$ and let index $p = 0$ denote the empty schedule for an aircraft. Next associate with each schedule $p \in \Omega^k$ the value c_p^k denoting the anticipated profit if this schedule is assigned to an aircraft of type $k \in K$ and a_{ip}^k a binary constant equal to 1 if this schedule covers flight leg $i \in N$ and 0 otherwise. Furthermore, let S be the set of stations and $S^k \subseteq S$ the subset having the facilities to serve aircraft of type $k \in K$. Then, define o_{sp}^k and d_{sp}^k to equal to 1 if schedule p, $p \in \Omega^k$, starts and ends respectively at station s, $s \in S^k$, and 0 otherwise.

Denote by θ_p^k, $p \in \Omega^k \backslash \{0\}$, $k \in K$, the binary decision variable which takes the value 1 if schedule p is assigned to an aircraft of type k, and 0 otherwise. Finally, let θ_0^k, $k \in K$, be a nonnegative integer variable which gives the number of unused aircraft of type k.

2.2. Formulation

Using these definitions, the DARSP can be formulated as:

$$\text{Maximize} \sum_{k \in K} \sum_{p \in \Omega^k} c_p^k \theta_p^k \tag{1}$$

subject to:

$$\sum_{k \in K} \sum_{p \in \Omega^k} a_{ip}^k \theta_p^k = 1 \quad \forall i \in N, \tag{2}$$

$$\sum_{p \in \Omega^k} (d_{sp}^k - o_{sp}^k) \theta_p^k = 0 \quad \forall k \in K, \forall s \in S^k, \tag{3}$$

$$\sum_{p \in \Omega^k} \theta_p^k = n^k \quad \forall k \in K, \tag{4}$$

$$\theta_p^k \geq 0 \quad \forall k \in K, \forall p \in \Omega^k, \tag{5}$$

$$\theta_p^k \text{ integer} \quad \forall k \in K, \forall p \in \Omega^k. \tag{6}$$

The objective function (1) states that we wish to maximize the total anticipated profit. Constraints (2) require that each operational flight leg be covered exactly once. Constraints (3) correspond to the flow conservation constraints at the beginning and the end of the day at each station and for each aircraft type. Constraints (4) limit the number of aircraft of type $k \in K$ that can be used to the number available. Finally, constraints (5) and (6) state that the decision variables are nonnegative integers. This model is a Set Partitioning problem with additional constraints.

[from G. Desaulniers, J. Desrosiers, Y. Dumas, M.M. Solomon and F. Soumis. Daily Aircraft Routing and Scheduling. Management Science, 1997, 43(6), 841-855]

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Matching

Definition (Matching Theory Terminology)

Matching: set of pairwise non adjacent edges

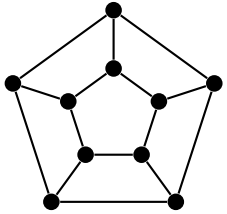
Covered (vertex): a vertex is covered by a matching M if it is incident to an edge in M

Perfect (matching): if M covers each vertex in G

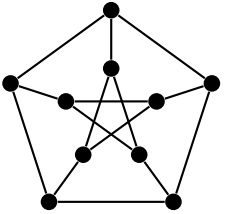
Maximal (matching): if M cannot be extended any further

Maximum (matching): if M covers as many vertices as possible

Matchable (graph): if the graph G has a perfect matching



Pentagonal prism graph



Petersen graph

$$\max \sum_{e \in E} w_e x_e$$

$$\sum_{e \in E : v \in e} x_e \leq 1 \qquad \forall v \in V$$

$$x_e \in \{0, 1\} \qquad \forall e \in E$$

Special case: bipartite matching $\equiv$ assignment problems

Vertex Cover

Select a subset $S \subseteq V$ such that each edge has at least one end vertex in S.

$$\begin{aligned} \min \quad & \sum_{v \in V} x_v & & \\ & x_v + x_u \geq 1 & & \forall u, v \in V, uv \in E \\ & x_v \in \{0,1\} & & \forall v \in V \end{aligned}$$

Approximation algorithm: set S derived from the LP solution in this way:

$$S_{LP} = \{v \in V : x_v^* \geq 1/2\}$$

(it is a cover since $x_v^* + x_u^* \geq 1$ implies $x_v^* \geq 1/2$ or $x_u^* \geq 1/2$)

Proposition

The LP rounding approximation algorithm gives a 2-approximation: $|S_{LP}| \leq 2|S_{OPT}|$
(at most as bad as twice the optimal solution)

Proof: Let $\bar{x}$ be opt to IP. Then $\sum x_v^* \leq \sum \bar{x}_v$.
$|S_{LP}| = \sum_{v \in S_{LP}} 1 \leq \sum_{v \in V} 2x_v^*$ since $x_v^* \geq 1/2$ for each $v \in S_{LP}$
$|S_{LP}| \leq 2 \sum_{v \in V} x_v^* \leq 2 \sum_{v \in V} \bar{x}_v = 2|S_{OPT}|$

Maximum Independent Set

Find the largest subset $S \subseteq V$ such that the induced graph has no edges

$$\begin{aligned} \max \quad & \sum_{v \in V} x_v \\ x_v + x_u &\leq 1 && \forall u, v \in V, uv \in E \\ x_v &= \{0,1\} && \forall v \in V \end{aligned}$$

Optimal sol of LP relaxation sets $x_v = 1/2$ for all variables and has value $|V|/2$.

What is the value of an optimal IP solution of a complete graph?

LP relaxation gives an $O(n)$-approximation (almost useless)

Traveling Salesman Problem

- Find the cheapest movement for a drilling, welding, drawing, soldering arm as, for example, in a printed circuit board manufacturing process or car manufacturing process

- n locations, c_{ij} cost of travel

Variables:

$$x_{ij} = \begin{cases} 1 \\ 0 \end{cases}$$

Objective:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$$

Constraints:

- visits

$$\sum_{j:j\neq i} x_{ij} = 1 \qquad \forall i = 1, \ldots, n$$

$$\sum_{i:i\neq j} x_{ij} = 1 \qquad \forall j = 1, \ldots, n$$

And one of the two types of constraints (they are equivalent):

- cut set constraints

$$\sum_{i\in S} \sum_{j\notin S} x_{ij} \geq 1 \qquad \forall S \subset N, S \neq \emptyset$$

- subtour elimination constraints

$$\sum_{i\in S} \sum_{j\in S} x_{ij} \leq |S| - 1 \qquad \forall S \subset N, 2 \leq |S| \leq n-1$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Modeling Tricks

Objective function and/or constraints do not appear to be linear?

- Absolute values
- Minimize the largest function value
- Maximize the smallest function value
- Constraints include variable division
- Constraints are either/or
- A variable must take one of several candidate values

Modeling: Absolute Values

$$\begin{array}{ll} \min & \sum_{i=1}^{n} |f_i(\mathbf{x})| \\ & \mathbf{x} \in \mathbb{R}^q \end{array}$$

$$\begin{array}{lrcll} \min & \sum_{i=1}^{n} z_i & & & \\ \text{s.t.} & z_i & \geq & f_i(\mathbf{x}) & i = 1..n \\ & z_i & \geq & -f_i(\mathbf{x}) & i = 1..n \\ & z_i & \in & \mathbb{R} & i = 1..n \\ & \mathbf{x} & \in & \mathbb{R}^q & \end{array}$$

n additional variables and $2n$ additional constraints.

$$\begin{array}{lrcll} \min & \sum_{i=1}^{n} (z_i^+ + z_i^-) & & & \\ \text{s.t.} & f_i(\mathbf{x}) & = & z_i^+ - z_i^- & i = 1..n \\ & z_i^+, z_i^- & \geq & 0 & i = 1..n \\ & \mathbf{x} & \in & \mathbb{R}^q & \end{array}$$

$2n$ additional variables and n additional constraints.

Modeling: Minimax

Minimize the largest of a number of function values:

$$\min \quad \max\{f_1(\mathbf{x}), \ldots, f_n(\mathbf{x})\}$$

- Introduce an auxiliary variable z:

$$\begin{aligned} \min \quad & z \\ \text{s. t.} \quad & f_1(\mathbf{x}) \leq z \\ & f_2(\mathbf{x}) \leq z \end{aligned}$$

Modeling: Divisions

Constraints include variable division:

- Constraint of the form

$$\frac{a_1x + a_2y + a_3z}{d_1x + d_2y + d_3z} \leq b$$

  - Rearrange:

$$a_1x + a_2y + a_3z \leq b(d_1x + d_2y + d_3z)$$

    which gives:

$$(a_1 - bd_1)x + (a_2 - bd_2)y + (a_3 - bd_3)z \leq 0$$

Modeling: “Either/Or Constraints”

In conventional mathematical models, the solution must satisfy all constraints.
Suppose that your constraints are “either/or”:

$$a_1 x_1 + a_2 x_2 \leq b_1 \qquad \text{or}$$
$$d_1 x_1 + d_2 x_2 \leq b_2$$

Introduce new variable $y \in \{0, 1\}$ and a large number M:

$$a_1 x_1 + a_2 x_2 \leq b_1 + My \qquad \text{if } y = 0 \text{ then this is active}$$
$$d_1 x_1 + d_2 x_2 \leq b_2 + M(1 - y) \qquad \text{if } y = 1 \text{ then this is active}$$

Modeling: “Either/Or Constraints”

Binary integer programming allows to model alternative choices:

- Eg: 2 feasible regions, ie, disjunctive constraints, not possible in LP.
  introduce y auxiliary binary variable and M, a big number:

$$Ax \leq b + My \qquad \text{if } y = 0 \text{ then this is active}$$
$$A'x \leq b' + M(1 - y) \qquad \text{if } y = 1 \text{ then this is active}$$

Modeling: “Either/Or Constraints”

Generally:

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \ldots + a_{1m}x_m &\leq d_1 \\
a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \ldots + a_{2m}x_m &\leq d_2 \\
&\vdots \\
a_{m1}x_1 + a_{N2}x_2 + a_{N3}x_3 + \ldots + a_{Nm}x_m &\leq d_N
\end{aligned}
$$

Exactly K of the N constraints must be satisfied.
Introduce binary variables $y_1, y_2, \ldots, y_N$ and a large number M

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \ldots + a_{1m}x_m &\leq d_1 + My_1 \\
a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \ldots + a_{2m}x_m &\leq d_2 + My_2 \\
&\vdots \\
a_{m1}x_1 + a_{N2}x_2 + a_{N3}x_3 + \ldots + a_{Nm}x_m &\leq d_N + My_N
\end{aligned}
$$

$$
y_1 + y_2 + \ldots y_N = N - K
$$

K of the y-variables are 0, so K constraints must be satisfied

Modeling: “Either/Or Constraints”

At least $K \leq N$ of $\sum_{j=1}^{n} a_{ij}x_j \leq b_i$, $i = 1, \ldots, N$ must be satisfied

introduce y_i, $i = 1, ..., N$ auxiliary binary variables

$$\sum_{j=1}^{n} a_{ij}x_j \leq b_i + My_i, \qquad i = 1..N$$

$$\sum_{i} y_i \leq N - K$$

Modeling: “Possible Constraints Values”

A constraint must take on one of N given values:

$$\begin{aligned} a_1x_1 + a_2x_2 + a_3x_3 + \ldots + a_mx_m &= d_1 \quad \text{or} \\ a_1x_1 + a_2x_2 + a_3x_3 + \ldots + a_mx_m &= d_2 \quad \text{or} \\ &\vdots \\ a_1x_1 + a_2x_2 + a_3x_3 + \ldots + a_mx_m &= d_N \end{aligned}$$

Introduce binary variables $y_1, y_2, \ldots, y_N$:

$$a_1x_1 + a_2x_2 + a_3x_3 + \ldots + a_mx_m = d_1y_1 + d_2y_2 + \ldots d_Ny_N$$

$$y_1 + y_2 + \ldots y_N = 1$$

Resume

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Uncapacited Facility Location (UFL)

Given:

- depots $N = \{1, \dots, n\}$
- clients $M = \{1, \dots, m\}$
- f_j fixed cost to use depot j
- transport cost for all orders c_{ij}

Task: Which depots to open and which depots serve which client

Variables: $y_j = \begin{cases} 1 & \text{if depot opened} \\ 0 & \text{otherwise} \end{cases}$, x_{ij} fraction of demand of i satisfied by j

Objective:

$$\min \sum_{i \in M} \sum_{j \in N} c_{ij} x_{ij} + \sum_{j \in N} f_j y_j$$

Constraints:

$$\sum_{j=1}^{n} x_{ij} = 1 \qquad \forall i = 1, \dots, m$$

$$\sum_{i \in M} x_{ij} \leq m y_j \qquad \forall j \in N$$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Good and Ideal Formulations

Definition (Formulation)

A polyhedron $P \subseteq \mathbb{R}^{n+p}$ is a **formulation** for a set $X \subseteq \mathbb{Z}^n \times \mathbb{R}^p$ if and only if $X = P \cap (\mathbb{Z}^n \times \mathbb{R}^p)$

That is, if it does not leave out any of the solutions of the feasible region X.

There are **infinitely many** formulations

Definition (Convex Hull)

Given a set $X \subseteq \mathbb{Z}^n$ the **convex hull** of X is defined as:

$$\text{conv}(X) = \Big\{ \mathbf{x} : \mathbf{x} = \sum_{i=1}^{t} \lambda_i \mathbf{x}^i, \qquad \sum_{i=1}^{t} \lambda_i = 1, \qquad \lambda_i \geq 0, \qquad \text{for } i = 1, \ldots, t,$$
$$\text{for all finite subsets } \{\mathbf{x}^1, \ldots, \mathbf{x}^t\} \text{ of } X \Big\}$$

Proposition

$conv(X)$ is a polyhedron (ie, representable as $A\mathbf{x} \leq \mathbf{b}$)

Proposition

Extreme points of $conv(X)$ all lie in X

Hence:

$$\max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in X\} \equiv \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in \text{conv}(X)\}$$

However it might require exponential number of inequalities to describe $\text{conv}(X)$
What makes a formulation better than another?

$$X \subseteq \text{conv}(X) \subseteq P_2 \subset P_1$$

P_2 is better than P_1

Definition

Given a set $X \subseteq \mathbb{R}^n$ and two formulations P_1 and P_2 for X, P_2 is a better formulation than P_1 if $P_2 \subset P_1$

Example

$P_1 =$ UFL with $\sum_{i \in M} x_{ij} \leq m y_j \quad \forall j \in N$
$P_2 =$ UFL with $x_{ij} \leq y_j \quad \forall i \in M, j \in N$

$$P_2 \subset P_1$$

- $P_2 \subseteq P_1$ because summing $x_{ij} \leq y_j$ over $i \in M$ we obtain $\sum_{i \in M} x_{ij} \leq m y_j$
- $P_2 \subset P_1$ because there exists a point in P_1 but not in P_2:
  Example: $m = 6$, $n = 2$ (ie, 6 clients, 2 depots)

  Point for P_1:

  $$x_{10} = 1, \; x_{20} = 1, \; x_{30} = 1, \; x_{41} = 1, \; x_{51} = 1, \; x_{61} = 1$$

  $$\sum_i x_{i0} \leq 6 y_0 \qquad y_0 = 1/2$$
  $$\sum_i x_{i1} \leq 6 y_1 \qquad y_1 = 1/2$$

  is not a point for P_2 because $x_{10} \leq y_0$ is violated.

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Optimality and Relaxation

$$z = \max\{c(\mathbf{x}) : \mathbf{x} \in X \subseteq \mathbb{Z}^n\}$$

How can we prove that $\mathbf{x}^*$ is optimal?
$\overline{z}$ is UB
$\underline{z}$ is LB
stop when $\overline{z} - \underline{z} \leq \epsilon$



- **Primal bounds** (here lower bounds): every feasible solution gives a primal bound, it may be easy or hard to find, heuristics
- **Dual bounds** (here upper bounds): Relaxations

Optimality gap (SCIP):

- If primal and dual bounds have opposite signs, the gap is "Infinity".
- If primal and dual bounds have the same sign, the gap is

$$\frac{|pb - db|}{\min(|pb|, |db|) + \epsilon}$$

decreases monotonously during the solving process.

Proposition

Given: (IP) $\quad z = \max\{c(\mathbf{x}) : \mathbf{x} \in X \subseteq \mathbb{R}^n\}$

a relaxation of it is: (RP) $\quad z^R = \max\{f(\mathbf{x}) : \mathbf{x} \in T \subseteq \mathbb{R}^n\}$ *if:*

(i) $X \subseteq T$ *or*

(ii) $f(\mathbf{x}) \geq c(\mathbf{x})\ \forall \mathbf{x} \in X$

Also:

$$\max_{\mathbf{x}\in T} f(\mathbf{x}) \geq \left\{ \begin{array}{c} \max_{\mathbf{x}\in T} c(\mathbf{x}) \\ \max_{\mathbf{x}\in X} f(\mathbf{x}) \end{array} \right\} \geq \max_{\mathbf{x}\in X} c(\mathbf{x})$$

- T: candidate solutions;
- $X \subseteq T$ feasible solutions;
- $f(\mathbf{x}) \geq c(\mathbf{x})\ \forall \mathbf{x} \in X$

Relaxations

How to construct relaxations?

1. $IP : \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in P \cap \mathbb{Z}^n\}, \qquad P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$
   $LP : \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in P\}$
   Better formulations ($P_1 \subseteq P_2$) give better bounds

Proposition

(i) *If a relaxation LP is infeasible, the original problem IP is infeasible.*

(ii) *Let $\mathbf{x}^*$ be optimal solution for LP. If $\mathbf{x}^* \in X$ and $f(\mathbf{x}^*) = c(\mathbf{x}^*)$ then $\mathbf{x}^*$ is optimal for IP.*

2. **Combinatorial relaxations** to easy problems that can be solved rapidly
   Eg: TSP to Assignment problem Eg: Symmetric TSP to 1-tree

3. **Lagrangian relaxation**

$$IP: \qquad z = \max\{\mathbf{c}^T\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \in X \subseteq \mathbb{Z}^n\}$$

$$LR: \qquad z(\mathbf{u}) = \max\{\mathbf{c}^T\mathbf{x} + \mathbf{u}(\mathbf{b} - A\mathbf{x}) : \mathbf{x} \in X\}$$

$$z(\mathbf{u}) \geq z \qquad \forall \mathbf{u} \geq \mathbf{0}$$

4. **Duality:**

Definition

Two problems:

$$z = \max\{c(\mathbf{x}) : \mathbf{x} \in X\} \qquad w = \min\{w(\mathbf{u}) : \mathbf{u} \in U\}$$

form a **weak-dual pair** if $c(\mathbf{x}) \leq w(\mathbf{u})$ for all $\mathbf{x} \in X$ and all $\mathbf{u} \in U$.
When $z = w$ they form a **strong-dual pair**

Proposition

$z = \max\{\mathbf{c}^T\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \in \mathbb{Z}^n_+\}$ *and* $w^{LP} = \min\{\mathbf{u}^T\mathbf{b} : A^T\mathbf{u} \geq \mathbf{c}, \mathbf{u} \in \mathbb{R}^m_+\}$
(ie, dual of linear relaxation) form a weak-dual pair.

Proposition

Let IP and D be weak-dual pair:

(i) *If D is unbounded, then IP is infeasible*

(ii) *If* $\mathbf{x}^* \in X$ *and* $\mathbf{u}^* \in U$ *satisfy* $c(\mathbf{x}^*) = w(\mathbf{u}^*)$ *then* $\mathbf{x}^*$ *is optimal for IP and* $\mathbf{u}^*$ *is optimal for D.*

The advantage is that we do not need to solve an LP like in the LP relaxation to have a bound, any feasible dual solution gives a bound.

Examples

Weak pairs:

Matching: $z = \max\{\mathbf{1}^T\mathbf{x} : A\mathbf{x} \leq \mathbf{1}, \mathbf{x} \in \mathbb{Z}_+^m\}$

V. Covering: $w = \min\{\mathbf{1}^T\mathbf{y} : A^T y \geq \mathbf{1}, \mathbf{y} \in \mathbb{Z}_+^n\}$

Proof: consider LP relaxations, then $z \leq z^{LP} = w^{LP} \leq w$.
(strong when graphs are bipartite)

Weak pairs:

S. Packing: $z = \max\{\mathbf{1}^T\mathbf{x} : A\mathbf{x} \leq \mathbf{1}, \mathbf{x} \in \mathbb{Z}_+^n\}$

S. Covering: $w = \min\{\mathbf{1}^T\mathbf{y} : A^T\mathbf{y} \geq \mathbf{1}, \mathbf{y} \in \mathbb{Z}_+^m\}$

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Separation problem

$\max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in X\} \equiv \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in \text{conv}(X)\}$
$X \subseteq \mathbb{Z}^n$, P a polyhedron $P \subseteq \mathbb{R}^n$ and $X = P \cap \mathbb{Z}^n$

Definition (Separation problem for a COP)

Given $\mathbf{x}^* \in P$; is $\mathbf{x}^* \in \text{conv}(X)$? If not find an inequality $\mathbf{ax} \leq \mathbf{b}$ satisfied by all points in X but violated by the point $\mathbf{x}^*$.

(Farkas' lemma states the existence of such an inequality.)

Properties of Easy Problems

Four properties that often go together:

Definition

(i) **Efficient optimization property**: $\exists$ a polynomial algorithm for $\max\{\mathbf{cx} : \mathbf{x} \in X \subseteq \mathbb{R}^n\}$

(ii) **Strong duality property**: $\exists$ strong dual D $\min\{w(\mathbf{u}) : \mathbf{u} \in U\}$ that allows to quickly verify optimality

(iii) **Efficient separation problem**: $\exists$ efficient algorithm for separation problem

(iv) **Efficient convex hull property**: a compact description of the convex hull is available

Example:

If explicit convex hull — strong duality holds
efficient separation property (just description of $\text{conv}(X)$)

Theoretical analysis to prove results about

- strength of certain inequalities that are facet defining
  2 ways
- descriptions of convex hull of some discrete $X \subseteq \mathbb{Z}^*$
  several ways, we see one next

Example

Let

$$X = \{(x, y) \in \mathbb{R}^m_+ \times \mathbb{B}^1 : \sum_{i=1}^{m} x_i \leq my, x_i \leq 1 \text{ for } i = 1, \ldots, m\}$$

$$P = \{(x, y) \in \mathbb{R}^m_+ \times \mathbb{R}^1 : x_i \leq y \text{ for } i = 1, \ldots, m, y \leq 1\}$$

.

Polyhedron P describes $\text{conv}(X)$

Totally Unimodular Matrices

When the LP solution to this problem

$$IP : \max\{\mathbf{c}^T\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \in \mathbb{Z}_+^n\}$$

with all data integer will have integer solution?

$$\left[\begin{array}{c|c|c|c} A_N & A_B & \mathbf{0} & \mathbf{b} \\ \hline \mathbf{c}_N^T & \mathbf{c}_B^T & 1 & 0 \end{array}\right]$$

$$A_B x_B + A_N x_N = b$$

$\mathbf{x}_N = \mathbf{0} \rightsquigarrow A_B\mathbf{x}_B = \mathbf{b}$,
A_B $m \times m$ non singular matrix
$\mathbf{x}_B \geq 0$

Cramer's rule for solving systems of linear equations:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} e \\ f \end{bmatrix}$$

$$x = \frac{\begin{vmatrix} e & b \\ f & d \end{vmatrix}}{\begin{vmatrix} a & b \\ c & d \end{vmatrix}} \qquad y = \frac{\begin{vmatrix} a & e \\ c & f \end{vmatrix}}{\begin{vmatrix} a & b \\ c & d \end{vmatrix}}$$

$$\mathbf{x} = A_B^{-1}\mathbf{b} = \frac{A_B^{adj}\mathbf{b}}{\det(A_B)}$$

Definition

- A square integer matrix B is called **unimodular** (UM) if $\det(B) = \pm 1$
- An integer matrix A is called **totally unimodular** (TUM) if every square, nonsingular submatrix of A is UM

Proposition

- *If A is TUM then all vertices of $R_1(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ are integer if $\mathbf{b}$ is integer*
- *If A is TUM then all vertices of $R_2(A) = \{\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ are integer if $\mathbf{b}$ is integer.*

Proof: if A is TUM then $\left[\begin{array}{c:c} A & I \end{array}\right]$ is TUM
Any square, nonsingular submatrix C of $\left[\begin{array}{c:c} A & I \end{array}\right]$ can be written as

$$C = \left[\begin{array}{c:c} B & 0 \\ \hdashline D & I_k \end{array}\right]$$

where B is square submatrix of A. Hence $\det(C) = \det(B) = \pm 1$

Proposition

The transpose matrix A^T of a TUM matrix A is also TUM.

Theorem (Sufficient condition)

An integer matrix A is TUM if

1. $a_{ij} \in \{0, -1, +1\}$ *for all* i, j
2. *each column contains at most two non-zero coefficients* ($\sum_{i=1}^{m} |a_{ij}| \leq 2$)
3. *if the rows can be partitioned into two sets* I_1, I_2 *such that:*
   - *if a column has 2 entries of same sign, their rows are in different sets*
   - *if a column has 2 entries of different signs, their rows are in the same set*

$$\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \qquad \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} 1 & -1 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Proof: by induction

Basis: one matrix of one element $\{0, +1, -1\}$ is TUM

Induction: let C be of size k.

If C has column with all 0s then it is singular.

If a column with only one 1 then expand on that by induction

If 2 non-zero in each column then

$$\forall j : \sum_{i \in I_1} a_{ij} = \sum_{i \in I_2} a_{ij}$$

but then a linear combination of rows is zero and $\det(C) = 0$

Other matrices with integrality property:

- TUM
- Balanced matrices
- Perfect matrices
- Integer vertices

Defined in terms of forbidden substructures that represent fractionating possibilities.

Proposition

A is always TUM if it comes from

- *node-edge incidence matrix of undirected bipartite graphs (ie, no odd cycles) ($I_1 = U, I_2 = V, B = (U, V, E)$)*
- *node-arc incidence matrix of directed graphs ($I_2 = \emptyset$)*

Eg: Shortest path, max flow, min cost flow, bipartite weighted matching

Summary

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

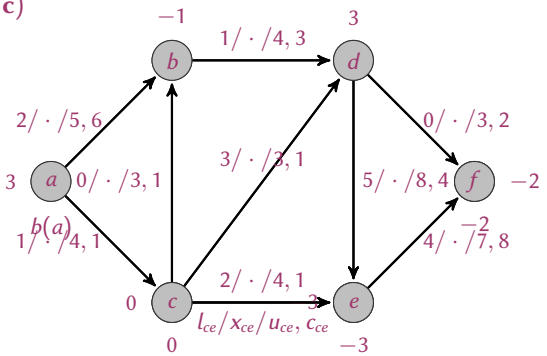
Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Terminology

Network:
- directed graph $D = (V, A)$
- arc, directed link, from tail to head
- lower bound $l_{ij} > 0, \forall ij \in A$, capacity $u_{ij} \geq l_{ij}, \forall ij \in A$
- cost c_{ij}, linear variation (if $ij \notin A$ then $l_{ij} = u_{ij} = 0, c_{ij} = 0$)
- balance vector $b(i)$, $b(i) > 0$ supply node (**source**), $b(i) < 0$ demand node (**sink, tank**), $b(i) = 0$ transhipment node (assumption $\sum_i b(i) = 0$)

$N = (V, A, \mathbf{l}, \mathbf{u}, \mathbf{b}, \mathbf{c})$



Network Flows

Flow $\mathbf{x} : A \to \mathbb{R}$

balance vector of $\mathbf{x}$: $b_{\mathbf{x}}(v) = \sum_{vu \in A} x_{vu} - \sum_{wv \in A} x_{wv}, \forall v \in V$

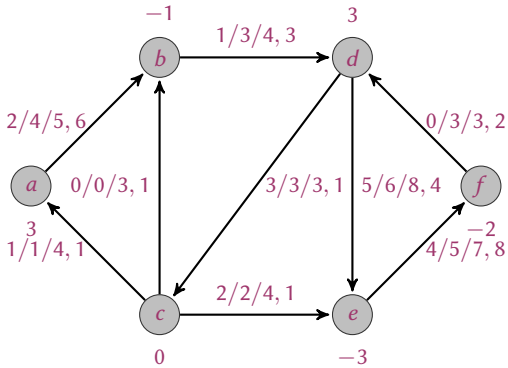
$$b_{\mathbf{x}}(v) \begin{cases} > 0 & \text{source} \\ < 0 & \text{sink/target/tank} \\ = 0 & \text{balanced} \end{cases}$$

(generalizes the concept of path with $b_{\mathbf{x}}(v) = \{0, 1, -1\}$)

feasible	$l_{ij} \leq x_{ij} \leq u_{ij}$, $b_{\mathbf{x}}(i) = b(i)$
cost	$\mathbf{c}^T\mathbf{x} = \sum_{ij \in A} c_{ij} x_{ij}$ (varies linearly with $\mathbf{x}$)

If iji is a 2-cycle and all $l_{ij} = 0$, then at least one of x_{ij} and x_{ji} is zero.

Example



Feasible flow of cost 109

Minimum Cost Network Flows

Find cheapest flow through a network in order to satisfy demands at certain nodes from available supplier nodes.

Variables:

$$x_{ij} \in \mathbb{R}_0^+$$

Objective:

$$\min \sum_{ij \in A} c_{ij} x_{ij}$$

Constraints: mass balance + flow bounds

$$\sum_{j:ij \in A} x_{ij} - \sum_{j:ji \in A} x_{ji} = b(i) \quad \forall i \in V$$

$$l_{ij} \leq x_{ij} \leq u_{ij}$$

$$\begin{aligned} \min \ & \mathbf{c}^T \mathbf{x} \\ & N\mathbf{x} = \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u} \end{aligned}$$

N node arc incidence matrix

if flow of indivisible goods: under the assumption that all parameter values are integer (we can multiply if rational) the LP relaxation solution is integer.

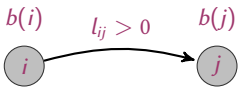
	x_{e_1}	x_{e_2}	$\dots$	x_{ij}	$\dots$	x_{e_m}		
	c_{e_1}	c_{e_2}	$\dots$	c_{ij}	$\dots$	c_{e_m}		
1	1	.	$\dots$	.	$\dots$	.	$=$	b_1
2	.	.	$\dots$	.	$\dots$	.	$=$	b_2
$\vdots$	$\vdots$	$\ddots$					$=$	$\vdots$
i	-1	.	$\dots$	1	$\dots$	.	$=$	b_i
$\vdots$	$\vdots$	$\ddots$					$=$	$\vdots$
j	.	.	$\dots$	-1	$\dots$	.	$=$	b_j
$\vdots$	$\vdots$	$\ddots$					$=$	$\vdots$
n	.	.	$\dots$	.	$\dots$	.	$=$	b_n
e_1	1						$\leq$	u_1
e_2		1					$\leq$	u_2
$\vdots$	$\vdots$	$\ddots$					$\leq$	$\vdots$
(i,j)				1			$\leq$	u_{ij}
$\vdots$	$\vdots$	$\ddots$					$\leq$	$\vdots$
e_m						1	$\leq$	u_m

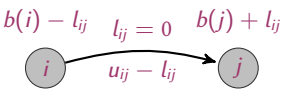
Reductions/Transformations

Lower bounds

Let $N = (V, A, \mathbf{l}, \mathbf{u}, \mathbf{b}, \mathbf{c})$

$$
\begin{aligned}
N' &= (V, A, \mathbf{l}', \mathbf{u}', \mathbf{b}', \mathbf{c}) \\
b'(i) &= b(i) - l_{ij} \\
b'(j) &= b(j) + l_{ij} \\
u'_{ij} &= u_{ij} - l_{ij} \\
l'_{ij} &= 0
\end{aligned}
$$





$\mathbf{c}^T\mathbf{x}$

$$\mathbf{c}^T\mathbf{x}' + \sum_{ij \in A} c_{ij} l_{ij}$$

Undirected arcs

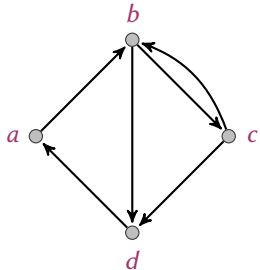


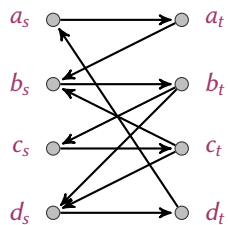


Vertex splitting

If there are bounds and costs of flow passing through vertices where $b(v) = 0$ (used to ensure that a node is visited):

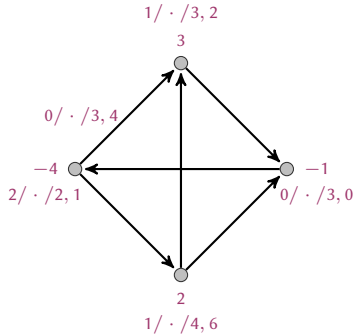
$N = (V, A, \mathbf{l}, \mathbf{u}, \mathbf{c}, \mathbf{l}^*, \mathbf{u}^*, \mathbf{c}^*)$

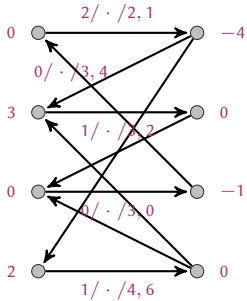




From D to D_{ST} as follows:

$$\begin{aligned} \forall v \in V \quad &\rightsquigarrow v_s, v_t \in V(D_{ST}) \text{ and } v_s v_t \in A(D_{ST}) \\ \forall xy \in A(D) \quad &\rightsquigarrow x_t y_s \in A(D_{ST}) \end{aligned}$$



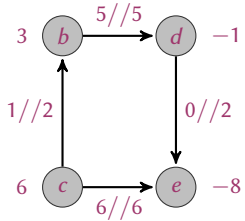


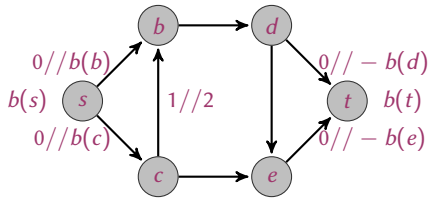
$$\forall v \in V \text{ and } v_s v_t \in A_{ST} \quad \rightsquigarrow h'(v_s v_t) = h^*(v), \quad h^* \in \{l^*, u^*, c^*\}$$
$$\forall xy \in A \text{ and } x_t y_s \in A_{ST} \quad \rightsquigarrow h'(x_t y_s) = h(x, y), \quad h \in \{l, u, c\}$$

If $b(v) = 0$, then $b'(v_s) = b'(v_t) = 0$
If $b(v) < 0$, then $b'(v_s) = 0$ and $b'(v_t) = b(v)$
If $b(v) > 0$, then $b'(v_s) = b(v)$ and $b'(v_t) = 0$

(s,t)-**flow:**

$$b_{\mathbf{x}}(v) = \begin{cases} k & \text{if } v = s \\ -k & \text{if } v = t \\ 0 & \text{otherwise} \end{cases}, \qquad |\mathbf{x}| = |b_{\mathbf{x}}(s)|$$





$$b(s) = \sum_{v:b(v)>0} b(v) = M$$
$$b(t) = \sum_{v:b(v)<0} b(v) = -M$$

$\exists$ feasible flow in $N \iff \exists\ (s,t)$-flow in N_{st} with $|x| = M \iff$ max flow in N_{st} is M

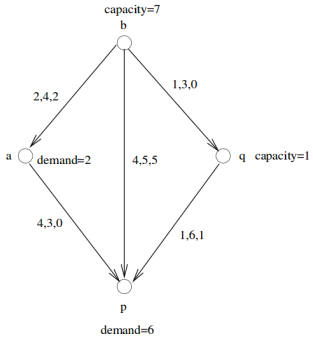
Residual Network

Residual Network $N(\mathbf{x})$**:** given that a flow $\mathbf{x}$ already exists, how much flow excess can be moved in G?

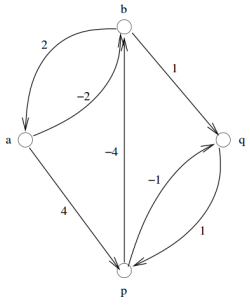
Replace arc $ij \in N$ with arcs:

	residual capacity	cost
ij :	$r_{ij} = u_{ij} - x_{ij}$	c_{ij}
ji :	$r_{ji} = x_{ij}$	$-c_{ij}$

$(N, \mathbf{c}, \mathbf{u}, \mathbf{x})$



$(N(\mathbf{x}), \mathbf{r}, \mathbf{c}')$



Special cases

Shortest path problem path of minimum cost from s to t with costs $\lessgtr 0$

$b(s) = 1, b(t) = -1, b(i) = 0$

if to any other node? $b(s) = (n-1), b(i) = -1, u_{ij} = n-1$

Max flow problem incur no cost but restricted by bounds

steady state flow from s to t

$b(i) = 0 \; \forall i \in V, \qquad c_{ij} = 0 \; \forall ij \in A \qquad ts \in A$

$c_{ts} = -1, \qquad u_{ts} = \infty$

Assignment problem min weighted bipartite matching,

$|V_1| = |V_2|, A \subseteq V_1 \times V_2$

c_{ij}

$b(i) = 1 \; \forall i \in V_1 \qquad b(i) = -1 \; \forall i \in V_2 \qquad u_{ij} = 1 \; \forall ij \in A$

Special cases

Transportation problem/Transhipment distribution of goods, warehouses-costumers

$|V_1| \neq |V_2|, \qquad u_{ij} = \infty$ for all $ij \in A$

$$\min \sum c_{ij} x_{ij}$$

$$\sum_i x_{ij} \geq b_j \qquad \forall j \in V_2$$

$$\sum_j x_{ij} \leq a_i \qquad \forall i \in V_1$$

$$x_{ij} \geq 0$$

if $\sum a_i = \sum b_i$ then $\geq / \leq$ become $=$
if $\sum a_i > \sum b_i$ then add dummy tank nodes
if $\sum a_i < \sum b_i$ then infeasible

Multi-commodity flow problem ship several commodities using the same network, different origin destination pairs separate mass balance constraints, share capacity constraints, min overall flow

$$
\begin{aligned}
\min \ & \textstyle\sum_k \mathbf{c}^k \mathbf{x}^k & & \\
& N\mathbf{x}^k & \geq \mathbf{b}^k & \quad \forall k \\
& \textstyle\sum_k \mathbf{x}^k_{ij} & \leq \mathbf{u}_{ij} & \quad \forall ij \in A \\
& 0 \leq \mathbf{x}^k_{ij} & \leq \mathbf{u}^k_{ij} &
\end{aligned}
$$

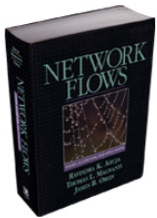
What is the structure of the matrix now? Is the matrix still TUM?

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Ship loading problem

Plenty of applications. See Ahuja Magnanti Orlin, Network Flows, 1993

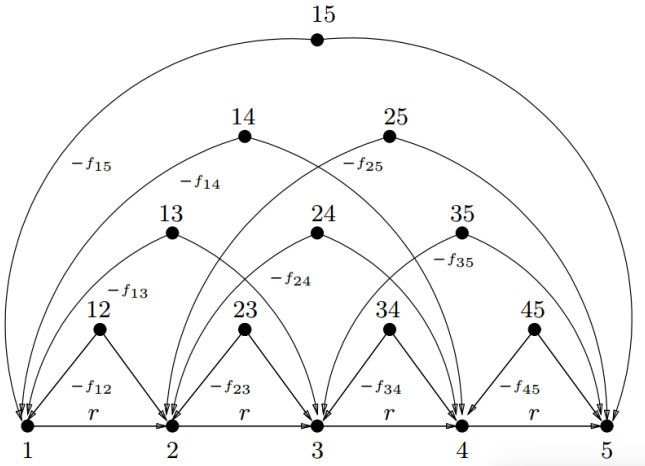


- A cargo company (eg, Maersk) uses a ship with a capacity to carry at most r units of cargo.
- The ship sails on a long route (say from Southampton to Alexandria) with several stops at ports in between.
- At these ports cargo may be unloaded and new cargo loaded.
- At each port there is an amount b_{ij} of cargo which is waiting to be shipped from port i to port $j > i$
- Let f_{ij} denote the income for the company from transporting one unit of cargo from port i to port j.
- The goal is to plan how much cargo to load at each port so as to maximize the total income while never exceeding ship's capacity.

Application Example: Modeling

- n number of stops including the starting port and the terminal port.
- $N = (V, A, \mathbf{l} \equiv \mathbf{0}, \mathbf{u}, \mathbf{c})$ be the network defined as follows:
  - $V = \{v_1, v_2, ..., v_n\} \cup \{v_{ij} : 1 \leq i < j \leq n\}$
  - $A = \{v_1v_2, v_2v_3, ...v_{n-1}v_n\} \cup \{v_{ij}v_i, v_{ij}v_j : 1 \leq i < j \leq n\}$
  - capacity: $u_{v_iv_{i+1}} = r$ for $i = 1, 2, ..., n-1$ and all other arcs have capacity ∞.
  - cost: $c_{v_{ij}v_i} = -f_{ij}$ for $1 \leq i < j \leq n$ and all other arcs have cost zero (including those of the form $v_{ij}v_j$)
  - balance vector: $b(v_{ij}) = b_{ij}$ for $1 \leq i < j \leq n$ and the balance vector of $b(v_i) = -b_{1i} - b_{2i} - ... - b_{i-1,i}$ for $i = 1, 2, ..., n$

Application Example: Modeling



Application Example: Modeling

Claim: the network models the ship loading problem.

- suppose that $t_{12}, t_{13}, \ldots, t_{1n}, t_{23}, \ldots, t_{n-1,n}$ are cargo numbers, where t_{ij} ($\leq b_{ij}$) is the amount of cargo the ship will transport from port i to port j and that the ship is never loaded above capacity.
- total income is

$$I = \sum_{1 \leq i < j \leq n} t_{ij} f_{ij}$$

- Let $\mathbf{x}$ be the flow in N defined as follows:
  - flow on an arc of the form $v_{ij}v_i$ is t_{ij}
  - flow on an arc of the form $v_{ij}v_j$ is $b_{ij} - t_{ij}$
  - flow on an arc of the form $v_i v_{i+1}$, $i = 1, 2, \ldots, n-1$, is the sum of those t_{ab} for which $a \leq i$ and $b \geq i+1$.
- since t_{ij}, $1 \leq i < j \leq n$, are legal cargo numbers then $\mathbf{x}$ is feasible with respect to the balance vector and the capacity restriction.
- the cost of $\mathbf{x}$ is $-I$.

Application Example: Modeling

- Conversely, suppose that $\mathbf{x}$ is a feasible flow in N of cost J.
- we construct a feasible cargo assignment $s_{ij}, 1 \leq i < j \leq n$ as follows:
  - let s_{ij} be the value of $\mathbf{x}$ on the arc $v_{ij}v_i$.
- income $-J$

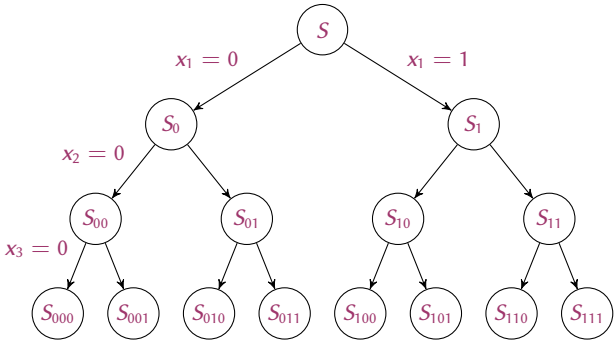
Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Branch and Bound

- Consider the problem $z = \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in S\}$
- Divide and conquer: let $S = S_1 \cup \ldots \cup S_k$ be a decomposition of S into smaller sets, and let $z^k = \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in S_k\}$ for $k = 1, \ldots, K$. Then $z = \max_k z^k$

For instance if $S \subseteq \{0,1\}^3$ the enumeration tree is:

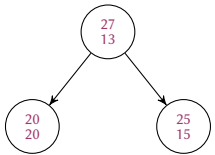


Bounding

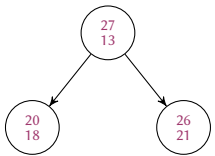
Let's consider a maximization problem

- Let $\overline{z}^k$ be an **upper bound** on z^k (dual bound)
- Let $\underline{z}^k$ be a **lower bound** on z^k (primal bound)
- $(\underline{z}^k \leq z^k \leq \overline{z}^k)$
- $\overline{z} = \max_k \overline{z}^k$ is an **upper bound** on z
- $\underline{z} = \max_k \underline{z}^k$ is a **lower bound** on z

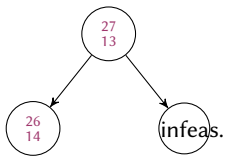
Pruning (Fathoming)



$\overline{z} = 25$
$\underline{z} = 20$
pruned by optimality

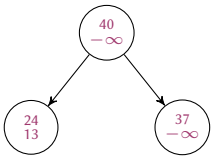


$\overline{z} = 26$
$\underline{z} = 21$
pruned by bounding



$\overline{z} = 26$
$\underline{z} = 14$
pruned by infeasibility

Pruning



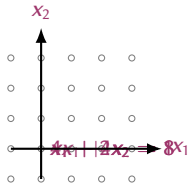
$\overline{z} = 37$

$\underline{z} = 13$

nothing to prune

LP Based Branch & Bound: Example

$$
\begin{aligned}
\max \quad & x_1 + 2x_2 \\
& x_1 + 4x_2 \leq 8 \\
& 4x_1 + x_2 \leq 8 \\
& x_1, x_2 \geq 0, \text{integer}
\end{aligned}
$$



- Solve LP

	x1	x2	x3	x4	-z	b
	1	4	1	0	0	8
	4	1	0	1	0	8
	1	2	0	0	1	0

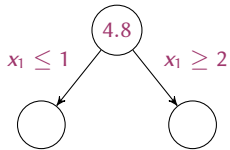
	x1	x2	x3	x4	-z	b
I'=I-II'	0	15/4	1	-1/4	0	6
II'=1/4II	1	1/4	0	1/4	0	2
III'=III-II'	0	7/4	0	-1/4	0	-2

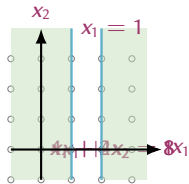
- continuing

	x1	x2	x3	x4	-z	b
I'=4/15I	0	1	4/15	-1/15	0	24/15
II'=II-1/4I'	1	0	-1/15	4/15	0	24/15
III'=III-7/4I'	0	0	-7/15	-3/5	1	-2-14/5

$x_2 = 1 + 3/5 = 1.6$
$x_1 = 8/5$
The optimal solution will not be more than $2 + 14/5 = 4.8$

- Both variables are fractional, we pick one of the two:





- Let's consider first the left branch:

	x1	x2	x3	x4	x5	-z	b
	1	0	0	0	1	0	1
	0	1	4/15	-1/15	0	0	24/15
	1	0	-1/15	4/15	0	0	24/15
	0	0	-7/15	-3/5	0	1	-24/5

	x1	x2	x3	x4	x5	b	-z
I'=I-III	0	0	1/15	-4/15	1	0	-9/15
	0	1	4/15	-1/15	0	0	24/15
	1	0	-1/15	4/15	0	0	24/15
	0	0	-7/15	-3/5	0	1	-24/5

	x1	x2	x3	x4	x5	b	-z
I'=-15/4I	0	0	-1/4	1	-15/4	0	9/4
II'=II-1/4I	0	1	15/60	0	-1/4	0	7/4
III'=III+I	1	0	0	0	1	0	1
	0	0	-37/60	0	-9/4	1	-90/20

always a b term negative after branching:

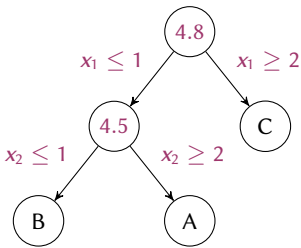
$b_1 = \lfloor \bar{b}_3 \rfloor$

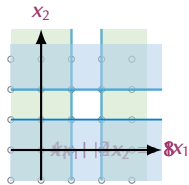
$\bar{b}_1 = \lfloor \bar{b}_3 \rfloor - \bar{b}_3 < 0$

Dual simplex:

$\min_j \{ | \frac{c_j}{a_{ij}} | \ : \ a_{ij} < 0 \}$

- Let's branch again





We have three open problems. Which one do we choose next?
Let's take A.

	x1	x2	x3	x4	x5	x6	b	-z
	0	-1	0	0	0	1	0	-2
	0	0	-1/4	1	-15/4		0	9/4
	0	1	15/60	0	-1/4		0	7/4
	1	0	0	0	1		0	1
	0	0	-37/60	0	-9/4		1	-9/2

	x1	x2	x3	x4	x5	x6	b	-z
III+I	0	0	1/4	0	-1/4	1	0	-1/4
	0	0	-1/4	1	-15/4		0	9/4
	0	1	15/60	0	-1/4		0	7/4
	1	0	0	0	1		0	1
	0	0	-37/60	0	-9/4		1	-9/2

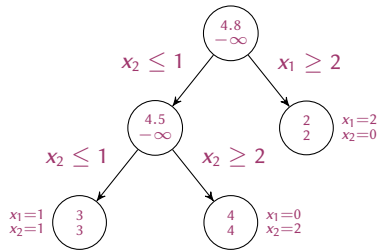
continuing we find:

$x_1 = 0$

$x_2 = 2$

$OPT = 4$

The final tree:

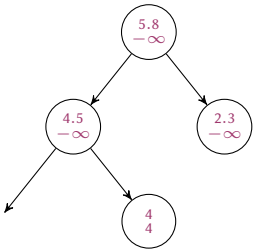


The optimal solution is 4.

Pruning

Pruning:

1. by optimality: $z^k = \max\{c^T x : x \in S^k\}$
2. by bound $\overline{z}^k \leq \underline{z}$
   Example:



3. by infeasibility $S^k = \emptyset$

B&B Components

Bounding:

1. LP relaxation
2. Lagrangian relaxation
3. Combinatorial relaxation
4. Duality

Branching:

$$\begin{aligned} S_1 &= S \cap \{x : x_j \leq \lfloor \bar{x}_j \rfloor\} \\ S_2 &= S \cap \{x : x_j \geq \lceil \bar{x}_j \rceil\} \end{aligned}$$

thus the current optimum is not feasible in S_1 and in S_2.
Which variable to choose?
Eg: Most fractional variable $\arg\max_{j \in C} \min\{f_j, 1 - f_j\}$, $(f_j = x_j - \lfloor x_j \rfloor)$

Choosing Node for Examination from the list of active (or open):

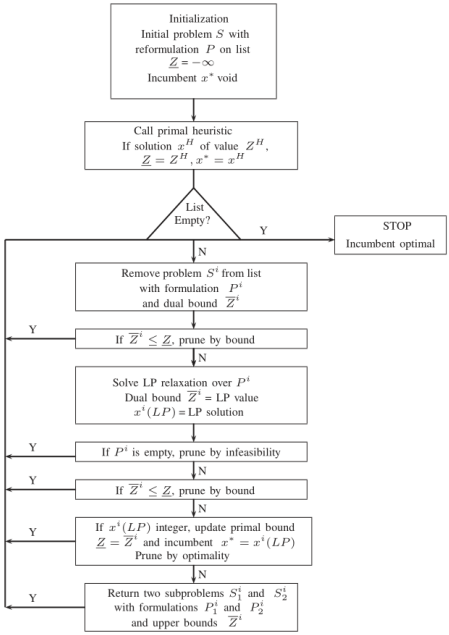
- Depth First Search (a good primal sol. is good for pruning + easier to reoptimize by just adding a new constraint)
- Best Bound First: (eg. largest upper: $\overline{z}^s = \max_k \overline{z}^k$ or largest lower - to die fast)
- Mixed strategies

Reoptimizing: dual simplex

Updating the Incumbent: when new best feasible solution is found:

$$\underline{z} = \max\{\underline{z}, 4\}$$

Store the active nodes: bounds + optimal basis (remember the revised simplex!)



$$Z = \max\{cx \mid x \in S\}$$

[Wolsey, 2021] **Lazy strategy:**

1. selection
2. LP calculation
3. pruning
4. branching
5. queue insertion

open nodes are stored with the bound of their father

```
Branch-and-Bound Algorithm ;
  begin
1.   m := 1; parent[1] := 0; Q := ∅;
     z_OPT := heuristic solution value (possibly +∞);
2.   solve the continuous relaxation min{c^T x : Ax = b , x ≥ 0}, and let
       x* be the optimal solution found;
3.   LB[1] := c^T x* ;
4.   if (x* integer) and (c^T x* < z_OPT) then
       begin
         x_OPT = x*; z_OPT := c^T x*
       end ;
5.   if LB[1] < z_OPT then
       begin
         choose the fractional branching variable x*_h ;
         vbranch[1] := h; value[1] := x*_h;
         Q := {1}
       end ;
6.   while Q ≠ ∅ do /* process the active open nodes*/
       begin
7.       choose a node t ∈ Q, and set Q := Q \ {t} ;
8.       h := vbranch[t]; val := value[t] ;
9.       for child := 1 to 2 do /* generate the children of node t */
           begin
10.          m := m + 1 ;
             if child = 1
               then parent[m] := t
               else parent[m] := −t ;
11.          define problem PL_m associated with node m
               (constraints of PL_t plus x_h ≤ ⌊val⌋ if child = 1,
               or x_h ≥ ⌈val⌉ if child = 2) ;
12.          solve problem PL_m, and let x* be the optimal solution found ;
13.          LB[m] := c^T x*;
14.          if (x* integer) and (c^T x* < z_OPT) then
               begin /* update the optimal solution */
                 x_OPT := x*; z_OPT := c^T x* ;
                 Q := Q \ {j ∈ Q : LB[j] ≥ z_OPT}
               end ;
15.          if LB[m] < z_OPT then
               begin
                 choose the fractional branching variable x*_k ;
                 vbranch[m] := k; value[m] := x*_k ;
                 Q := Q ∪ {m}
               end
           end
       end
  end .
```

Warning: the algorithm on the left is for a minimization problem

[Fischetti, 2019]

Eager strategy:

1. selection
2. branching
3. LP calculation
4. pruning
5. queue insertion

open nodes are stored with their own bounds

Enhancements

- Preprocessor: constraint/problem/structure specific
  tightening bounds
  redundant constraints
  variable fixing: eg: $\max\{\mathbf{c}^T\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}\}$
  fix $x_j = l_j$ if $c_j < 0$ and $a_{ij} > 0$ for all i
  fix $x_j = u_j$ if $c_j > 0$ and $a_{ij} < 0$ for all i
- User defined branching priorities: establish the next variable to branch (not in gurobi)
- Special ordered sets, SOS (or generalized upper bound, GUB)

$$\sum_{j=1}^{k} x_j = 1 \qquad x_j \in \{0, 1\}$$

instead of: $S_0 = S \cap \{\mathbf{x} : x_j = 0\}$ and $S_1 = S \cap \{\mathbf{x} : x_j = 1\}$
$\{\mathbf{x} : x_j = 0\}$ leaves $k - 1$ possibilities
$\{\mathbf{x} : x_j = 1\}$ leaves only 1 possibility
hence tree unbalanced
here: $S_1 = S \cap \{\mathbf{x} : x_{j_i} = 0, i = 1..r\}$ and $S_2 = S \cap \{x : x_{j_i} = 0, i = r + 1, .., k\}$,
$r = \min\{t : \sum_{i=1}^{t} x^*_{j_i} \geq \frac{1}{2}\}$

- Cutoff value: a user-defined primal bound to pass to the system.

- Simplex strategies: simplex is good for reoptimizing but for large models interior points methods may work best.

- Strong branching: extra work to decide more accurately on which variable to branch:
  1. choose a set C of fractional variables
  2. reoptimize for each of them (in case for limited iterations)
  3. $\overline{z}_j^{\downarrow}, \overline{z}_j^{\uparrow}$ (dual bound of down and up branch)

$$j^* = \arg\max_{j \in C} \left(\text{score}(\overline{z}_j^{\downarrow}, \overline{z}_j^{\uparrow}) \right)$$

ie, choose variable with most change in objective function, ie, largest decrease of dual bound, eg, largest decrease of UB for max problem.
$\text{score}(\overline{z}_j^{\downarrow}, \overline{z}_j^{\uparrow})$ is typically $\overline{z}_j^{\downarrow} + \overline{z}_j^{\uparrow}$ or $\max(\overline{z}_j^{\downarrow}, \epsilon) \cdot \max(\overline{z}_j^{\uparrow}, \epsilon)$

There are four common reasons why integer programs can require a significant amount of solution time:

1. There is lack of node throughput due to troublesome linear programming node solves.
2. There is lack of progress in the best integer solution, i.e., the primal bound.
3. There is lack of progress in the best dual bound.
4. There is insufficient node throughput due to numerical instability in the problem data or excessive memory usage.

For 2) or 3) the gap best feasible-dual bound is large:

$$\text{gap} = \frac{|\text{Primal bound} - \text{Dual bound}|}{\text{Primal bound} + \epsilon} \cdot 100$$

- heuristics for finding feasible solutions (generally NP-complete problem)
- find better dual bounds if they are weak: addition of cuts, stronger formulation, **branch and cut**
- Branch and cut: a B&B algorithm with cut generation at all nodes of the tree.
  (instead of reoptimizing, do as much work as possible to tighten)

  Cut pool: stores all cuts centrally
  Store for active node: bounds, basis, pointers to constraints in the cut pool that apply at the node

Relative Optimality Gap

In CPLEX:

$$\text{gap} = \frac{|\text{best dual bound} - \text{best integer}|}{|\text{best integer} + 10^{-11}|}$$

In Gurobi, primal gap:

$$\text{gap} = \frac{|db - pb|}{|pb|}$$

plus if $pb = db = 0$, then the $\text{gap} := 0$.
If $pb = 0$ and $db \neq 0$, then $\text{gap} := \infty$.

Issues...

Problem of non decreasing gap if we go through zero

```
   3186  2520      -666.6217  4096       956.6330      -667.2010  1313338  169.74%
   3226  2560      -666.6205  4097       956.6330      -667.2010  1323797  169.74%
   3266  2600      -666.6201  4095       956.6330      -667.2010  1335602  169.74%
Elapsed real time = 2801.61 sec. (tree size = 77.54 MB, solutions = 2)
*  3324+ 2656                           -125.5775      -667.2010  1363079  431.31%
   3334  2668      -666.5811  4052      -125.5775      -667.2010  1370748  431.31%
   3380  2714      -666.5799  4017      -125.5775      -667.2010  1388391  431.31%
   3422  2756      -666.5791  4011      -125.5775      -667.2010  1403440  431.31%
```

Issue resolved by
MIPLIB standard (Koch et al. MIPLIB 2010, Math Prog Comput):

$$\text{gap}(pb, db) = \begin{cases} 0.0 & pb = db = 0 \\ -- & \text{no feasible solution found,} \\ & \text{problem reported as infeasible,} \quad \text{(minimization prob.)} \\ & \text{or } db \leq 0 \leq pb \\ \frac{pb-db}{\inf\{|z|, z \in [db, pb]\}} \cdot 100 & \text{else} \end{cases}$$

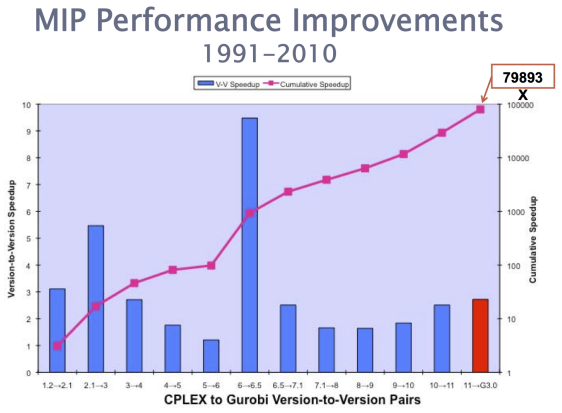
The gap decreases monotonously during the solving process

In SCIP `https://www.scipopt.org/doc/html/FAQ.php#gapdefinition`

$$\text{gap}(pb, db) = \begin{cases} 0.0 & pb = db \\ \infty & pb \cdot db \leq 0 \\ \frac{|pb-db|}{\min\{|pb|, |db|\}} & \text{else}(pb \cdot db > 0) \end{cases}$$

MILP Solvers Breakthroughs

We have seen Fractional Gomory cuts.
The introduction of Mixed Integer Gomory cuts in CPLEX was the major breakthrough of CPLEX 6.5 and produced the version-to-version speed-up given by the blue bars in the chart below



(source: R. Bixby. Mixed-Integer Programming: It works better than you may think. 2010. Slides on the net)

Speedup over the past 25 years:

Hardware 2 000 times
Software 2 000 000 times

In the Exercises

Solve linear programming problems with:

- Simple method tool https://www.zweigmedia.com/simplex/simplex.php
- In Python: scipy.optimize.linprog
- In Python: Python-MIP, pyscipopt, gurobipy

Summary

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
    - Tableaux and Dictionaries
8. Exception Handling

Outline

1. Course Organization
2. Preliminaries
3. Introduction: Operations Research
    - Resource Allocation
    - Duality
4. Introduction
    - Diet Problem
5. Solving LP Problems
    - Fourier-Motzkin method
6. Mathematical Programming
    - Definitions
    - Fundamental Theorem of LP
    - Gaussian Elimination
7. Simplex Method
    - Standard Form
    - Basic Feasible Solutions
    - Algorithm
12. Derivation
    - Lagrangian Duality
13. Dual Simplex
14. Sensitivity Analysis
15. Revised Simplex Method
16. Efficiency Issues
17. Integer Programming
18. Modeling
    - Assignment Problem
    - Knapsack Problem
    - Set Problems
19. More on Modeling
    - Graph Problems
    - Modeling Tricks
20. Formulations
    - Uncapacited Facility Location
    - Alternative Formulations
21. Relaxations

Valid Inequalities

- IP: $z = \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in X\},\ X = \{\mathbf{x} : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \in \mathbb{Z}^n_+\}$
- Proposition: $\text{conv}(X) = \{\mathbf{x} : \tilde{A}\mathbf{x} \leq \tilde{\mathbf{b}}, \mathbf{x} \geq 0\}$ is a polyhedron
- LP: $z = \max\{\mathbf{c}^T\mathbf{x} : \tilde{A}\mathbf{x} \leq \tilde{\mathbf{b}}, \mathbf{x} \geq \mathbf{0}\}$ would be the best formulation
- $\tilde{\mathbf{a}}\mathbf{x} \leq \tilde{\mathbf{b}}$ **facet defining inequalities**
- Key idea: try to approximate the best formulation.

Definition (Valid inequalities)

$\mathbf{a}\mathbf{x} \leq \mathbf{b}$ is a **valid inequality** for $X \subseteq \mathbb{R}^n$ if $\mathbf{a}\mathbf{x} \leq \mathbf{b}\ \forall \mathbf{x} \in X$

Which are useful inequalities? and how can we find them? How can we use them?

Example: Pre-processing

- $X = \{(x, y) : x \leq 999y; \quad 0 \leq x \leq 5, \quad y \in \mathbb{B}^1\}$

  $x \leq 5y$

- $X = \{x \in \mathbb{Z}_+^4 : 13x_1 + 20x_2 + 11x_3 + 6x_4 \geq 72\}$

  $$2x_1 + 2x_2 + x_3 + x_4 \geq \frac{13}{11}x_1 + \frac{20}{11}x_2 + x_3 + \frac{6}{11}x_4 \geq \frac{72}{11} = 6 + \frac{6}{11}$$
  $$2x_1 + 2x_2 + x_3 + x_4 \geq 7$$

- Capacitated facility location:

  $$\sum_{i \in M} x_{ij} \leq b_j y_j \quad \forall j \in N \qquad\qquad x_{ij} \leq b_j y_j$$
  $$\sum_{j \in N} x_{ij} = a_i \quad \forall i \in M \qquad\qquad x_{ij} \leq a_i$$
  $$x_{ij} \geq 0, \quad y_j \in B^n \qquad\qquad x_{ij} \leq \min\{a_i, b_j\} y_j$$

Converting Weak to Strong MIP Formulations

Strong formulations $\equiv$ better, tighter formulations

Detection possible from the log output of a solver.

Possible actions:

1. Add cuts to existing models
   - Combining constraints
   - Using a graph representation (clique cuts)
   - Using a disjunctive approach
2. (Change the model)
3. (Change the algorithm, eg, column generation)

$\rightsquigarrow$ Many found automatically by the solver in pre-solver phase

(Lazy) constraints $\neq$ cuts

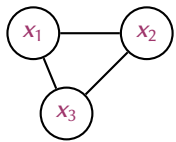
Add cuts to the existing model

$$
\begin{aligned}
\text{maximize } & x_1 + x_2 + x_3 \\
\text{subject to } & x_1 + x_2 \leq 1 \\
& x_2 + x_3 \leq 1 \\
& x_1 + x_3 \leq 1 \\
& x_i \in \{0, 1\} \qquad i = 1, 2, 3
\end{aligned}
$$

Combine and round constraints:

$$
\begin{aligned}
2x_1 + 2x_2 + 2x_3 &\leq 3 \\
x_1 + x_2 + x_3 &\leq \frac{3}{2} \\
x_1 + x_2 + x_3 &\leq 1
\end{aligned}
$$

Create a conflict graph; at most one binary in a clique can be 1



$$x_1 + x_2 + x_3 \leq 1$$

Chvátal-Gomory cuts

- $X \in P \cap \mathbb{Z}^n_+, \quad P = \{\mathbf{x} \in \mathbb{R}^n_+ : A\mathbf{x} \leq \mathbf{b}\}, \quad A \in \mathbb{R}^{m \times n}$
- $\mathbf{u} \in \mathbb{R}^m_+$, $\{\mathbf{a}_1, \mathbf{a}_2, \ldots \mathbf{a}_n\}$ columns of A

CG procedure to construct valid inequalities

1) $\sum_{j=1}^{n} \mathbf{u}^T \mathbf{a}_j x_j \leq \mathbf{u}^T \mathbf{b}$ valid: $\mathbf{u} \geq \mathbf{0}$

2) $\sum_{j=1}^{n} \lfloor \mathbf{u}^T \mathbf{a}_j \rfloor x_j \leq \mathbf{u}^T \mathbf{b}$ valid: $\mathbf{x} \geq \mathbf{0}$ and $\sum \lfloor \mathbf{u}^T \mathbf{a}_j \rfloor x_j \leq \sum \mathbf{u}^T \mathbf{a}_j x_j$

3) $\sum_{j=1}^{n} \lfloor \mathbf{u}^T \mathbf{a}_j \rfloor x_j \leq \lfloor \mathbf{u}^T \mathbf{b} \rfloor$ valid for X since $\mathbf{x} \in \mathbb{Z}^n$

Theorem

by applying this CG procedure a finite number of times every valid inequality for X can be obtained

However not all the constraints generated by $\mathbf{u} \in \mathbb{R}^m_+$ are tightenings.

Cutting Plane Algorithms

- $X \in P \cap \mathbb{Z}^n_+$
- a family of valid inequalities $\mathcal{F} : \mathbf{a}^T\mathbf{x} \leq b, (\mathbf{a}, b) \in \mathcal{F}$ for X
- we do not find them all a priori, only interested in those close to optimum

Cutting Plane Algorithm

Init.: $t = 0, P^0 = P$

Iter. t: Solve $\bar{z}^t = \max\{\mathbf{c}^T\mathbf{x} : \mathbf{x} \in P^t\}$
let $\mathbf{x}^t$ be an optimal solution
if $\mathbf{x}^t \in \mathbb{Z}^n$ stop, $\mathbf{x}^t$ is opt to the IP
if $\mathbf{x}^t \notin \mathbb{Z}^n$ solve separation problem for $\mathbf{x}^t$ and $\mathcal{F}$
if $(\mathbf{a}^t, b^t)$ is found with $\mathbf{a}^t\mathbf{x}^t > b^t$ that cuts off x^t

$$P^{t+1} = P \cap \{\mathbf{x} : \mathbf{a}^i\mathbf{x} \leq b^i, i = 1, \ldots, t\}$$

else stop (P^t is in any case an improved formulation)

Gomory's fractional cutting plane algorithm

Cutting plane algorithm + Chvátal-Gomory cuts

- $\max\{\mathbf{c}^T\mathbf{x} : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0, \mathbf{x} \in \mathbb{Z}^n\}$
- Solve LPR to optimality

$$\left[\begin{array}{c|c|c|c} I & \bar{A}_N = A_B^{-1}A_N & 0 & \bar{b} \\ \hline \bar{c}_B & \bar{c}_N (\leq 0) & 1 & -\bar{d} \end{array}\right]$$

$$x_{B_u} = \bar{b}_u - \sum_{j \in N} \bar{a}_{uj} x_j, \quad u = 1..m$$
$$z = \bar{d} + \sum_{j \in N} \bar{c}_j x_j$$

- If basic optimal solution to LPR is not integer then $\exists$ some row u: $\bar{b}_u \notin \mathbb{Z}^1$.
  The Chvatál-Gomory cut applied to this row is:

$$x_{B_u} + \sum_{j \in N} \lfloor \bar{a}_{uj} \rfloor x_j \leq \lfloor \bar{b}_u \rfloor$$

(B_u is the index in the basis B corresponding to the row u) (cntd)

- Eliminating $x_{B_u} = \bar{b}_u - \sum_{j \in N} \bar{a}_{uj} x_j$ in the CG cut we obtain:

$$\sum_{j \in N} (\underbrace{\bar{a}_{uj} - \lfloor \bar{a}_{uj} \rfloor}_{0 \leq f_{uj} < 1}) x_j \geq \underbrace{\bar{b}_u - \lfloor \bar{b}_u \rfloor}_{0 < f_u < 1}$$

$$\sum_{j \in N} f_{uj} x_j \geq f_u$$

$f_u > 0$ or else u would not be row of fractional solution. It implies that x^* in which $x^*_N = 0$ is cut out!

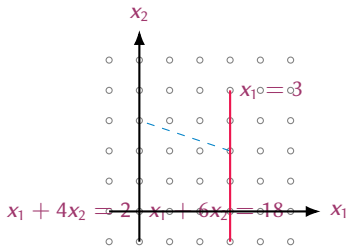
(theoretically it terminates after a finite number of iterations, but in practice not successful.)

Example

$$
\begin{aligned}
\max\ & x_1 + 4x_2 & & \\
& x_1 + 6x_2 & \leq & \ 18 \\
& x_1 & \leq & \ 3 \\
& x_1, x_2 & \geq & \ 0 \\
& x_1, x_2 \text{integer} & &
\end{aligned}
$$

	x1	x2	x3	x4	-z	b
	1	6	1	0	0	18
	1	0	0	1	0	3
	1	4	0	0	1	0

	x1	x2	x3	x4	-z	b
	0	1	1/6	-1/6	0	15/6
	1	0	0	1	0	3
	0	0	-2/3	-1/3	1	-13



$x_2 = 5/2, x_1 = 3$

Optimum, not integer

- We take the first row: | | 0 | 1 | 1/6 | -1/6 | 0 | 15/6 |
- CG cut $\sum_{j \in N} f_{uj} x_j \geq f_u \quad \rightsquigarrow \quad \frac{1}{6} x_3 + \frac{5}{6} x_4 \geq \frac{1}{2}$
- Let's verify that it is a CG cut:

$$\begin{array}{rl} 1/6 & (x_1 + 6x_2 \leq 18) \\ 5/6 & (x_1 \leq 3) \\ \hline & x_1 + x_2 \leq 3 + 5/2 = 5.5 \end{array}$$

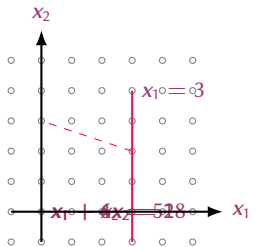
since x_1, x_2 are integer $x_1 + x_2 \leq 5$. And it leaves out x^*.

- Let's see how it looks in the space of the original variables: from the first tableau:

$$x_3 = 18 - 6x_2 - x_1$$
$$x_4 = 3 - x_1$$

$$\frac{1}{6}(18 - 6x_2 - x_1) + \frac{5}{6}(3 - x_1) \geq \frac{1}{2} \qquad \rightsquigarrow \qquad x_1 + x_2 \leq 5$$

- Graphically:



- Let's continue:

	x1	x2	x3	x4	x5	-z	b
	0	0	-1/6	-5/6	1	0	-1/2
	0	1	1/6	-1/6	0	0	5/2
	1	0	0	1	0	0	3
	0	0	-2/3	-1/3	0	1	-13

We need to apply dual-simplex (will always be the case, why?)

ratio rule: $\min\{|\frac{c_j}{a_{ij}}| \ : \ a_{ij} < 0\}$

- After the dual simplex iteration:

	x1	x2	x3	x4	x5	-z	b
	0	0	1/5	1	-6/5	0	3/5
	0	1	1/5	0	-1/5	0	13/5
	1	0	-1/5	0	6/5	0	12/5
	0	0	-3/5	0	-2/5	1	-64/5

- In the space of the original variables:

$$4(18 - x_1 - 6x_2) + (5 - x_1 - x_2) \geq 2$$
$$x_1 + 5x_2 \leq 15$$

- ...

We can choose any of the three rows.

Let's take the third: CG cut: $\frac{4}{5}x_3 + \frac{1}{5}x_5 \geq \frac{2}{5}$

